CFGS Desarrollo de Aplicaciones Multiplataforma

Acceso a Datos

con Python

Módulo profesional 0486 · 126 horas

Segundo curso

Apuntes del módulo — versión 0.1 (borrador)
Curso 2026–2027

Basado en el currículo del módulo para la Comunidad de Castilla y León.

Presentación

Estos apuntes acompañan las clases del módulo *Acceso a Datos* del segundo curso del ciclo de grado superior de Desarrollo de Aplicaciones Multiplataforma. El objetivo del módulo es que aprendas a construir programas que **guardan, recuperan y transforman información** en los distintos lugares donde puede vivir: ficheros, bases de datos relacionales, herramientas de mapeo objeto-relacional, bases de datos de objetos y bases de datos documentales.

A lo largo del curso trabajaremos con **Python**. Es un lenguaje con una sintaxis muy legible, una biblioteca estándar enorme (que ya trae módulos para ficheros, CSV, JSON, XML o SQLite) y conectores para prácticamente cualquier gestor de bases de datos. Como no todo el mundo lo ha usado antes, el Tema 0 repasa los conceptos que necesitarás durante el curso.

Cómo están organizados los apuntes

Cada tema sigue la misma estructura:

- Un recuadro inicial con lo que aprenderás en el tema.
- Explicaciones teóricas acompañadas de **ejemplos de código completos** que puedes copiar y ejecutar.
- Recuadros de **Nota** con aclaraciones y de **Cuidado** con errores habituales.
- **Ejercicios** cortos para practicar cada apartado, con una indicación de dificultad (★ a ★★★).
- Un **Proyecto del tema** final, más largo, para trabajar de forma autónoma.

Relación entre temas y resultados de aprendizaje

Tema	Contenido	RA
0	Repaso de Python	—
1	Manejo de ficheros	RA1
2	Manejo de conectores	RA2
3	Herramientas de mapeo objeto-relacional (ORM)	RA3
4	Bases de datos objeto-relacionales y orientadas a objetos	RA4
5	Bases de datos XML (y documentales)	RA5
6	Programación de componentes de acceso a datos	RA6

El RA6 es transversal: los componentes del Tema 6 se construyen sobre lo aprendido en los temas 1 a 5, por lo que el proyecto final del curso consiste en empaquetar como componentes reutilizables el trabajo de los temas anteriores.

Convenciones

- El código Python aparece así:

```python
def saludar(nombre: str) -> str:
    return f"Hola, {nombre}"   # comentario
```

- Las órdenes que se escriben en la terminal aparecen sobre fondo oscuro:

```
$ python -m venv .venv
$ pip install sqlalchemy
```

- Todos los ejemplos están pensados para **Python 3.12 o superior**.

Software necesario

El Apéndice A explica cómo preparar el entorno de trabajo: Python, un editor (Visual Studio Code o PyCharm), entornos virtuales y Docker para levantar los servidores de bases de datos (PostgreSQL, BaseX y MongoDB) sin instalarlos en el equipo.

Índice

Presentación **i**

0 Repaso de Python **1**
0.1 Primeros pasos . . . 1
0.2 Tipos de datos básicos . . . 2
0.3 Colecciones . . . 3
0.4 Control de flujo . . . 4
0.5 Funciones . . . 5
0.6 Módulos y paquetes . . . 5
0.7 Excepciones . . . 6
0.8 Gestores de contexto: la sentencia `with` . . . 7
0.9 Programación orientada a objetos . . . 7
0.10 Iteradores y generadores . . . 9
0.11 Decoradores . . . 10
0.12 Otros elementos útiles . . . 10
0.13 Ejercicios . . . 11

1 Manejo de ficheros **14**
1.1 Gestión de ficheros y directorios . . . 14
1.2 Formas de acceso a un fichero . . . 16
1.3 Flujos . . . 17
1.4 Operaciones básicas con ficheros de texto . . . 17
1.5 Ficheros binarios . . . 18
1.6 Ficheros CSV . . . 21
1.7 Ficheros JSON . . . 22
1.8 Trabajo con ficheros XML . . . 23
1.9 Conversión de documentos XML a otros formatos . . . 27
1.10 Excepciones en el manejo de ficheros . . . 28
1.11 Pruebas y documentación . . . 29
1.12 Ejercicios . . . 30

2 Manejo de conectores **33**
2.1 El desfase objeto-relacional . . . 33
2.2 Protocolos de acceso a bases de datos y conectores . . . 34
2.3 Establecimiento de conexiones . . . 35
2.4 Ejecución de sentencias de descripción de datos (DDL) . . . 37

2.5 Ejecución de sentencias de modificación de datos (DML) . . . 37
2.6 Ejecución de consultas . . . 39
2.7 Ejecución de procedimientos almacenados . . . 41
2.8 Gestión de transacciones . . . 42
2.9 Excepciones de la DB-API . . . 44
2.10 Ejercicios . . . 45

3 Herramientas de mapeo objeto-relacional (ORM) 47
3.1 Concepto de mapeo objeto-relacional . . . 47
3.2 Características de las herramientas ORM . . . 48
3.3 Instalación de SQLAlchemy . . . 48
3.4 Configuración de la herramienta . . . 49
3.5 Clases persistentes y estructura del mapeo . . . 50
3.6 Mapeo de relaciones y colecciones . . . 51
3.7 Mapeo de la herencia . . . 53
3.8 Sesiones y estados de un objeto . . . 54
3.9 Carga, almacenamiento y modificación de objetos . . . 55
3.10 Consultas . . . 56
3.11 Gestión de transacciones . . . 58
3.12 Organización de una aplicación con ORM . . . 59
3.13 Ejercicios . . . 60

4 Bases de datos objeto-relacionales y orientadas a objetos 62
4.1 Bases de datos objeto-relacionales . . . 62
4.2 Gestión de objetos con SQL:1999 . . . 63
4.3 Acceso a las funciones del gestor desde Python . . . 65
4.4 Bases de datos orientadas a objetos . . . 67
4.5 Tipos de datos: básicos y estructurados . . . 69
4.6 El lenguaje de consultas OQL . . . 69
4.7 Una base de datos de objetos en Python: ZODB . . . 70
4.8 Pruebas y documentación . . . 74
4.9 Ejercicios . . . 75

5 Bases de datos XML y documentales 77
5.1 Bases de datos XML nativas . . . 77
5.2 Instalación y configuración de BaseX . . . 79
5.3 Establecimiento y cierre de conexiones . . . 80
5.4 Colecciones y documentos . . . 80
5.5 Lenguajes de consulta: XPath y XQuery . . . 81
5.6 Realización de consultas desde Python . . . 83
5.7 Indexación . . . 84

5.8 Identificadores únicos . . . 85
5.9 Gestión de transacciones . . . 85
5.10 Tratamiento de excepciones . . . 86
5.11 Bases de datos documentales JSON: MongoDB . . . 86
5.12 Ejercicios . . . 90

6 Programación de componentes de acceso a datos 92
6.1 Concepto de componente . . . 92
6.2 Programación orientada a componentes . . . 93
6.3 Propiedades y atributos . . . 94
6.4 Eventos y asociación de acciones a eventos . . . 96
6.5 Propiedades compartidas y restringidas . . . 97
6.6 Introspección y reflexión . . . 99
6.7 Editores de propiedades . . . 101
6.8 Persistencia del componente . . . 101
6.9 Componentes de acceso a datos . . . 102
6.10 Herramientas para el desarrollo de componentes no visuales . . . 105
6.11 Empaquetado de componentes . . . 106
6.12 Ejercicios . . . 107

A Preparación del entorno de trabajo 110
A.1 Python y el editor . . . 110
A.2 Servidores de bases de datos con Docker . . . 110
A.3 Referencias . . . 111

TEMA 0

Repaso de Python

En este tema aprenderás a…

- Ejecutar programas Python y gestionar dependencias con entornos virtuales y `pip`.
- Manejar los tipos de datos básicos y las colecciones (listas, tuplas, diccionarios y conjuntos).
- Escribir funciones con anotaciones de tipo, organizar el código en módulos y paquetes.
- Tratar errores mediante excepciones y usar gestores de contexto (`with`).
- Programar con clases, herencia, propiedades y *dataclasses*.
- Entender iteradores, generadores y decoradores, que aparecerán en el resto de temas.

Este tema no pretende ser un curso completo de Python, sino un repaso de las piezas que usaremos constantemente en el módulo. Si ya programas en Java o C#, verás que los conceptos son los mismos; lo que cambia es la sintaxis y algunas costumbres del lenguaje.

0.1 Primeros pasos

0.1.1 El intérprete y los scripts

Python es un lenguaje **interpretado** y de **tipado dinámico**: no hay fase de compilación explícita y las variables no se declaran con un tipo. Hay dos formas de ejecutar código:

- En el **intérprete interactivo** (REPL), escribiendo `python` en la terminal. Es muy útil para probar cosas rápidamente.
- Guardando el código en un fichero `.py` y ejecutándolo con `python fichero.py`.

```
$ python
>>> 2 + 3
5
>>> nombre = "Ada"
>>> f"Hola, {nombre}"
'Hola, Ada'
>>> exit()
```

En Python **la indentación forma parte de la sintaxis**: los bloques no se delimitan con llaves sino con sangría (por convenio, cuatro espacios), y la línea que abre un bloque termina en dos puntos.

```
edad = 19
if edad >= 18:
    print("Mayor de edad")
```

```
else:
    print("Menor de edad")
```

0.1.2 Entornos virtuales y paquetes

Durante el curso instalaremos bibliotecas de terceros (`lxml`, `psycopg`, `sqlalchemy`, `pymongo`...). Para que cada proyecto tenga sus propias dependencias y no interfieran entre sí, se usa un **entorno virtual**: una carpeta con una copia aislada del intérprete y sus paquetes.

```
$ python -m venv .venv               # crea el entorno en la carpeta .venv
$ source .venv/bin/activate          # Linux / macOS
> .venv\Scripts\activate             # Windows
(.venv) $ pip install lxml           # instala un paquete
(.venv) $ pip freeze > requirements.txt   # guarda las dependencias
(.venv) $ pip install -r requirements.txt # las reinstala en otro equipo
```

> **Nota.** Existen herramientas más modernas que combinan entornos virtuales y gestión de dependencias, como `uv` o `poetry`. En el Tema 6 veremos cómo describir un proyecto con `pyproject.toml`. Mientras tanto, `venv` + `pip` es suficiente.

0.2 Tipos de datos básicos

Tipo	Ejemplo	Comentario
`int`	`42`, `-7`, `1_000_000`	Enteros de precisión ilimitada.
`float`	`3.14`, `1e-3`	Coma flotante de doble precisión.
`bool`	`True`, `False`	Subtipo de `int`.
`str`	`"hola"`, `'hola'`	Cadenas Unicode inmutables.
`bytes`	`b"\x00\xff"`	Secuencias inmutables de bytes (Tema 1).
`NoneType`	`None`	Ausencia de valor (equivale a `null`).

```
x = 10            # int
x = "diez"        # ahora es str: el tipo lo tiene el valor, no la variable
print(type(x))    # <class 'str'>
print(7 / 2)      # 3.5  (división real)
print(7 // 2)     # 3    (división entera)
print(7 % 2)      # 1    (resto)
print(2 ** 10)    # 1024 (potencia)
```

0.2.1 Cadenas de caracteres

Las cadenas son inmutables y admiten índices (empezando en 0), índices negativos (contando desde el final) y *slicing* `[inicio:fin:paso]`.

```
s = "Acceso a Datos"
print(s[0], s[-1])        # A s
print(s[0:6])             # Acceso
```

```
print(s.upper())          # ACCESO A DATOS
print(s.split(" "))       # ['Acceso', 'a', 'Datos']
print("-".join(["a", "b", "c"]))   # a-b-c
print("  texto  ".strip())         # 'texto'
print(s.replace("Datos", "Ficheros"))
print("Datos" in s)       # True
```

Las **f-strings** son la forma recomendada de construir cadenas con valores:

```
nombre, nota = "Lucía", 8.456
print(f"{nombre} ha sacado un {nota:.2f}")   # Lucía ha sacado un 8.46
print(f"{nombre:>10}|")                       # alineado a la derecha en 10 posiciones
print(f"{nota=}")                             # nota=8.456 (útil para depurar)
```

0.3 Colecciones

0.3.1 Listas

Secuencias **mutables** y ordenadas. Pueden contener elementos de cualquier tipo.

```
modulos = ["Acceso a Datos", "PSP", "DI"]
modulos.append("PMDM")          # añade al final
modulos.insert(0, "SGE")        # inserta en una posición
modulos.remove("DI")            # elimina por valor
ultimo = modulos.pop()          # extrae el último
print(len(modulos), modulos)
print(sorted(modulos))          # nueva lista ordenada
modulos.sort(key=len)           # ordena en el sitio por longitud
```

0.3.2 Tuplas

Secuencias **inmutables**. Se usan para agrupar valores relacionados, por ejemplo una fila devuelta por una consulta SQL.

```
fila = (1, "Ana", 7.5)
ident, nombre, nota = fila      # desempaquetado
print(nombre)
```

0.3.3 Diccionarios

Colecciones de pares *clave: valor*. Son, con diferencia, la estructura más utilizada en Python y la que encaja de forma natural con JSON y con los documentos de MongoDB.

```
alumno = {"nombre": "Ana", "edad": 20, "modulos": ["AD", "PSP"]}
print(alumno["nombre"])
print(alumno.get("email", "sin email"))   # valor por defecto si no existe
alumno["email"] = "ana@ejemplo.es"         # añadir o modificar
del alumno["edad"]
for clave, valor in alumno.items():
    print(clave, "->", valor)
```

0.3.4 Conjuntos

Colecciones sin orden y **sin elementos repetidos**.

```
a = {"python", "java", "sql"}
b = {"sql", "xml"}
print(a | b, a & b, a - b)     # unión, intersección, diferencia
```

0.3.5 Comprensiones

Permiten construir colecciones a partir de otras en una sola expresión:

```
cuadrados = [n * n for n in range(10)]
pares = [n for n in range(20) if n % 2 == 0]
notas = {"Ana": 8, "Luis": 4, "Eva": 6}
aprobados = {nombre: nota for nombre, nota in notas.items() if nota >= 5}
```

0.4 Control de flujo

```
nota = 7
if nota >= 9:
    calificacion = "Sobresaliente"
elif nota >= 7:
    calificacion = "Notable"
elif nota >= 5:
    calificacion = "Aprobado"
else:
    calificacion = "Suspenso"

for i in range(3):                      # 0, 1, 2
    print(i)
for i, modulo in enumerate(["AD", "PSP"], start=1):
    print(i, modulo)
for nombre, nota in zip(["Ana", "Luis"], [8, 4]):
    print(nombre, nota)

contador = 0
while contador < 5:
    contador += 1
```

Desde Python 3.10 existe `match`, útil para procesar menús o estructuras de datos:

```
opcion = input("Opción: ")
match opcion:
    case "1":
        print("Alta")
    case "2" | "3":
        print("Modificación o baja")
    case _:
        print("Opción no válida")
```

0.5 Funciones

```python
def media(valores: list[float], decimales: int = 2) -> float:
    """Devuelve la media aritmética redondeada."""
    if not valores:
        raise ValueError("La lista está vacía")
    return round(sum(valores) / len(valores), decimales)

print(media([7, 8, 9]))
print(media([7, 8, 9], decimales=0))   # argumento por nombre
```

> **Anotaciones de tipo.** Las anotaciones (`list[float]`, `-> float`) **no se comprueban al ejecutar**: son documentación que entienden el editor y herramientas como `mypy`. Aun así las usaremos siempre, porque hacen el código mucho más legible y algunas bibliotecas (SQLAlchemy, *dataclasses*) las aprovechan.

Una función puede recibir un número variable de argumentos posicionales (`*args`) o con nombre (`**kwargs`):

```python
def registrar(mensaje: str, *etiquetas: str, **datos) -> None:
    print(mensaje, etiquetas, datos)

registrar("Alta", "alumno", "2DAM", id=3, nombre="Ana")
# Alta ('alumno', '2DAM') {'id': 3, 'nombre': 'Ana'}
```

Las funciones son objetos: se pueden guardar en variables y pasar como argumento. Las **funciones lambda** son funciones anónimas de una sola expresión:

```python
alumnos = [("Ana", 8), ("Luis", 4), ("Eva", 6)]
alumnos.sort(key=lambda a: a[1], reverse=True)   # ordenar por nota
```

0.6 Módulos y paquetes

Cada fichero `.py` es un **módulo**. Una carpeta con módulos es un **paquete**. Para usar el código de otro módulo se importa:

```python
import json                          # módulo completo
from pathlib import Path             # un nombre concreto
from datetime import date as Fecha   # con alias
```

Un módulo puede ejecutarse como programa o importarse desde otro. Para que cierto código se ejecute sólo en el primer caso se usa este patrón, que veremos en todos los ejemplos:

```python
def main() -> None:
    print("Programa principal")

if __name__ == "__main__":
    main()
```

Una estructura típica de proyecto del curso será:

```
proyecto/├──
 .venv/├──
 requirements.txt├──
 datos/│
   └── alumnos.csv└──
 gestion/            <- paquete
    ├── __init__.py
    ├── modelo.py
    ├── repositorio.py
    └── main.py
```

Y se ejecuta desde la carpeta `proyecto` con `python -m gestion.main`.

0.7 Excepciones

Cuando algo va mal (un fichero no existe, una conexión falla, un dato no tiene el formato esperado) Python **lanza una excepción**. Si nadie la captura, el programa termina mostrando la traza del error. Este mecanismo es central en el módulo: el criterio *"se han previsto y gestionado las excepciones"* aparece en varios resultados de aprendizaje.

```python
def leer_entero(texto: str) -> int | None:
    try:
        valor = int(texto)
    except ValueError as e:
        print(f"'{texto}' no es un número: {e}")
        return None
    else:
        # se ejecuta sólo si no hubo excepción
        return valor
    finally:
        # se ejecuta SIEMPRE, haya excepción o no
        print("Conversión terminada")
```

Algunas excepciones habituales:

`ValueError`	Valor con tipo correcto pero contenido inválido (`int("abc")`).
`TypeError`	Operación con un tipo inadecuado (`"a" + 1`).
`KeyError` / `IndexError`	Clave de diccionario o índice de lista inexistente.
`FileNotFoundError`	El fichero no existe (subclase de `OSError`).
`PermissionError`	Sin permisos sobre un fichero (subclase de `OSError`).
`sqlite3.Error`	Base de la jerarquía de errores de SQLite (Tema 2).

Podemos lanzar excepciones con `raise` y definir las nuestras heredando de `Exception`:

```python
class AlumnoNoEncontrado(Exception):
    """El alumno solicitado no existe en el almacén de datos."""

def buscar(alumnos: dict[int, str], ident: int) -> str:
    if ident not in alumnos:
        raise AlumnoNoEncontrado(f"No existe el alumno {ident}")
    return alumnos[ident]
```

Cuidado. Evita el `except:` vacío o `except Exception:` sin más, porque ocultan errores de programación. Captura sólo las excepciones que sabes tratar y deja que el resto se propague.

0.8 Gestores de contexto: la sentencia `with`

Muchos recursos (ficheros, conexiones a bases de datos, transacciones) deben **liberarse siempre**, incluso si se produce un error. La sentencia `with` lo garantiza:

```
with open("datos.txt", "w", encoding="utf-8") as f:
    f.write("Hola\n")
# aquí el fichero ya está cerrado, aunque write() hubiera fallado
```

Cualquier objeto con los métodos `__enter__` y `__exit__` es un gestor de contexto. También se pueden crear de forma sencilla con el decorador `contextlib.contextmanager`:

```
from contextlib import contextmanager
import time

@contextmanager
def cronometro(etiqueta: str):
    inicio = time.perf_counter()
    try:
        yield                       # aquí se ejecuta el bloque del with
    finally:
        print(f"{etiqueta}: {time.perf_counter() - inicio:.3f} s")

with cronometro("Carga de datos"):
    total = sum(range(10_000_000))
```

0.9 Programación orientada a objetos

0.9.1 Clases y objetos

```
class Alumno:
    centro = "IES Ejemplo"            # atributo de clase (compartido)

    def __init__(self, nombre: str, edad: int) -> None:
        self.nombre = nombre          # atributos de instancia
        self.edad = edad
        self.notas: list[float] = []

    def anadir_nota(self, nota: float) -> None:
        self.notas.append(nota)

    def media(self) -> float:
        return sum(self.notas) / len(self.notas) if self.notas else 0.0

    def __str__(self) -> str:         # representación para el usuario
        return f"{self.nombre} ({self.edad} años)"
```

```
    def __repr__(self) -> str:        # representación para el programador
        return f"Alumno(nombre={self.nombre!r}, edad={self.edad})"

a = Alumno("Ana", 20)
a.anadir_nota(8)
print(a, a.media())
```

- `__init__` es el inicializador (equivalente al constructor).
- `self` es la referencia al propio objeto y se declara explícitamente como primer parámetro.
- No existen `private` ni `protected`: por convenio, un nombre que empieza por `_` se considera interno.
- Los métodos con doble guion bajo (`__str__`, `__eq__`, `__len__`...) se llaman **métodos especiales** y permiten que nuestros objetos se integren con el lenguaje.

0.9.2 Herencia

```
class Persona:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

    def presentarse(self) -> str:
        return f"Soy {self.nombre}"

class Profesor(Persona):
    def __init__(self, nombre: str, departamento: str) -> None:
        super().__init__(nombre)
        self.departamento = departamento

    def presentarse(self) -> str:      # sobrescritura
        return f"{super().presentarse()} y trabajo en {self.departamento}"
```

0.9.3 Propiedades

En lugar de escribir *getters* y *setters* como en Java, Python permite acceder a los atributos directamente y, si más adelante hace falta validar, convertirlos en **propiedades** sin cambiar el código que los usa. Las propiedades serán muy importantes en el Tema 6.

```
class Producto:
    def __init__(self, nombre: str, precio: float) -> None:
        self.nombre = nombre
        self.precio = precio           # pasa por el setter

    @property
    def precio(self) -> float:
        return self._precio

    @precio.setter
    def precio(self, valor: float) -> None:
        if valor < 0:
            raise ValueError("El precio no puede ser negativo")
        self._precio = valor
```

```
p = Producto("Teclado", 25.0)
p.precio = 30            # llama al setter
p.precio = -1            # ValueError
```

0.9.4 Dataclasses

Para clases cuyo propósito principal es **guardar datos** (lo que en Java serían los POJO o los *records*), el decorador `@dataclass` genera automáticamente `__init__`, `__repr__` y `__eq__`. Las usaremos constantemente para representar los registros que leemos de ficheros y bases de datos.

```
from dataclasses import dataclass, field, asdict
from datetime import date

@dataclass
class Libro:
    isbn: str
    titulo: str
    autor: str
    anio: int
    disponible: bool = True
    etiquetas: list[str] = field(default_factory=list)

l = Libro("978-84-00000-00-0", "El Quijote", "Cervantes", 1605)
print(l)              # Libro(isbn='978-...', titulo='El Quijote', ...)
print(asdict(l))      # convierte a diccionario (útil para JSON)

@dataclass(frozen=True)     # inmutable
class Punto:
    x: float
    y: float
```

0.10 Iteradores y generadores

Un objeto es **iterable** si se puede recorrer con `for`. Los ficheros, los cursores de base de datos y los resultados de muchas consultas son iterables: los datos se van obteniendo *a medida que se piden*, sin cargarlo todo en memoria.

Una función que contiene `yield` es un **generador**: cada vez que se le pide un valor, se ejecuta hasta el siguiente `yield` y se detiene.

```
def lineas_no_vacias(ruta: str):
    with open(ruta, encoding="utf-8") as f:
        for linea in f:
            linea = linea.strip()
            if linea:
                yield linea

for l in lineas_no_vacias("datos.txt"):
    print(l)
```

Las **expresiones generadoras** son como las comprensiones de listas pero con paréntesis, y no construyen la lista en memoria:

```
total = sum(len(l) for l in lineas_no_vacias("datos.txt"))
```

0.11 Decoradores

Un decorador es una función que recibe otra función (o clase) y devuelve una versión modificada. Ya hemos usado varios (`@property`, `@dataclass`, `@contextmanager`). Escribir uno sencillo ayuda a entender cómo funcionan:

```
import functools

def registrar_llamadas(func):
    @functools.wraps(func)
    def envoltorio(*args, **kwargs):
        print(f"→ llamando a {func.__name__}{args}")
        resultado = func(*args, **kwargs)
        print(f"← {func.__name__} devolvió {resultado!r}")
        return resultado
    return envoltorio

@registrar_llamadas
def sumar(a: int, b: int) -> int:
    return a + b

sumar(2, 3)
```

0.12 Otros elementos útiles

0.12.1 Enumeraciones

```
from enum import Enum

class Estado(Enum):
    PENDIENTE = "P"
    ENVIADO = "E"
    ENTREGADO = "D"

e = Estado("E")          # Estado.ENVIADO
print(e.name, e.value)   # ENVIADO E
```

0.12.2 Fechas

```
from datetime import date, datetime, timedelta

hoy = date.today()
nacimiento = date.fromisoformat("2005-03-14")
print((hoy - nacimiento).days // 365, "años")
ahora = datetime.now()
print(ahora.strftime("%d/%m/%Y %H:%M"))
print(hoy + timedelta(days=30))
```

0.12.3 Argumentos de línea de comandos

Muchos de los programas que haremos serán utilidades de terminal. El módulo `argparse` simplifica leer sus argumentos:

```
import argparse

parser = argparse.ArgumentParser(description="Convierte CSV a JSON")
parser.add_argument("entrada", help="fichero CSV de entrada")
parser.add_argument("-o", "--salida", default="salida.json")
parser.add_argument("-v", "--verbose", action="store_true")
args = parser.parse_args()
print(args.entrada, args.salida, args.verbose)
```

0.13 Ejercicios

Ejercicio 0.1 — Calificaciones

Dificultad: ★

Escribe un programa que pida por teclado notas numéricas hasta que el usuario introduzca una línea vacía. Si una entrada no es un número o no está entre 0 y 10, muestra un aviso y sigue pidiendo. Al terminar, muestra cuántas notas se han introducido, la media, la máxima, la mínima y cuántas hay de cada calificación (Suspenso, Aprobado, Notable, Sobresaliente) usando un diccionario.

Ejercicio 0.2 — Comprensiones

Dificultad: ★

Dada la lista `palabras = ["python", "Java", "sql", "XML", "json", "Kotlin"]`, obtén usando comprensiones:

a) Una lista con las palabras en minúsculas.

b) Un diccionario *palabra* → *longitud*.

c) Un conjunto con las letras iniciales (en mayúsculas).

d) La lista de palabras con más de cuatro letras, ordenada alfabéticamente sin distinguir mayúsculas.

Ejercicio 0.3 — Funciones y excepciones

Dificultad: ★★

Implementa una función `validar_dni(dni: str) -> str` que reciba un DNI (8 dígitos y una letra), compruebe que el formato es correcto y que la letra coincide con la calculada (`"TRWAGMYFPDXBNJZSQVHLCKE"[numero % 23]`). Si es válido, lo devuelve normalizado en mayúsculas; si no, lanza una excepción propia `DNIInvalido` con un mensaje que explique el problema. Escribe un pequeño programa que pruebe la función con varios DNI correctos e incorrectos.

Ejercicio 0.4 — Clases

Dificultad: ★★

Crea una clase `CuentaBancaria` con titular, IBAN y saldo. El saldo debe ser una propiedad de sólo lectura; las operaciones `ingresar` y `retirar` lanzarán `ValueError` si la cantidad no es positiva o si no hay saldo suficiente. Añade un historial de movimientos (fecha, tipo, cantidad) e implementa `__str__` y `__len__` (número de movimientos). Crea después una subclase `CuentaAhorro` con un tipo de interés y un método `aplicar_intereses()`.

Ejercicio 0.5 — Dataclasses y ordenación

Dificultad: ★★

Define una *dataclass* `Pelicula` (título, director, año, duración en minutos, géneros). Crea una lista con al menos ocho películas y escribe funciones que:

a) Devuelvan las películas de un género dado, ordenadas por año.

b) Calculen la duración media por director (diccionario).

c) Devuelvan, mediante un generador, las películas que duran más de un valor dado.

Ejercicio 0.6 — Gestor de contexto y decorador

Dificultad: ★★★

a) Escribe un decorador `@reintentar(veces=3)` que vuelva a ejecutar la función decorada si lanza una excepción, hasta el número de veces indicado, y relance la última excepción si todos los intentos fallan. (Pista: un decorador con parámetros es una función que devuelve un decorador.)

b) Escribe un gestor de contexto `directorio_temporal()` que cree una carpeta temporal, la devuelva para usarla dentro del `with` y la borre al salir. Puedes usar `tempfile.mkdtemp` y `shutil.rmtree`.

Proyecto del tema: Agenda de contactos en memoria

Desarrolla una aplicación de consola para gestionar una agenda de contactos. En este tema los datos sólo se guardan en memoria; en el Tema 1 la ampliaremos para que persistan en ficheros.

Requisitos:

1. Un paquete `agenda` con, al menos, los módulos `modelo.py`, `servicio.py` y `main.py`.
2. En `modelo.py`, una *dataclass* `Contacto` (id, nombre, apellidos, teléfono, email, fecha de nacimiento opcional, etiquetas).
3. En `servicio.py`, una clase `Agenda` con métodos para añadir, buscar por id, buscar por texto (nombre o apellidos, sin distinguir mayúsculas), modificar, eliminar y listar ordenando por cualquier campo.
4. Excepciones propias: `ContactoNoEncontrado` y `ContactoDuplicado` (mismo email).
5. Validación del teléfono (9 dígitos) y del email (debe contener una sola `@` y un punto después) mediante propiedades o en `__post_init__`.
6. En `main.py`, un menú con `match` que use la clase `Agenda` y nunca termine con una traza de error: todos los errores previstos se muestran al usuario con un mensaje

claro.

7. Una opción que muestre los próximos cumpleaños de los siguientes 30 días.

Entrega: el código del paquete y un fichero `README.md` con instrucciones para ejecutarlo.

TEMA 1

Manejo de ficheros

En este tema aprenderás a…

- Gestionar ficheros y directorios: crear, borrar, copiar, mover, recorrer y consultar sus propiedades.
- Distinguir entre acceso secuencial y acceso aleatorio, y entre flujos de bytes y de caracteres.
- Leer y escribir ficheros de texto, binarios, CSV, JSON y ficheros de registros de longitud fija.
- Procesar documentos XML con analizadores DOM, SAX y ElementTree, y vincularlos a objetos (*binding*).
- Convertir documentos XML a otros formatos (JSON, CSV, HTML) mediante bibliotecas y XSLT.
- Prever y tratar las excepciones propias del trabajo con ficheros, y probar el código con `pytest`.

Resultado de aprendizaje asociado: **RA1**.

1.1 Gestión de ficheros y directorios

Un **fichero** es un conjunto de bytes almacenado en un dispositivo y al que el sistema operativo asigna un nombre. Un **directorio** (o carpeta) es un fichero especial que contiene referencias a otros ficheros y directorios, formando un árbol.

En Python hay tres módulos de la biblioteca estándar para trabajar con el sistema de ficheros:

`pathlib`	Representa rutas como objetos (`Path`). Es la forma **moderna y recomendada**.
`shutil`	Operaciones de alto nivel: copiar, mover y borrar árboles de directorios, comprimir.
`os`, `os.path`	Interfaz clásica, basada en cadenas. La encontrarás en mucho código existente.

1.1.1 Rutas con `pathlib`

```python
from pathlib import Path

base = Path.cwd()                       # directorio de trabajo actual
home = Path.home()                      # carpeta personal del usuario
ruta = base / "datos" / "alumnos.csv"   # el operador / une rutas

print(ruta)                 # /home/ana/proyecto/datos/alumnos.csv
print(ruta.name)            # alumnos.csv
```

```
print(ruta.stem)            # alumnos
print(ruta.suffix)          # .csv
print(ruta.parent)          # /home/ana/proyecto/datos
print(ruta.parts)           # ('/', 'home', 'ana', ...)
print(ruta.is_absolute())   # True
print(ruta.with_suffix(".json"))   # .../alumnos.json
print(Path("datos/../datos/a.txt").resolve())  # ruta absoluta normalizada
```

Rutas multiplataforma. Windows separa las carpetas con `\` y Linux/macOS con `/`. Si construyes las rutas con `Path` y el operador `/`, tu programa funcionará en todos los sistemas sin cambios. Nunca concatenes rutas con `+`.

1.1.2 Consultar información

```
from pathlib import Path
from datetime import datetime

p = Path("datos/alumnos.csv")
if p.exists():
    print("?`Fichero?", p.is_file(), "?`Directorio?", p.is_dir())
    info = p.stat()
    print("Tamaño:", info.st_size, "bytes")
    print("Modificado:", datetime.fromtimestamp(info.st_mtime))
```

1.1.3 Crear, renombrar y borrar

```
from pathlib import Path

carpeta = Path("salida/informes/2026")
carpeta.mkdir(parents=True, exist_ok=True)   # crea toda la ruta si no existe

f = carpeta / "notas.txt"
f.touch()                                    # crea un fichero vacío
f.write_text("Primera línea\n", encoding="utf-8")
nuevo = f.rename(carpeta / "notas_1T.txt")   # renombrar o mover
nuevo.unlink(missing_ok=True)                # borrar un fichero
carpeta.rmdir()                              # borrar directorio (debe estar vacío)
```

1.1.4 Copiar, mover y borrar árboles: `shutil`

```
import shutil
from pathlib import Path

shutil.copy("datos/alumnos.csv", "copia/")          # copia el contenido
shutil.copy2("datos/alumnos.csv", "copia/a.csv")    # copia también fechas y permisos
shutil.copytree("datos", "copia_datos", dirs_exist_ok=True)  # copia un árbol
shutil.move("copia/a.csv", "historico/a.csv")       # mueve (o renombra)
shutil.rmtree("copia_datos")                        # !`borra el árbol completo!
shutil.make_archive("backup", "zip", "datos")       # crea backup.zip
```

```
total, usado, libre = shutil.disk_usage("/")
print(f"Libre: {libre / 2**30:.1f} GiB")
```

Cuidado. `shutil.rmtree` borra sin pasar por la papelera y sin pedir confirmación. Comprueba siempre la ruta antes de usarlo.

1.1.5 Recorrer directorios

```
from pathlib import Path

carpeta = Path("proyecto")
for elemento in carpeta.iterdir():              # sólo el primer nivel
    tipo = "DIR " if elemento.is_dir() else "FILE"
    print(tipo, elemento.name)

for py in carpeta.glob("*.py"):                  # patrón en el primer nivel
    print(py)

for py in carpeta.rglob("*.py"):                 # patrón recursivo
    print(py, py.stat().st_size)

# Tamaño total de una carpeta
total = sum(f.stat().st_size for f in carpeta.rglob("*") if f.is_file())
```

1.2 Formas de acceso a un fichero

Según cómo se llegue a los datos, distinguimos dos formas de acceso:

Acceso secuencial. Los datos se leen o escriben en orden, desde el principio hasta el final. Para llegar al registro 1000 hay que pasar por los 999 anteriores.

Acceso aleatorio (o directo). Se puede ir directamente a cualquier posición del fichero indicando su desplazamiento en bytes (`seek`). Requiere saber dónde está cada dato, lo que normalmente implica **registros de longitud fija** o un índice.

	Secuencial	**Aleatorio**
Ventajas	Sencillo; aprovecha todo el espacio; ideal para procesar todos los datos; registros de longitud variable.	Lectura y modificación de un registro concreto muy rápida, sin reescribir el fichero.
Inconvenientes	Buscar o modificar un registro concreto es lento; modificar suele implicar reescribir el fichero.	Registros de tamaño fijo (se desperdicia espacio); más complejo de programar; no apto para texto libre.
Aplicaciones	Logs, CSV, JSON, XML, exportaciones, copias de seguridad, ficheros de configuración.	Ficheros de índices, bases de datos, cachés, formatos binarios (imágenes, vídeo).

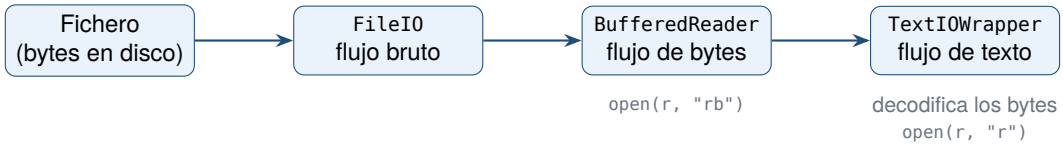
1.3 Flujos

Un **flujo** (*stream*) es una abstracción que representa una secuencia de datos que fluye desde un origen (fichero, teclado, conexión de red, memoria) hacia el programa, o desde el programa hacia un destino. Permite tratar todos esos orígenes y destinos de la misma forma.

Python distingue dos tipos de flujos:

Flujos de bytes (binarios). Trabajan con objetos `bytes`. No interpretan el contenido. Se usan para imágenes, ficheros comprimidos, ficheros de registros, etc. Se abren con modo `"b"`.

Flujos de caracteres (texto). Trabajan con objetos `str`. Internamente leen bytes y los **decodifican** usando una codificación de caracteres (UTF-8, Latin-1...). También traducen los saltos de línea de cada sistema operativo a `\n`.



Las clases que intervienen están en el módulo `io`, aunque casi nunca se instancian directamente: la función `open()` construye la cadena adecuada según el modo. El módulo `io` también ofrece flujos en memoria (`io.StringIO` e `io.BytesIO`), que se comportan como ficheros pero no tocan el disco; son muy útiles en pruebas.

1.3.1 La función `open` y los modos de apertura

Modo	Significado
`"r"`	Lectura (por defecto). Error si el fichero no existe.
`"w"`	Escritura. Crea el fichero o **lo vacía** si ya existe.
`"a"`	Añadir al final. Crea el fichero si no existe.
`"x"`	Creación exclusiva. Error si el fichero ya existe.
`"+"`	Añadido a otro modo, permite leer y escribir (`"r+"`, `"w+"`).
`"b"`	Modo binario (`"rb"`, `"wb"`, `"r+b"`...).
`"t"`	Modo texto (por defecto).

Indica siempre la codificación. Si abres un fichero de texto sin `encoding`, Python usa la codificación por defecto del sistema, que en Windows puede no ser UTF-8. El mismo programa podría leer bien las tildes en Linux y mal en Windows. Escribe siempre `encoding="utf-8"`.

1.4 Operaciones básicas con ficheros de texto

1.4.1 Lectura

```
from pathlib import Path
```

```python
# 1) Todo el contenido de golpe (ficheros pequeños)
texto = Path("poema.txt").read_text(encoding="utf-8")

# 2) Línea a línea (ficheros de cualquier tamaño: no se carga todo en memoria)
with open("poema.txt", encoding="utf-8") as f:
    for num, linea in enumerate(f, start=1):
        print(f"{num:3}: {linea.rstrip()}")

# 3) Otros métodos del objeto fichero
with open("poema.txt", encoding="utf-8") as f:
    primera = f.readline()       # una línea (incluye '\n')
    cinco = f.read(5)            # 5 caracteres
    resto = f.readlines()        # lista con las líneas restantes
```

1.4.2 Escritura

```python
lineas = ["Ana;8\n", "Luis;5\n"]
with open("notas.txt", "w", encoding="utf-8") as f:
    f.write("nombre;nota\n")
    f.writelines(lineas)
    print("Eva;9", file=f)       # print también puede escribir en un fichero

with open("notas.txt", "a", encoding="utf-8") as f:   # añadir al final
    f.write("Marta;7\n")
```

1.4.3 Modificar un fichero de texto

Un fichero de texto no permite insertar o borrar en medio de su contenido: la técnica habitual es **leer, transformar y reescribir**. Para no perder datos si el programa falla a mitad, se escribe en un fichero temporal y después se sustituye el original:

```python
import os
from pathlib import Path

def reemplazar_en_fichero(ruta: Path, viejo: str, nuevo: str) -> int:
    """Sustituye un texto en todo el fichero. Devuelve cuántas líneas cambiaron."""
    temporal = ruta.with_suffix(ruta.suffix + ".tmp")
    cambios = 0
    with open(ruta, encoding="utf-8") as origen, \
         open(temporal, "w", encoding="utf-8") as destino:
        for linea in origen:
            if viejo in linea:
                cambios += 1
                linea = linea.replace(viejo, nuevo)
            destino.write(linea)
    os.replace(temporal, ruta)     # sustitución atómica
    return cambios
```

1.5 Ficheros binarios

1.5.1 Lectura y escritura de bytes

```
# Copiar un fichero binario por bloques
TAM_BLOQUE = 64 * 1024
with open("foto.jpg", "rb") as origen, open("copia.jpg", "wb") as destino:
    while bloque := origen.read(TAM_BLOQUE):    # operador "morsa" (:=)
        destino.write(bloque)

# Comprobar la "firma" de un fichero PNG
with open("imagen.png", "rb") as f:
    cabecera = f.read(8)
print(cabecera == b"\x89PNG\r\n\x1a\n")
```

1.5.2 Registros de longitud fija y acceso aleatorio con `struct`

Para guardar datos estructurados en binario y acceder directamente al registro n, todos los registros deben ocupar lo mismo. El módulo `struct` convierte valores de Python en bytes con un formato fijo:

Código	Tipo	Bytes
`i` / `I`	entero con / sin signo	4
`q`	entero largo	8
`d`	real de doble precisión	8
`?`	booleano	1
`30s`	cadena de 30 bytes	30
`<`	(prefijo) orden *little-endian*, sin relleno	—

El siguiente ejemplo implementa un pequeño fichero de productos con acceso directo por posición:

```
import struct
from dataclasses import dataclass
from pathlib import Path

FORMATO = "<i30sdi?"           # id, nombre, precio, stock, activo
TAM = struct.calcsize(FORMATO)  # 4 + 30 + 8 + 4 + 1 = 47 bytes

@dataclass
class Producto:
    id: int
    nombre: str
    precio: float
    stock: int
    activo: bool = True

    def a_bytes(self) -> bytes:
        nombre = self.nombre.encode("utf-8")[:30]   # se recorta/rellena a 30
        return struct.pack(FORMATO, self.id, nombre, self.precio,
                           self.stock, self.activo)

    @classmethod
    def desde_bytes(cls, datos: bytes) -> "Producto":
```

```
        id_, nombre, precio, stock, activo = struct.unpack(FORMATO, datos)
        nombre = nombre.rstrip(b"\x00").decode("utf-8", errors="ignore")
        return cls(id_, nombre, precio, stock, activo)


class FicheroProductos:
    def __init__(self, ruta: str | Path) -> None:
        self.ruta = Path(ruta)
        self.ruta.touch(exist_ok=True)

    def __len__(self) -> int:
        return self.ruta.stat().st_size // TAM

    def anadir(self, p: Producto) -> int:
        with open(self.ruta, "ab") as f:
            f.write(p.a_bytes())
        return len(self) - 1                    # posición del registro

    def leer(self, pos: int) -> Producto:
        if not 0 <= pos < len(self):
            raise IndexError(f"No existe el registro {pos}")
        with open(self.ruta, "rb") as f:
            f.seek(pos * TAM)                   # salto directo
            return Producto.desde_bytes(f.read(TAM))

    def escribir(self, pos: int, p: Producto) -> None:
        if not 0 <= pos < len(self):
            raise IndexError(f"No existe el registro {pos}")
        with open(self.ruta, "r+b") as f:       # lectura + escritura
            f.seek(pos * TAM)
            f.write(p.a_bytes())

    def borrar(self, pos: int) -> None:
        """Borrado lógico: se marca como inactivo."""
        p = self.leer(pos)
        p.activo = False
        self.escribir(pos, p)

    def __iter__(self):
        with open(self.ruta, "rb") as f:
            while datos := f.read(TAM):
                p = Producto.desde_bytes(datos)
                if p.activo:
                    yield p


if __name__ == "__main__":
    fp = FicheroProductos("productos.dat")
    fp.anadir(Producto(1, "Teclado", 24.90, 10))
    fp.anadir(Producto(2, "Ratón", 12.50, 25))
    p = fp.leer(1)
    p.stock -= 1
    fp.escribir(1, p)
    for prod in fp:
        print(prod)
```

Borrado lógico y compactación. En los ficheros de acceso aleatorio no se suele eliminar físicamente un registro (habría que desplazar todos los siguientes). Se marca como borrado y, de vez en cuando, se *compacta* el fichero copiando sólo los registros activos a uno nuevo.

1.5.3 Serialización de objetos con `pickle`

Serializar es convertir un objeto en memoria en una secuencia de bytes para guardarlo o enviarlo; **deserializar** es el proceso inverso. El módulo `pickle` serializa casi cualquier objeto de Python (equivale a la serialización de Java con `ObjectOutputStream`).

```
import pickle

agenda = {"Ana": ["600111222"], "Luis": ["611222333", "922000000"]}
with open("agenda.pkl", "wb") as f:
    pickle.dump(agenda, f)

with open("agenda.pkl", "rb") as f:
    recuperada = pickle.load(f)
```

Cuidado. `pickle` **sólo debe usarse con datos de confianza**: al deserializar se puede ejecutar código arbitrario. Además, el formato sólo lo entiende Python. Para intercambiar datos con otros programas usa formatos de texto estándar como JSON, CSV o XML.

1.6 Ficheros CSV

Un fichero CSV (*comma-separated values*) almacena una tabla en texto: una fila por línea y los campos separados por un delimitador (coma, punto y coma, tabulador...). Aunque parece sencillo, hay detalles (campos con el delimitador dentro, comillas, saltos de línea) que hacen que **no debamos procesarlo con `split`**, sino con el módulo `csv`.

```
import csv
from pathlib import Path

alumnos = [
    {"nombre": "Ana", "apellidos": "García, López", "nota": 8.5},
    {"nombre": "Luis", "apellidos": "Pérez", "nota": 4.25},
]

# Escritura a partir de diccionarios
with open("alumnos.csv", "w", newline="", encoding="utf-8") as f:
    escritor = csv.DictWriter(f, fieldnames=["nombre", "apellidos", "nota"],
                              delimiter=";")
    escritor.writeheader()
    escritor.writerows(alumnos)

# Lectura como diccionarios (la primera fila es la cabecera)
with open("alumnos.csv", newline="", encoding="utf-8") as f:
    for fila in csv.DictReader(f, delimiter=";"):
        print(fila["nombre"], float(fila["nota"]))   # ¡todo se lee como str!

# Lectura como listas
```

```python
with open("alumnos.csv", newline="", encoding="utf-8") as f:
    lector = csv.reader(f, delimiter=";")
    cabecera = next(lector)
    filas = list(lector)
```

> **Nota.** Se abre con `newline=""` porque el módulo `csv` gestiona los saltos de línea por su cuenta. Los ficheros CSV generados por Excel en español suelen usar `;` como separador y, a veces, la codificación `"utf-8-sig"` (UTF-8 con BOM) o `"cp1252"`.

1.7 Ficheros JSON

JSON (*JavaScript Object Notation*) es hoy el formato de intercambio de datos más utilizado, sobre todo en servicios web. Su correspondencia con Python es directa:

JSON	Python
objeto `{}`	`dict`
array `[]`	`list`
cadena	`str`
número	`int` / `float`
`true` / `false` / `null`	`True` / `False` / `None`

```python
import json
from dataclasses import dataclass, asdict
from datetime import date

@dataclass
class Alumno:
    nombre: str
    nacimiento: date
    modulos: list[str]

alumnos = [Alumno("Ana", date(2005, 3, 14), ["AD", "PSP"])]

# Serializar: las fechas no son tipos JSON, se convierten con default=
def a_json(obj):
    if isinstance(obj, date):
        return obj.isoformat()
    raise TypeError(f"Tipo no serializable: {type(obj)}")

with open("alumnos.json", "w", encoding="utf-8") as f:
    json.dump([asdict(a) for a in alumnos], f, indent=2,
              ensure_ascii=False, default=a_json)

# Deserializar y reconstruir los objetos
with open("alumnos.json", encoding="utf-8") as f:
    datos = json.load(f)
alumnos = [Alumno(d["nombre"], date.fromisoformat(d["nacimiento"]), d["modulos"])
           for d in datos]

texto = json.dumps({"ok": True})      # a cadena
obj = json.loads('{"ok": true}')      # desde cadena
```

1.8 Trabajo con ficheros XML

1.8.1 Repaso de XML

XML (*eXtensible Markup Language*) es un lenguaje de marcas para representar información estructurada de forma jerárquica. Lo usaremos en este tema y en el Tema 5. Este será nuestro documento de ejemplo, `biblioteca.xml`:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<biblioteca nombre="Biblioteca del IES">
  <libro isbn="978-84-376-0494-7" prestado="no">
    <titulo>Cien años de soledad</titulo>
    <autor>Gabriel García Márquez</autor>
    <anio>1967</anio>
    <precio moneda="EUR">19.95</precio>
  </libro>
  <libro isbn="978-84-206-3311-6" prestado="si">
    <titulo>La colmena</titulo>
    <autor>Camilo José Cela</autor>
    <anio>1951</anio>
    <precio moneda="EUR">12.50</precio>
  </libro>
</biblioteca>
```

- Un documento es **bien formado** si cumple las reglas sintácticas de XML: un único elemento raíz, etiquetas correctamente anidadas y cerradas, atributos entrecomillados, etc.
- Es **válido** si además cumple las reglas de un esquema (DTD o XML Schema/XSD) que define qué elementos y atributos se permiten y en qué orden.

1.8.2 Analizadores sintácticos (*parsers*)

Un **parser** lee el texto XML, comprueba que es correcto y ofrece su contenido al programa. Hay dos grandes modelos:

	DOM (*Document Object Model*)	**SAX** (*Simple API for XML*)
Funcionamiento	Carga el documento completo en memoria como un **árbol de nodos**.	Recorre el documento de forma secuencial y **lanza eventos** (inicio de elemento, texto, fin de elemento…).
Memoria	Proporcional al tamaño del documento.	Muy baja y constante.
Navegación	Libre: se puede ir hacia delante, atrás, padres, hijos.	Sólo hacia delante; el programa debe guardar el estado que necesite.
Modificación	Sí: se puede modificar el árbol y volver a guardarlo.	No: sólo lectura.
Uso típico	Documentos pequeños o medianos que hay que modificar o consultar varias veces.	Documentos muy grandes; extraer unos pocos datos.

Python incluye ambos (`xml.dom.minidom` y `xml.sax`) y además una API intermedia más cómoda, **ElementTree** (`xml.etree.ElementTree`), que es la que usaremos normalmente. La biblioteca externa **lxml** implementa la misma API que ElementTree pero añade XPath completo, validación con esquemas y XSLT.

1.8.3 DOM con `minidom`

Es la API estándar del W3C, idéntica en concepto a la de Java o JavaScript (métodos como `getElementsByTagName`, `createElement` o `appendChild`).

```
from xml.dom import minidom

doc = minidom.parse("biblioteca.xml")
raiz = doc.documentElement
print("Raíz:", raiz.tagName, raiz.getAttribute("nombre"))

for libro in doc.getElementsByTagName("libro"):
    isbn = libro.getAttribute("isbn")
    titulo = libro.getElementsByTagName("titulo")[0].firstChild.data
    print(isbn, titulo)

# Añadir un libro nuevo
nuevo = doc.createElement("libro")
nuevo.setAttribute("isbn", "978-84-9759-000-1")
nuevo.setAttribute("prestado", "no")
titulo = doc.createElement("titulo")
titulo.appendChild(doc.createTextNode("Nada"))
nuevo.appendChild(titulo)
raiz.appendChild(nuevo)

with open("biblioteca_dom.xml", "w", encoding="utf-8") as f:
    doc.writexml(f, encoding="utf-8")
```

1.8.4 SAX

Con SAX escribimos una clase manejadora (*handler*) cuyos métodos se llaman a medida que el parser encuentra cada parte del documento.

```
import xml.sax

class ManejadorLibros(xml.sax.ContentHandler):
    def __init__(self) -> None:
        super().__init__()
        self.libros: list[dict] = []
        self.actual: dict | None = None
        self.texto = ""

    def startElement(self, nombre, atributos):
        if nombre == "libro":
            self.actual = {"isbn": atributos.get("isbn")}
        self.texto = ""

    def characters(self, contenido):
        self.texto += contenido          # puede llegar en varios trozos
```

```
    def endElement(self, nombre):
        if self.actual is not None and nombre in ("titulo", "autor", "anio"):
            self.actual[nombre] = self.texto.strip()
        elif nombre == "libro":
            self.libros.append(self.actual)
            self.actual = None

manejador = ManejadorLibros()
xml.sax.parse("biblioteca.xml", manejador)
for libro in manejador.libros:
    print(libro)
```

1.8.5 ElementTree

ElementTree representa el documento como un árbol de objetos `Element`. Cada elemento tiene un nombre (`tag`), un diccionario de atributos (`attrib`), un texto (`text`) y se comporta como una lista de sus hijos.

```
import xml.etree.ElementTree as ET

arbol = ET.parse("biblioteca.xml")
raiz = arbol.getroot()

for libro in raiz.findall("libro"):
    titulo = libro.findtext("titulo")
    anio = int(libro.findtext("anio"))
    precio = float(libro.find("precio").text)
    print(f"{libro.get('isbn')}: {titulo} ({anio}) {precio:.2f} €")

# Búsquedas con un subconjunto de XPath
prestados = raiz.findall("./libro[@prestado='si']")
todos_los_titulos = [t.text for t in raiz.iter("titulo")]

# Modificar: subir un 10 % los precios
for precio in raiz.iter("precio"):
    precio.text = f"{float(precio.text) * 1.10:.2f}"

# Añadir un elemento
libro = ET.SubElement(raiz, "libro", isbn="978-84-9759-000-1", prestado="no")
ET.SubElement(libro, "titulo").text = "Nada"
ET.SubElement(libro, "autor").text = "Carmen Laforet"
ET.SubElement(libro, "anio").text = "1945"

# Eliminar los libros anteriores a 1950
for libro in raiz.findall("libro"):
    if int(libro.findtext("anio")) < 1950:
        raiz.remove(libro)

ET.indent(arbol, space="  ")              # sangría bonita (Python 3.9+)
arbol.write("biblioteca_mod.xml", encoding="utf-8", xml_declaration=True)
```

1.8.6 XPath y validación con `lxml`

```
$ pip install lxml
```

```python
from lxml import etree

arbol = etree.parse("biblioteca.xml")

# XPath completo: devuelve elementos, atributos, textos o números
titulos = arbol.xpath("//libro[anio > 1960]/titulo/text()")
total = arbol.xpath("sum(//precio)")
isbns = arbol.xpath("//libro/@isbn")

# Validación contra un XML Schema
esquema = etree.XMLSchema(etree.parse("biblioteca.xsd"))
if esquema.validate(arbol):
    print("Documento válido")
else:
    for error in esquema.error_log:
        print(f"Línea {error.line}: {error.message}")
```

1.8.7 Vinculación (*binding*) de XML a objetos

Con los parsers anteriores el programa trabaja con nodos, atributos y textos, y es responsabilidad nuestra convertirlos en objetos del dominio (`Libro`) y viceversa. La **vinculación de datos** (*data binding*) automatiza esa traducción: se define una correspondencia entre clases y la estructura XML, y una biblioteca se encarga de **deserializar** (*unmarshalling*: XML → objetos) y **serializar** (*marshalling*: objetos → XML). En Java la herramienta clásica es JAXB; en Python, una de las más completas es **xsdata**, que además puede generar las clases automáticamente a partir de un XSD.

```
$ pip install xsdata
```

```python
from dataclasses import dataclass, field
from pathlib import Path
from xsdata.formats.dataclass.parsers import XmlParser
from xsdata.formats.dataclass.serializers import XmlSerializer
from xsdata.formats.dataclass.serializers.config import SerializerConfig

@dataclass
class Libro:
    class Meta:
        name = "libro"
    isbn: str = field(metadata={"type": "Attribute"})
    titulo: str = field(metadata={"type": "Element"})
    autor: str = field(metadata={"type": "Element"})
    anio: int = field(metadata={"type": "Element"})

@dataclass
class Biblioteca:
    class Meta:
        name = "biblioteca"
    nombre: str = field(default="", metadata={"type": "Attribute"})
    libros: list[Libro] = field(default_factory=list,
                                metadata={"type": "Element", "name": "libro"})
```

```python
# Unmarshalling: XML -> objetos (con los tipos ya convertidos: anio es int)
bib = XmlParser().from_path(Path("biblioteca_simple.xml"), Biblioteca)
for libro in bib.libros:
    print(libro.titulo, libro.anio + 1)

# Marshalling: objetos -> XML
bib.libros.append(Libro("978-84-9759-000-1", "Nada", "Carmen Laforet", 1945))
xml = XmlSerializer(config=SerializerConfig(indent="  ")).render(bib)
with open("biblioteca_nueva.xml", "w", encoding="utf-8") as f:
    f.write(xml)
```

Si se dispone de un esquema XSD, las clases no hace falta escribirlas a mano:

```
$ xsdata generate biblioteca.xsd --package modelo
```

> **Nota.** Alternativas a xsdata: `pydantic-xml` (basada en Pydantic, con validación de datos) o `xmltodict`, que no vincula a clases propias sino a diccionarios, pero es muy útil para conversiones rápidas.

1.9 Conversión de documentos XML a otros formatos

1.9.1 XML a JSON y CSV

```python
import csv, json
import xml.etree.ElementTree as ET

raiz = ET.parse("biblioteca.xml").getroot()
libros = [
    {
        "isbn": l.get("isbn"),
        "titulo": l.findtext("titulo"),
        "autor": l.findtext("autor"),
        "anio": int(l.findtext("anio")),
        "precio": float(l.findtext("precio")),
        "prestado": l.get("prestado") == "si",
    }
    for l in raiz.findall("libro")
]

with open("biblioteca.json", "w", encoding="utf-8") as f:
    json.dump(libros, f, indent=2, ensure_ascii=False)

with open("biblioteca.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=libros[0].keys(), delimiter=";")
    w.writeheader()
    w.writerows(libros)
```

Con `xmltodict` la conversión genérica a diccionario es inmediata (los atributos aparecen con el prefijo `@`):

```python
import json, xmltodict

with open("biblioteca.xml", encoding="utf-8") as f:
```

```
    datos = xmltodict.parse(f.read())
print(json.dumps(datos, indent=2, ensure_ascii=False))
xml_de_nuevo = xmltodict.unparse(datos, pretty=True)
```

1.9.2 Transformaciones XSLT

XSLT es un lenguaje (también escrito en XML) para transformar un documento XML en otro documento: XML con otra estructura, HTML o texto plano. Es la herramienta estándar de conversión de XML. `lxml` incluye un procesador XSLT 1.0.

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- biblioteca_html.xsl -->
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="html" encoding="UTF-8" indent="yes"/>
  <xsl:template match="/biblioteca">
    <html>
      <head><title><xsl:value-of select="@nombre"/></title></head>
      <body>
        <h1><xsl:value-of select="@nombre"/></h1>
        <table border="1">
          <tr><th>Título</th><th>Autor</th><th>Año</th></tr>
          <xsl:for-each select="libro">
            <xsl:sort select="anio" data-type="number"/>
            <tr>
              <td><xsl:value-of select="titulo"/></td>
              <td><xsl:value-of select="autor"/></td>
              <td><xsl:value-of select="anio"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

```
from lxml import etree

xml = etree.parse("biblioteca.xml")
transformar = etree.XSLT(etree.parse("biblioteca_html.xsl"))
resultado = transformar(xml)
resultado.write_output("biblioteca.html")   # respeta xsl:output
```

> **Nota.** Para convertir XML a otros formatos también son útiles: `pandas` (`read_xml`/`to_xml`, `to_excel`), `openpyxl` para hojas de cálculo y, para generar PDF, la combinación XML → HTML (XSLT) → PDF con `weasyprint`.

1.10 Excepciones en el manejo de ficheros

Casi cualquier operación con ficheros puede fallar por motivos ajenos al programa: el fichero no existe, no hay permisos, el disco está lleno, el contenido está corrupto... Un programa robusto debe **prever** estos casos y **tratarlos**.

Excepción	Cuándo se produce
`OSError`	Base de todos los errores del sistema de ficheros.
`FileNotFoundError`	El fichero o directorio no existe.
`FileExistsError`	Ya existe (modo `"x"`, `mkdir` sin `exist_ok`).
`PermissionError`	Sin permisos de lectura/escritura.
`IsADirectoryError`	Se intenta abrir un directorio como fichero.
`UnicodeDecodeError`	El contenido no corresponde a la codificación indicada.
`json.JSONDecodeError`	JSON mal formado (subclase de `ValueError`).
`csv.Error`	CSV mal formado.
`xml.etree.ElementTree.ParseError`	XML mal formado (con ElementTree).
`lxml.etree.XMLSyntaxError`	XML mal formado (con lxml).
`struct.error`	Datos binarios con tamaño o formato incorrecto.

```python
import json
from pathlib import Path

def cargar_configuracion(ruta: Path) -> dict:
    try:
        with open(ruta, encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"No existe {ruta}; se usará la configuración por defecto")
        return {}
    except PermissionError:
        raise SystemExit(f"Sin permiso para leer {ruta}")
    except json.JSONDecodeError as e:
        raise SystemExit(f"{ruta} no es JSON válido (línea {e.lineno}): {e.msg}")
```

¿Comprobar antes o capturar después? Podríamos comprobar con `ruta.exists()` antes de abrir, pero entre la comprobación y la apertura otro proceso podría borrar el fichero. En Python se prefiere el estilo **EAFP** (*easier to ask forgiveness than permission*): intentar la operación y capturar la excepción.

1.11 Pruebas y documentación

El RA1 exige probar y documentar las aplicaciones. Usaremos dos herramientas:

- **Docstrings**: cadenas de documentación al principio de módulos, clases y funciones. Las muestran los editores y `help()`, y herramientas como `pdoc` generan documentación HTML a partir de ellas.
- **pytest**: el marco de pruebas más usado en Python. Las pruebas son funciones cuyo nombre empieza por `test_` y usan `assert`. La *fixture* `tmp_path` proporciona un directorio temporal limpio para cada prueba, ideal para probar código que trabaja con ficheros.

```python
# test_productos.py
import pytest
```

```python
from productos import FicheroProductos, Producto

def test_anadir_y_leer(tmp_path):
    fp = FicheroProductos(tmp_path / "p.dat")
    pos = fp.anadir(Producto(1, "Teclado", 24.9, 10))
    assert pos == 0
    assert fp.leer(0).nombre == "Teclado"

def test_leer_posicion_inexistente(tmp_path):
    fp = FicheroProductos(tmp_path / "p.dat")
    with pytest.raises(IndexError):
        fp.leer(5)

def test_borrado_logico(tmp_path):
    fp = FicheroProductos(tmp_path / "p.dat")
    fp.anadir(Producto(1, "A", 1.0, 1))
    fp.anadir(Producto(2, "B", 2.0, 2))
    fp.borrar(0)
    assert [p.id for p in fp] == [2]
```

```
$ pip install pytest
$ pytest -v
```

1.12 Ejercicios

Ejercicio 1.1 — Explorador de directorios

Dificultad: ★

Escribe un programa que reciba una ruta por línea de comandos (`argparse`) y muestre su contenido en forma de árbol, con sangría según la profundidad, indicando el tamaño de cada fichero en un formato legible (B, KB, MB). Al final debe mostrar el número total de ficheros, de directorios y el tamaño total. Si la ruta no existe o no es un directorio, muestra un mensaje de error y termina con código 1.

Ejercicio 1.2 — Organizador de descargas

Dificultad: ★★

Programa una utilidad que ordene los ficheros de una carpeta moviéndolos a subcarpetas según su extensión (`Imagenes`, `Documentos`, `Audio`, `Comprimidos`, `Otros`). Las correspondencias extensión–carpeta se leen de un fichero `reglas.json`. Si en el destino ya existe un fichero con el mismo nombre, añade un sufijo numérico. Incluye una opción `--simular` que muestre lo que haría sin mover nada, y guarda en `organizador.log` cada movimiento realizado con fecha y hora.

Ejercicio 1.3 — Estadísticas de un fichero de texto

Dificultad: ★

Lee un fichero de texto (por ejemplo, un libro descargado del Proyecto Gutenberg) y genera un fichero `estadisticas.txt` con: número de líneas, palabras y caracteres; las 20 palabras más frecuentes (ignorando mayúsculas y signos de puntuación) y la longitud media de palabra. Prueba a leerlo con una codificación incorrecta y trata la excepción.

Ejercicio 1.4 — Fichero de acceso aleatorio

Dificultad: ★★

Partiendo del ejemplo `FicheroProductos`, crea un fichero de acceso aleatorio de **empleados** (id, DNI, nombre, departamento, salario) en el que la posición del registro sea `id - 1`. Implementa: alta (si el id ya existe y está activo, error), consulta por id, modificación del salario, baja lógica, listado de activos y una operación `compactar()` que elimine físicamente los registros borrados. Escribe pruebas con `pytest` para cada operación.

Ejercicio 1.5 — Conversor CSV ↔ JSON

Dificultad: ★★

Desarrolla un conversor de línea de comandos: `python conversor.py entrada.csv salida.json` y viceversa (el sentido se deduce de las extensiones). En la conversión CSV → JSON intenta convertir automáticamente los valores numéricos y booleanos. Permite indicar el delimitador y la codificación. Gestiona: fichero inexistente, extensión no soportada, JSON que no es una lista de objetos y filas CSV con distinto número de campos.

Ejercicio 1.6 — DOM vs SAX

Dificultad: ★★

Genera un fichero XML con 200 000 libros aleatorios. Después, escribe dos programas que calculen el precio medio de los libros publicados en cada década: uno con ElementTree (`parse`) y otro con SAX. Mide el tiempo y la memoria máxima de cada uno (`time.perf_counter` y `tracemalloc`) y explica los resultados en un breve informe. Como ampliación, prueba `ElementTree.iterparse`, que combina ambos enfoques.

Ejercicio 1.7 — Validación y XPath

Dificultad: ★★

Escribe el esquema XSD de `biblioteca.xml` (el ISBN con un patrón, el año entre 1450 y el año actual, el atributo `prestado` con valores `si`/`no`). Con `lxml`: valida varios documentos, mostrando los errores con su número de línea, y resuelve con XPath las consultas: títulos de los libros prestados; autor del libro más caro; número de libros por autor; precio medio.

Ejercicio 1.8 — XSLT

Dificultad: ★★★

Escribe dos hojas XSLT para `biblioteca.xml`: una que genere una página HTML con los libros agrupados por estado (prestados / disponibles) y otra que genere un fichero CSV (`xsl:output method="text"`). Aplica ambas desde Python con `lxml`, pasando como parámetro de la transformación el año mínimo de los libros a incluir.

Proyecto del tema: Agenda persistente multiformato

Amplía la agenda del proyecto del Tema 0 para que los contactos se guarden en ficheros.
Requisitos:

1. Define una clase abstracta `AlmacenContactos` (módulo `abc`) con los métodos `cargar() -> list[Contacto]` y `guardar(contactos)`.

2. Implementa cuatro almacenes: `AlmacenCSV`, `AlmacenJSON`, `AlmacenXML` (con ElementTree o xsdata) y `AlmacenBinario` (registros de longitud fija con `struct`).
3. La clase `Agenda` recibe el almacén en su constructor (no sabe qué formato se usa). El formato se elige en un fichero `config.json` o con un argumento de línea de comandos.
4. Opción de **exportar** la agenda a otro formato y de **importar** contactos desde un CSV, informando de las filas que no se pudieron importar y por qué.
5. Generación de un listado en HTML a partir del XML mediante una hoja XSLT.
6. Copias de seguridad: cada vez que se guarda, la versión anterior se copia a `backup/` con la fecha y hora en el nombre, conservando sólo las 5 más recientes.
7. Todas las excepciones de E/S se tratan; el programa nunca termina con una traza.
8. Pruebas con `pytest` (usando `tmp_path`) que comprueben que cada almacén guarda y recupera exactamente los mismos contactos.

Entrega: código, pruebas, XSD de la agenda y `README.md` con instrucciones y una explicación razonada de las ventajas e inconvenientes de cada formato.

TEMA 2

Manejo de conectores

En este tema aprenderás a…

- Comprender el desfase objeto-relacional y cómo afecta al diseño de las aplicaciones.
- Conocer los protocolos y conectores de acceso a bases de datos, en especial la especificación DB-API de Python.
- Conectar con un gestor embebido (SQLite) y con uno cliente-servidor (PostgreSQL).
- Ejecutar sentencias de definición y modificación de datos y consultas de forma segura, con parámetros.
- Recoger los resultados de una consulta en objetos del programa.
- Ejecutar funciones y procedimientos almacenados y gestionar transacciones.

Resultado de aprendizaje asociado: **RA2**.

2.1 El desfase objeto-relacional

Nuestros programas están escritos con un lenguaje **orientado a objetos**, pero los datos se guardan casi siempre en bases de datos **relacionales**, que organizan la información en tablas. Ambos modelos representan la realidad de forma distinta, y a esa diferencia se la llama **desfase objeto-relacional** (*object-relational impedance mismatch*).

Aspecto	Modelo de objetos	Modelo relacional
Estructura	Clases con atributos y métodos.	Tablas con columnas; sin comportamiento.
Tipos	Tipos ricos: listas, diccionarios, objetos anidados, enumerados.	Tipos atómicos (primera forma normal): no hay listas dentro de una celda.
Identidad	Cada objeto tiene identidad propia (`id(obj)`, `is`).	Un registro se identifica por el valor de su clave primaria.
Relaciones	Referencias directas entre objetos, en uno o ambos sentidos.	Claves ajenas; las relaciones N:M necesitan una tabla intermedia.
Navegación	Se recorre el grafo: `pedido.cliente.direccion`.	Se combinan tablas con `JOIN`.
Herencia	Nativa.	No existe; hay que simularla.
Encapsulación	Atributos internos y métodos.	Todos los datos son visibles.

Por ejemplo, un objeto `Pedido` con una lista de `Linea` y una referencia a su `Cliente` se convierte en tres tablas relacionadas por claves ajenas. Para guardar el pedido hay que

"desmontar" el objeto en filas, y para recuperarlo hay que ejecutar consultas y "volver a montarlo".

```python
@dataclass
class Cliente:
    id: int
    nombre: str

@dataclass
class Linea:
    producto: str
    cantidad: int
    precio: float

@dataclass
class Pedido:
    id: int
    cliente: Cliente              # referencia a otro objeto
    lineas: list[Linea]           # colección
```

```sql
CREATE TABLE cliente (id INTEGER PRIMARY KEY, nombre TEXT NOT NULL);
CREATE TABLE pedido  (id INTEGER PRIMARY KEY,
                      cliente_id INTEGER NOT NULL REFERENCES cliente(id));
CREATE TABLE linea   (pedido_id INTEGER REFERENCES pedido(id),
                      num INTEGER, producto TEXT, cantidad INTEGER, precio REAL,
                      PRIMARY KEY (pedido_id, num));
```

Este tema trata de cómo hacer esa traducción **a mano** usando conectores y SQL. En el Tema 3 veremos cómo las herramientas ORM la automatizan, y en el Tema 4, cómo las bases de datos objeto-relacionales y orientadas a objetos intentan reducir el desfase desde el lado de la base de datos.

2.2 Protocolos de acceso a bases de datos y conectores

Cada gestor de bases de datos (SGBD) tiene su propio protocolo de red y su propia forma de comunicarse con los clientes. Un **conector** (o *driver*) es una biblioteca que implementa ese protocolo y ofrece al programador una API para conectarse, enviar sentencias SQL y recibir los resultados.

Para no tener que aprender una API distinta para cada SGBD, existen **especificaciones estándar** que los conectores implementan:

ODBC (*Open Database Connectivity*). Estándar de Microsoft, independiente del lenguaje, escrito en C. Muy extendido en Windows. En Python se usa con `pyodbc`.

JDBC (*Java Database Connectivity*). El estándar de Java.

ADO.NET. El estándar de la plataforma .NET.

DB-API 2.0 (PEP 249). La especificación de Python. Define los objetos `Connection` y `Cursor`, sus métodos (`execute`, `fetchone`, `commit`…) y una jerarquía común de excepciones. Todos los conectores de Python la siguen, por lo que el código cambia muy poco de un SGBD a otro.

SGBD	Conector (paquete que se instala con `pip`)	Marcadores
SQLite	`sqlite3` (incluido en Python)	`?` y `:nombre`
PostgreSQL	`psycopg` versión 3 (`psycopg[binary]`)	`%s` y `%(nombre)s`
MySQL / MariaDB	`mysql.connector` (`mysql-connector-python`)	`%s` y `%(nombre)s`
Oracle	`oracledb`	`:1` y `:nombre`
SQL Server	`pyodbc`	`?`

2.2.1 Ventajas e inconvenientes de usar conectores

+ Control total sobre el SQL que se ejecuta: se pueden optimizar las consultas y aprovechar características propias del SGBD.
+ Rendimiento máximo y poca "magia": lo que se escribe es lo que se ejecuta.
+ Pocas dependencias y curva de aprendizaje baja si se conoce SQL.
– Hay que escribir mucho código repetitivo para convertir filas en objetos y viceversa (el desfase objeto-relacional se resuelve a mano).
– El SQL queda mezclado con el código y, si usa características propias de un SGBD, cuesta cambiar de gestor.
– Los errores en el SQL sólo se detectan al ejecutar.

2.2.2 Gestores embebidos y gestores cliente-servidor

Embebidos. El motor de base de datos es una biblioteca que se ejecuta **dentro del propio proceso** de la aplicación, y la base de datos es un fichero local. No hay servidor, ni usuarios, ni instalación. Ejemplos: **SQLite**, H2, DuckDB. Son ideales para aplicaciones de escritorio y móviles (Android e iOS incluyen SQLite), pruebas y prototipos.

Cliente-servidor (independientes). El SGBD es un proceso independiente, normalmente en otra máquina, al que se conectan muchos clientes por red. Gestionan usuarios, permisos y concurrencia elevada. Ejemplos: **PostgreSQL**, MySQL/MariaDB, Oracle, SQL Server.

En este tema usaremos los dos: **SQLite** (no requiere instalar nada) y **PostgreSQL** (en un contenedor Docker; consulta el Apéndice A).

2.3 Establecimiento de conexiones

2.3.1 SQLite

```python
import sqlite3

conexion = sqlite3.connect("instituto.db")   # crea el fichero si no existe
# conexion = sqlite3.connect(":memory:")     # base de datos en memoria
print(sqlite3.sqlite_version)
conexion.execute("PRAGMA foreign_keys = ON") # SQLite no las comprueba por defecto
conexion.close()
```

2.3.2 PostgreSQL

```
import psycopg

conexion = psycopg.connect(
    host="localhost", port=5432,
    dbname="instituto", user="dam", password="dam",
)
print(conexion.info.server_version)
conexion.close()
```

No escribas las contraseñas en el código. Los datos de conexión deben leerse de la configuración (variables de entorno, un fichero `.env` o un fichero de configuración que no se sube al repositorio).

```
import os
import psycopg

def conectar() -> psycopg.Connection:
    return psycopg.connect(
        host=os.getenv("DB_HOST", "localhost"),
        port=int(os.getenv("DB_PORT", "5432")),
        dbname=os.getenv("DB_NAME", "instituto"),
        user=os.getenv("DB_USER", "dam"),
        password=os.environ["DB_PASSWORD"],     # obligatoria
    )
```

2.3.3 Cerrar las conexiones

Una conexión consume recursos tanto en el cliente como en el servidor (que admite un número limitado de conexiones simultáneas). Debe cerrarse cuando ya no se necesita. Lo más seguro es usar `with`:

```
# psycopg: al salir del with se hace commit (o rollback si hubo error)
# y se CIERRA la conexión
with psycopg.connect(conninfo) as conn:
    ...

# sqlite3: al salir del with se hace commit/rollback pero NO se cierra
from contextlib import closing
with closing(sqlite3.connect("instituto.db")) as conn:
    with conn:            # transacción
        ...
```

Pool de conexiones. Abrir una conexión con un servidor es una operación costosa. Las aplicaciones con muchos usuarios (por ejemplo, servidores web) mantienen un **pool**: un conjunto de conexiones abiertas que se reutilizan. Para psycopg existe el paquete `psycopg-pool`.

2.4 Ejecución de sentencias de descripción de datos (DDL)

Las sentencias DDL (`CREATE`, `ALTER`, `DROP`) definen la estructura de la base de datos. Se ejecutan como cualquier otra sentencia, a través de un **cursor**, que es el objeto que envía las sentencias y recibe los resultados.

```
import sqlite3

ESQUEMA = """
CREATE TABLE IF NOT EXISTS curso (
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre  TEXT NOT NULL UNIQUE
);
CREATE TABLE IF NOT EXISTS alumno (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    dni        TEXT NOT NULL UNIQUE,
    nombre     TEXT NOT NULL,
    apellidos  TEXT NOT NULL,
    nacimiento TEXT,
    curso_id   INTEGER REFERENCES curso(id) ON DELETE SET NULL
);
"""

with sqlite3.connect("instituto.db") as conn:
    conn.executescript(ESQUEMA)      # varias sentencias separadas por ';'
```

En PostgreSQL los tipos y la sintaxis cambian ligeramente:

```
CREATE TABLE IF NOT EXISTS alumno (
    id         SERIAL PRIMARY KEY,           -- o INTEGER GENERATED ALWAYS AS IDENTITY
    dni        VARCHAR(9) NOT NULL UNIQUE,
    nombre     VARCHAR(50) NOT NULL,
    apellidos  VARCHAR(100) NOT NULL,
    nacimiento DATE,
    curso_id   INTEGER REFERENCES curso(id) ON DELETE SET NULL
);
```

```
with psycopg.connect(conninfo) as conn:
    conn.execute(open("esquema_pg.sql", encoding="utf-8").read())
```

2.5 Ejecución de sentencias de modificación de datos (DML)

2.5.1 Sentencias parametrizadas

Inyección SQL. **Nunca** construyas una sentencia SQL concatenando o formateando valores introducidos por el usuario:

```
dni = input("DNI: ")
cur.execute(f"DELETE FROM alumno WHERE dni = '{dni}'")   # !`!`MAL!!
```

Si el usuario escribe `' OR '1'='1`, la sentencia resultante borra **todos** los alumnos. Este ataque se llama **inyección SQL** y sigue estando entre las vulnerabilidades más

explotadas.

La forma correcta es usar **marcadores de parámetro** y pasar los valores aparte. El conector los envía al servidor de forma segura y además se encarga de convertir los tipos (fechas, `None` → `NULL`, etc.).

```
# SQLite: marcador ?
cur.execute("INSERT INTO alumno (dni, nombre, apellidos) VALUES (?, ?, ?)",
            ("12345678Z", "Ana", "García"))
# SQLite: marcadores con nombre
cur.execute("UPDATE alumno SET curso_id = :curso WHERE dni = :dni",
            {"curso": 2, "dni": "12345678Z"})

# PostgreSQL / MySQL: marcador %s (¡aunque el valor no sea una cadena!)
cur.execute("INSERT INTO alumno (dni, nombre, apellidos, nacimiento) "
            "VALUES (%s, %s, %s, %s)",
            ("12345678Z", "Ana", "García", date(2005, 3, 14)))
```

Nota. Los parámetros siempre se pasan como una **secuencia** (tupla o lista) o un diccionario. Para un único parámetro hay que escribir la tupla con coma: `(dni,)`. Los marcadores sólo sirven para *valores*, no para nombres de tablas o columnas. Si necesitas componer identificadores dinámicamente en PostgreSQL, usa `psycopg.sql.Identifier`.

2.5.2 Inserciones, modificaciones y borrados

```
import sqlite3

with sqlite3.connect("instituto.db") as conn:
    cur = conn.cursor()

    cur.execute("INSERT INTO curso (nombre) VALUES (?)", ("2º DAM",))
    curso_id = cur.lastrowid                # id generado automáticamente

    alumnos = [
        ("11111111H", "Ana", "García", "2005-03-14", curso_id),
        ("22222222J", "Luis", "Pérez", "2004-11-02", curso_id),
    ]
    cur.executemany(
        "INSERT INTO alumno (dni, nombre, apellidos, nacimiento, curso_id) "
        "VALUES (?, ?, ?, ?, ?)", alumnos)
    print(cur.rowcount, "alumnos insertados")

    cur.execute("UPDATE alumno SET apellidos = ? WHERE dni = ?",
                ("García López", "11111111H"))
    if cur.rowcount == 0:
        print("No existe ese alumno")

    cur.execute("DELETE FROM alumno WHERE dni = ?", ("22222222J",))
    cur.close()
# al salir del with: commit
```

En PostgreSQL (y en SQLite desde la versión 3.35) se puede usar `RETURNING` para obtener los valores generados:

```
cur.execute("INSERT INTO curso (nombre) VALUES (%s) RETURNING id", ("1º DAM",))
nuevo_id = cur.fetchone()[0]
```

2.6 Ejecución de consultas

2.6.1 Recuperar filas

Tras ejecutar un `SELECT`, el cursor permite obtener las filas resultantes:

`fetchone()`	Devuelve la siguiente fila (una tupla) o `None` si no quedan.
`fetchmany(n)`	Devuelve una lista con hasta `n` filas.
`fetchall()`	Devuelve una lista con todas las filas restantes.
`for fila in cur`	Itera sobre las filas una a una (recomendado para muchas filas).
`cur.description`	Metadatos de las columnas del resultado (nombre, tipo...).

```
with closing(sqlite3.connect("instituto.db")) as conn:
    cur = conn.execute(
        "SELECT a.dni, a.nombre, a.apellidos, c.nombre "
        "FROM alumno a LEFT JOIN curso c ON a.curso_id = c.id "
        "WHERE a.apellidos LIKE ? ORDER BY a.apellidos", ("G%",))

    print([col[0] for col in cur.description])   # nombres de columnas
    for dni, nombre, apellidos, curso in cur:
        print(f"{dni} {apellidos}, {nombre} ({curso or 'sin curso'})")

    total = conn.execute("SELECT COUNT(*) FROM alumno").fetchone()[0]
```

2.6.2 Objetos para almacenar el resultado de las consultas

Trabajar con tuplas y acceder a las columnas por posición es frágil. Es mejor que cada fila se convierta en un objeto del dominio. Hay varias opciones:

Filas con acceso por nombre. En SQLite se configura `row_factory`:

```
conn.row_factory = sqlite3.Row
fila = conn.execute("SELECT * FROM alumno WHERE id = ?", (1,)).fetchone()
print(fila["nombre"], fila["apellidos"], dict(fila))
```

Fábricas de filas en psycopg. psycopg incluye varias en `psycopg.rows`; `class_row` construye directamente objetos de una clase:

```
from dataclasses import dataclass
from datetime import date
from psycopg.rows import dict_row, class_row

@dataclass
class Alumno:
```

```
    id: int
    dni: str
    nombre: str
    apellidos: str
    nacimiento: date | None
    curso_id: int | None

with psycopg.connect(conninfo) as conn:
    with conn.cursor(row_factory=class_row(Alumno)) as cur:
        cur.execute("SELECT * FROM alumno ORDER BY apellidos")
        alumnos: list[Alumno] = cur.fetchall()

    with conn.cursor(row_factory=dict_row) as cur:
        cur.execute("SELECT nombre, apellidos FROM alumno")
        for d in cur:
            print(d["nombre"])
```

2.6.3 El patrón DAO

Para no repartir SQL por toda la aplicación, se agrupa el acceso a cada entidad en una clase **DAO** (*Data Access Object*) o **repositorio**. El resto del programa sólo trabaja con objetos y no sabe qué SGBD hay detrás.

```
import sqlite3
from dataclasses import dataclass

@dataclass
class Alumno:
    dni: str
    nombre: str
    apellidos: str
    nacimiento: str | None = None
    curso_id: int | None = None
    id: int | None = None


class AlumnoDAO:
    def __init__(self, conn: sqlite3.Connection) -> None:
        self.conn = conn

    @staticmethod
    def _a_objeto(fila: sqlite3.Row) -> Alumno:
        return Alumno(**dict(fila))

    def insertar(self, a: Alumno) -> Alumno:
        cur = self.conn.execute(
            "INSERT INTO alumno (dni, nombre, apellidos, nacimiento, curso_id) "
            "VALUES (?, ?, ?, ?, ?)",
            (a.dni, a.nombre, a.apellidos, a.nacimiento, a.curso_id))
        a.id = cur.lastrowid
        return a

    def buscar_por_id(self, ident: int) -> Alumno | None:
        fila = self.conn.execute("SELECT * FROM alumno WHERE id = ?",
                                 (ident,)).fetchone()
        return self._a_objeto(fila) if fila else None
```

```
    def listar(self) -> list[Alumno]:
        cur = self.conn.execute("SELECT * FROM alumno ORDER BY apellidos, nombre")
        return [self._a_objeto(f) for f in cur]

    def actualizar(self, a: Alumno) -> bool:
        cur = self.conn.execute(
            "UPDATE alumno SET dni=?, nombre=?, apellidos=?, nacimiento=?, "
            "curso_id=? WHERE id=?",
            (a.dni, a.nombre, a.apellidos, a.nacimiento, a.curso_id, a.id))
        return cur.rowcount == 1

    def borrar(self, ident: int) -> bool:
        return self.conn.execute("DELETE FROM alumno WHERE id = ?",
                                 (ident,)).rowcount == 1


if __name__ == "__main__":
    conn = sqlite3.connect("instituto.db")
    conn.row_factory = sqlite3.Row
    dao = AlumnoDAO(conn)
    with conn:                                      # una transacción
        nuevo = dao.insertar(Alumno("33333333P", "Eva", "Martín"))
    print(dao.buscar_por_id(nuevo.id))
    conn.close()
```

2.7 Ejecución de procedimientos almacenados

Un **procedimiento almacenado** es un programa escrito en un lenguaje procedimental del SGBD (PL/pgSQL en PostgreSQL, PL/SQL en Oracle, SQL/PSM en MySQL) que se guarda y ejecuta **dentro del servidor**. Ventajas: reducen el tráfico de red, centralizan reglas de negocio y permiten conceder permisos de ejecución sin dar acceso directo a las tablas. Inconvenientes: atan la aplicación a un SGBD concreto y son más difíciles de versionar y probar.

> **Nota.** SQLite no tiene procedimientos almacenados: al ser embebido, la lógica se escribe directamente en la aplicación. Para este apartado usaremos PostgreSQL.

PostgreSQL distingue entre **funciones** (devuelven un valor o un conjunto de filas y se usan dentro de un `SELECT`) y **procedimientos** (se invocan con `CALL` y pueden gestionar transacciones).

```
-- Función que devuelve un valor
CREATE OR REPLACE FUNCTION num_alumnos(p_curso INTEGER)
RETURNS INTEGER LANGUAGE sql AS $$
    SELECT COUNT(*) FROM alumno WHERE curso_id = p_curso;
$$;

-- Función que devuelve filas
CREATE OR REPLACE FUNCTION alumnos_de(p_curso TEXT)
RETURNS TABLE (dni VARCHAR, nombre VARCHAR, apellidos VARCHAR)
LANGUAGE sql AS $$
    SELECT a.dni, a.nombre, a.apellidos
    FROM alumno a JOIN curso c ON a.curso_id = c.id
    WHERE c.nombre = p_curso;
```

```
$$;

-- Procedimiento con parámetro de salida
CREATE OR REPLACE PROCEDURE matricular(p_dni VARCHAR, p_curso TEXT,
                                       INOUT p_resultado TEXT DEFAULT NULL)
LANGUAGE plpgsql AS $$
DECLARE v_curso INTEGER;
BEGIN
    SELECT id INTO v_curso FROM curso WHERE nombre = p_curso;
    IF v_curso IS NULL THEN
        RAISE EXCEPTION 'No existe el curso %', p_curso;
    END IF;
    UPDATE alumno SET curso_id = v_curso WHERE dni = p_dni;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe el alumno %', p_dni;
    END IF;
    p_resultado := 'Alumno ' || p_dni || ' matriculado en ' || p_curso;
END;
$$;
```

Desde Python se invocan con sentencias normales:

```
import psycopg

with psycopg.connect(conninfo) as conn:
    n = conn.execute("SELECT num_alumnos(%s)", (1,)).fetchone()[0]

    for dni, nombre, apellidos in conn.execute(
            "SELECT * FROM alumnos_de(%s)", ("2º DAM",)):
        print(dni, nombre, apellidos)

    try:
        fila = conn.execute("CALL matricular(%s, %s, NULL)",
                            ("11111111H", "2º DAM")).fetchone()
        print(fila[0])        # valor del parámetro INOUT
    except psycopg.errors.RaiseException as e:
        print("Error del procedimiento:", e.diag.message_primary)
```

Nota. En MySQL, el conector `mysql-connector-python` ofrece además el método `cursor.callproc("nombre", args)` que define la DB-API para llamar a procedimientos.

2.8 Gestión de transacciones

Una **transacción** es un conjunto de operaciones que se ejecutan como una unidad: o se realizan todas, o no se realiza ninguna. El ejemplo clásico es una transferencia bancaria: restar de una cuenta y sumar en otra no puede quedarse a medias. Las transacciones garantizan las propiedades **ACID**:

Atomicidad. Todo o nada.

Consistencia. La base de datos pasa de un estado válido a otro estado válido (se respetan las restricciones).

Aislamiento. Las transacciones concurrentes no ven los cambios a medio hacer de las demás

(según el nivel de aislamiento).

Durabilidad. Una vez confirmada, la transacción persiste aunque falle el sistema.

2.8.1 Transacciones con la DB-API

Según la DB-API, una conexión **no está en modo autocommit**: la primera sentencia inicia implícitamente una transacción, que dura hasta que se llama a `commit()` (confirmar) o `rollback()` (deshacer).

```
def transferir(conn, origen: str, destino: str, cantidad: float) -> None:
    cur = conn.cursor()
    try:
        cur.execute("UPDATE cuenta SET saldo = saldo - ? WHERE iban = ?",
                    (cantidad, origen))
        cur.execute("SELECT saldo FROM cuenta WHERE iban = ?", (origen,))
        if cur.fetchone()[0] < 0:
            raise ValueError("Saldo insuficiente")
        cur.execute("UPDATE cuenta SET saldo = saldo + ? WHERE iban = ?",
                    (cantidad, destino))
        if cur.rowcount != 1:
            raise ValueError("La cuenta destino no existe")
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        cur.close()
```

El mismo comportamiento se consigue de forma más compacta con `with conn:` en `sqlite3`, que hace `commit` si el bloque termina bien y `rollback` si se lanza una excepción.

> **El modo de transacciones de `sqlite3`.** Por compatibilidad histórica, el módulo `sqlite3` no inicia transacciones antes de las sentencias DDL y tiene un comportamiento algo peculiar. Desde Python 3.12 se recomienda abrir la conexión con `sqlite3.connect("bd.db", autocommit=False)`, que sigue exactamente la DB-API. Con `autocommit=True` cada sentencia se confirma inmediatamente.

2.8.2 Transacciones en psycopg

psycopg ofrece el bloque `conn.transaction()`, que además permite anidar transacciones mediante *savepoints*: si falla el bloque interior, sólo se deshace esa parte.

```
with psycopg.connect(conninfo) as conn:
    with conn.transaction():
        conn.execute("INSERT INTO curso (nombre) VALUES (%s)", ("1º ASIR",))
        for dni, nombre, apellidos in nuevos_alumnos:
            try:
                with conn.transaction():          # SAVEPOINT
                    conn.execute(
                        "INSERT INTO alumno (dni, nombre, apellidos) "
                        "VALUES (%s, %s, %s)", (dni, nombre, apellidos))
            except psycopg.errors.UniqueViolation:
                print(f"DNI duplicado: {dni}; se omite")
    # aquí se confirma la transacción exterior
```

2.8.3 Niveles de aislamiento

`READ UNCOMMITTED`	Puede leer datos no confirmados por otras transacciones (lecturas sucias).
`READ COMMITTED`	Sólo lee datos confirmados. Es el nivel por defecto en PostgreSQL.
`REPEATABLE READ`	Si se relee una fila, se obtiene el mismo valor durante toda la transacción.
`SERIALIZABLE`	Equivale a ejecutar las transacciones una detrás de otra. Máxima seguridad, menor concurrencia.

```
from psycopg import IsolationLevel
conn.isolation_level = IsolationLevel.SERIALIZABLE
```

2.9 Excepciones de la DB-API

Todos los conectores definen la misma jerarquía de excepciones, accesible como atributos del módulo (`sqlite3.IntegrityError`, `psycopg.IntegrityError`...):

```
Exception
 └── Error
      ├── InterfaceError        error del propio conector
      └── DatabaseError         error de la base de datos
           ├── DataError        valor fuera de rango, tipo incorrecto...
           ├── OperationalError conexión perdida, BD no encontrada, bloqueo...
           ├── IntegrityError   clave duplicada, clave ajena, NOT NULL...
           ├── InternalError
           ├── ProgrammingError tabla inexistente, error de sintaxis SQL...
           └── NotSupportedError
```

```
try:
    with conn:
        dao.insertar(alumno)
except sqlite3.IntegrityError as e:
    print("No se pudo dar de alta: ya existe un alumno con ese DNI")
except sqlite3.OperationalError as e:
    print("Problema con la base de datos:", e)
```

psycopg define además subclases específicas para cada código de error de PostgreSQL en `psycopg.errors` (`UniqueViolation`, `ForeignKeyViolation`, `UndefinedTable`...).

2.10 Ejercicios

Ejercicio 2.1 — Primeros pasos con SQLite

Dificultad: ★

Crea una base de datos SQLite `biblioteca.db` con las tablas `autor`, `libro` (con clave ajena a autor) y `socio`. Escribe un programa que cree las tablas si no existen, inserte datos de ejemplo con `executemany` y muestre: todos los libros con el nombre de su autor; el número de libros por autor; los autores sin libros. Muestra los nombres de las columnas usando `cursor.description`.

Ejercicio 2.2 — Inyección SQL

Dificultad: ★

Escribe una función de *login* vulnerable que compruebe usuario y contraseña construyendo el SQL con una f-string, y demuestra que se puede entrar sin conocer la contraseña. Corrígela usando parámetros y almacenando las contraseñas con un *hash* (`hashlib.scrypt` o `hashlib.pbkdf2_hmac` con sal). Explica en un comentario por qué la versión corregida es segura.

Ejercicio 2.3 — De CSV a base de datos

Dificultad: ★★

Descarga un conjunto de datos abiertos en CSV (por ejemplo, del portal de datos abiertos de la Junta de Castilla y León) e impórtalo en SQLite: el programa debe crear la tabla deduciendo los nombres de columna de la cabecera, insertar los datos en una única transacción con `executemany` e informar de cuántas filas se han insertado y cuánto ha tardado. Compara el tiempo con el de hacer un `commit` por fila.

Ejercicio 2.4 — Mismo DAO, dos gestores

Dificultad: ★★

Implementa un DAO para la tabla `libro` del ejercicio 2.1 con una versión para SQLite y otra para PostgreSQL que compartan la misma interfaz (clase abstracta). El programa principal debe funcionar con cualquiera de las dos cambiando sólo una línea de configuración. Anota qué diferencias has tenido que resolver entre ambos gestores.

Ejercicio 2.5 — Procedimientos almacenados

Dificultad: ★★

En PostgreSQL, crea para la base de datos de la biblioteca: una función `libros_disponibles(autor TEXT)` que devuelva una tabla; y un procedimiento `prestar(isbn, id_socio)` que compruebe que el libro está disponible y que el socio no tiene más de 3 préstamos activos, registre el préstamo y lance una excepción con un mensaje claro en otro caso. Llámalos desde Python y trata las excepciones mostrando el mensaje del servidor.

Ejercicio 2.6 — Transacciones

Dificultad: ★★★

Crea una tabla `cuenta(iban, titular, saldo)` con la restricción `CHECK (saldo >= 0)` e

implementa `transferir(origen, destino, cantidad)`. Demuestra con dos programas ejecutándose a la vez (o dos hilos) qué ocurre si dos transferencias concurrentes leen el saldo antes de modificarlo. Soluciónalo usando `SELECT ... FOR UPDATE` o el nivel de aislamiento `SERIALIZABLE` con reintento, y explica la diferencia entre ambas soluciones.

Proyecto del tema: Gestión de un videoclub (o plataforma de alquiler)

Desarrolla una aplicación de consola para gestionar el alquiler de películas.
Modelo de datos: `cliente`, `pelicula`, `copia` (cada película tiene varias copias físicas o licencias), `alquiler` (copia, cliente, fecha de alquiler, fecha prevista de devolución, fecha real de devolución, importe).
Requisitos:

1. Script SQL de creación con todas las restricciones (claves primarias, ajenas, `NOT NULL`, `UNIQUE`, `CHECK`).
2. Capa de acceso a datos con un DAO por entidad y *dataclasses* para el modelo. Los DAO nunca hacen `commit`: las transacciones se gestionan en una capa de servicio.
3. Operaciones: CRUD de clientes y películas; alquilar (comprueba que haya copia disponible y que el cliente no tenga alquileres vencidos); devolver (calcula el recargo por retraso); listados de películas disponibles, alquileres vencidos y ranking de películas más alquiladas.
4. Las operaciones de alquiler y devolución son transaccionales.
5. La aplicación funciona con SQLite **y** con PostgreSQL. En PostgreSQL, la devolución se implementa como procedimiento almacenado.
6. Configuración de la conexión en un fichero `.env` o `config.toml` (`tomllib`) que no se incluye en el repositorio.
7. Exportación del historial de alquileres de un cliente a CSV y JSON (reutilizando lo aprendido en el Tema 1).
8. Pruebas con `pytest` sobre una base de datos SQLite en memoria.

TEMA 3

Herramientas de mapeo objeto-relacional (ORM)

En este tema aprenderás a…

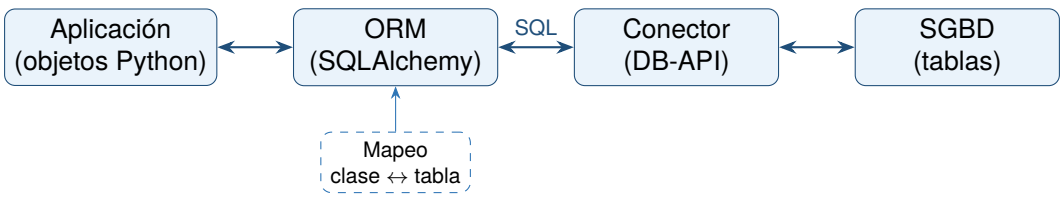
- Explicar qué es un ORM, qué problema resuelve y cuáles son las herramientas más utilizadas.
- Instalar y configurar SQLAlchemy y conectarlo con SQLite y PostgreSQL.
- Definir el mapeo de clases a tablas, incluyendo colecciones, relaciones y herencia.
- Comprender las sesiones y los estados de un objeto persistente.
- Cargar, guardar, modificar y borrar objetos; consultar con SQL y con el lenguaje propio del ORM.
- Gestionar transacciones con el ORM.

Resultado de aprendizaje asociado: **RA3**.

3.1 Concepto de mapeo objeto-relacional

En el tema anterior resolvimos el desfase objeto-relacional a mano: cada DAO contenía el SQL necesario y el código para convertir filas en objetos. Un **ORM** (*Object-Relational Mapping*) es una herramienta que automatiza esa traducción. A partir de una descripción de **cómo se corresponden las clases con las tablas** (el *mapeo*), el ORM:

- Genera las sentencias SQL para insertar, modificar, borrar y recuperar objetos.
- Convierte los tipos de datos en ambos sentidos.
- Traduce las relaciones entre objetos (referencias y colecciones) en claves ajenas y `JOIN`.
- Lleva la cuenta de qué objetos se han modificado para guardar sólo los cambios.
- Permite escribir consultas en términos de clases y atributos en lugar de tablas y columnas.



3.1.1 Ventajas e inconvenientes

+ Mucho menos código repetitivo: desaparecen los DAO escritos a mano.
+ Se programa con objetos; el modelo de dominio es el centro de la aplicación.

+ Independencia del SGBD: el ORM genera el SQL de cada dialecto.
+ Funcionalidades "gratis": caché de objetos, carga perezosa, control de cambios, protección frente a inyección SQL.
– Curva de aprendizaje: hay que entender cómo y cuándo el ORM ejecuta SQL.
– Riesgo de consultas ineficientes si no se controla (el famoso problema **N+1**).
– Sobrecarga de rendimiento en operaciones masivas.
– Algunas consultas complejas o características específicas del SGBD siguen requiriendo SQL.

3.2 Características de las herramientas ORM

Los ORM se suelen clasificar según el patrón que implementan:

Active Record. Cada objeto sabe guardarse a sí mismo (`alumno.save()`). Sencillo, pero mezcla el modelo con la persistencia. Ejemplos: Django ORM, Peewee, Ruby on Rails.

Data Mapper. Los objetos del modelo no saben nada de la base de datos; un componente aparte (la **sesión**) se encarga de guardarlos. Más flexible. Ejemplos: SQLAlchemy, Hibernate/JPA, Doctrine.

Lenguaje	ORM	Comentario
Python	**SQLAlchemy**	El ORM de referencia en Python; Data Mapper. Lo usaremos en el curso.
Python	Django ORM	Integrado en el framework web Django; Active Record.
Python	SQLModel, Peewee, Pony, Tortoise	Alternativas más ligeras o asíncronas. SQLModel está construido sobre SQLAlchemy.
Java	Hibernate / JPA	Estándar *de facto* en Java. Lenguaje de consultas HQL/JPQL.
C#	Entity Framework Core	ORM oficial de .NET. Consultas con LINQ.
PHP	Doctrine, Eloquent	Symfony y Laravel, respectivamente.
JS/TS	Prisma, TypeORM, Sequelize	Ecosistema Node.js.

3.3 Instalación de SQLAlchemy

```
$ pip install sqlalchemy               # el ORM
$ pip install "psycopg[binary]"        # el conector de PostgreSQL (si se usa)
$ python -c "import sqlalchemy; print(sqlalchemy.__version__)"
```

SQLAlchemy no sustituye al conector: lo usa por debajo. Para SQLite no hace falta instalar nada más.

> **Nota.** Todos los ejemplos usan la API de **SQLAlchemy 2.x**. En Internet encontrarás mucho código con el estilo antiguo (1.x: `session.query(...)`, `Column(...)` sin `Mapped`); sigue

> funcionando, pero no lo mezcles con el nuevo.

3.4 Configuración de la herramienta

3.4.1 El motor (*engine*)

El objeto `Engine` representa la base de datos: sabe qué dialecto de SQL usar, qué conector y cómo obtener conexiones (gestiona un *pool*). Se crea una única vez en la aplicación a partir de una **URL de conexión**:

```
dialecto+conector://usuario:contraseña@servidor:puerto/basededatos
```

```python
from sqlalchemy import create_engine

engine = create_engine("sqlite:///instituto.db", echo=True)
engine = create_engine("sqlite:///:memory:")
engine = create_engine("postgresql+psycopg://dam:dam@localhost:5432/instituto")
engine = create_engine("mysql+mysqlconnector://dam:dam@localhost/instituto")
```

Propiedades configurables más habituales de `create_engine`:

`echo=True`	Muestra por consola todo el SQL generado. **Muy recomendable mientras se aprende.**
`pool_size`, `max_overflow`	Tamaño del pool de conexiones y conexiones extra permitidas.
`pool_pre_ping=True`	Comprueba que la conexión sigue viva antes de usarla.
`pool_recycle`	Segundos tras los que una conexión se renueva.
`isolation_level`	Nivel de aislamiento por defecto de las transacciones.
`connect_args`	Diccionario de argumentos que se pasan al conector.

3.4.2 Fichero de configuración

Igual que con los conectores, los datos de conexión deben estar fuera del código. Una opción sencilla es un fichero TOML, que Python lee con `tomllib`:

```
# config.toml
[database]
url = "postgresql+psycopg://dam:dam@localhost:5432/instituto"
echo = false
pool_size = 5
```

```python
# basedatos.py
import tomllib
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

with open("config.toml", "rb") as f:
    cfg = tomllib.load(f)["database"]

engine = create_engine(cfg["url"], echo=cfg.get("echo", False),
```

```
                        pool_size=cfg.get("pool_size", 5))
Session = sessionmaker(bind=engine)     # fábrica de sesiones
```

3.5 Clases persistentes y estructura del mapeo

Una **clase persistente** es una clase cuyos objetos se pueden guardar en la base de datos. El **mapeo** indica a qué tabla corresponde, qué atributo corresponde a cada columna, cuál es la clave primaria y cómo se relaciona con otras clases.

Históricamente, herramientas como Hibernate describían el mapeo en **ficheros XML** separados (`Alumno.hbm.xml`); hoy lo habitual es describirlo en el propio código, con anotaciones (Java) o, en Python, con **anotaciones de tipo**. SQLAlchemy admite ambos estilos.

3.5.1 Mapeo declarativo

```python
# modelo.py
from datetime import date
from sqlalchemy import String, ForeignKey, CheckConstraint
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    """Clase base de todas las clases persistentes."""

class Alumno(Base):
    __tablename__ = "alumno"
    __table_args__ = (CheckConstraint("length(dni) = 9"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    dni: Mapped[str] = mapped_column(String(9), unique=True)
    nombre: Mapped[str] = mapped_column(String(50))
    apellidos: Mapped[str] = mapped_column(String(100))
    nacimiento: Mapped[date | None]                 # nullable por ser Optional
    email: Mapped[str | None] = mapped_column(String(120), index=True)
    activo: Mapped[bool] = mapped_column(default=True)

    def __repr__(self) -> str:
        return f"Alumno(id={self.id}, dni={self.dni!r}, nombre={self.nombre!r})"
```

Elementos del mapeo:

`__tablename__`	Nombre de la tabla.
`__table_args__`	Restricciones e índices a nivel de tabla.
`Mapped[tipo]`	Declara un atributo persistente. El tipo Python determina el tipo SQL (`int` → `INTEGER`, `str` → `VARCHAR`, `date` → `DATE`...). `tipo | None` significa que admite `NULL`.
`mapped_column(...)`	Opciones de la columna: `primary_key`, `unique`, `index`, `nullable`, `default`, `server_default`, `name` (si la columna se llama distinto), tipo SQL explícito (`String(50)`, `Numeric(10, 2)`...).
`ForeignKey("tabla.col")`	Clave ajena.
`relationship()`	Relación con otra clase (siguiente apartado).

Una vez definidas las clases, el ORM puede crear las tablas:

```
from basedatos import engine
from modelo import Base

Base.metadata.create_all(engine)    # CREATE TABLE ... para las que no existan
# Base.metadata.drop_all(engine)    # DROP TABLE de todas
```

3.5.2 Mapeo imperativo

Si se prefiere mantener las clases del dominio totalmente independientes del ORM (como ocurría con los ficheros de mapeo XML), se describe la tabla por separado y se asocia a la clase:

```
from dataclasses import dataclass
from sqlalchemy import Table, Column, Integer, String, MetaData
from sqlalchemy.orm import registry

@dataclass
class Curso:                        # clase "limpia", no sabe nada del ORM
    nombre: str
    id: int | None = None

metadata = MetaData()
tabla_curso = Table("curso", metadata,
    Column("id", Integer, primary_key=True),
    Column("nombre", String(30), nullable=False, unique=True))

registry().map_imperatively(Curso, tabla_curso)   # el "fichero de mapeo"
```

Migraciones. `create_all` sólo crea tablas que no existen: si cambias una clase ya mapeada, no modifica la tabla. En proyectos reales se usa **Alembic**, la herramienta de migraciones de SQLAlchemy, que genera los `ALTER TABLE` necesarios y mantiene un historial de versiones del esquema.

3.6 Mapeo de relaciones y colecciones

3.6.1 Uno a muchos / muchos a uno

Un curso tiene muchos alumnos; un alumno pertenece a un curso. En la base de datos, la clave ajena está en `alumno`. En los objetos, el curso tiene una **colección** de alumnos y cada alumno una **referencia** a su curso:

```
from sqlalchemy import ForeignKey, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

class Curso(Base):
    __tablename__ = "curso"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str] = mapped_column(String(30), unique=True)

    alumnos: Mapped[list["Alumno"]] = relationship(
```

```
        back_populates="curso",
        cascade="all, delete-orphan",       # ver nota
        order_by="Alumno.apellidos")

class Alumno(Base):
    __tablename__ = "alumno"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str] = mapped_column(String(50))
    apellidos: Mapped[str] = mapped_column(String(100))
    curso_id: Mapped[int | None] = mapped_column(ForeignKey("curso.id"))

    curso: Mapped["Curso | None"] = relationship(back_populates="alumnos")
```

```
dam = Curso(nombre="2º DAM")
dam.alumnos.append(Alumno(nombre="Ana", apellidos="García"))
dam.alumnos.append(Alumno(nombre="Luis", apellidos="Pérez"))
session.add(dam)          # los alumnos se guardan en cascada
session.commit()
print(dam.alumnos[0].curso.nombre)    # navegación en ambos sentidos
```

Cascadas. `cascade` indica qué operaciones sobre el objeto padre se propagan a los hijos. `"all, delete-orphan"` hace que al guardar o borrar un curso se guarden o borren sus alumnos, y que un alumno que se quite de la colección se borre. Úsalo sólo cuando los hijos no tengan sentido sin el padre (por ejemplo, las líneas de un pedido).

3.6.2 Uno a uno

Se mapea como uno a muchos pero indicando que el lado de la colección es un único objeto:

```
class Expediente(Base):
    __tablename__ = "expediente"
    id: Mapped[int] = mapped_column(primary_key=True)
    alumno_id: Mapped[int] = mapped_column(ForeignKey("alumno.id"), unique=True)
    observaciones: Mapped[str | None]
    alumno: Mapped["Alumno"] = relationship(back_populates="expediente")

# En Alumno:
#   expediente: Mapped["Expediente | None"] = relationship(back_populates="alumno")
```

3.6.3 Muchos a muchos

Un alumno se matricula en muchos módulos y un módulo tiene muchos alumnos. Hace falta una tabla intermedia:

```
from sqlalchemy import Table, Column, ForeignKey

matricula = Table(
    "matricula", Base.metadata,
    Column("alumno_id", ForeignKey("alumno.id"), primary_key=True),
    Column("modulo_id", ForeignKey("modulo.id"), primary_key=True),
)
```

```python
class Modulo(Base):
    __tablename__ = "modulo"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str]
    horas: Mapped[int]
    alumnos: Mapped[list["Alumno"]] = relationship(
        secondary=matricula, back_populates="modulos")

# En Alumno:
#   modulos: Mapped[list["Modulo"]] = relationship(
#       secondary=matricula, back_populates="alumnos")
```

Si la relación tiene atributos propios (por ejemplo, la **nota** de cada alumno en cada módulo), la tabla intermedia se convierte en una clase, llamada **objeto de asociación**:

```python
class Matricula(Base):
    __tablename__ = "matricula"
    alumno_id: Mapped[int] = mapped_column(ForeignKey("alumno.id"), primary_key=True)
    modulo_id: Mapped[int] = mapped_column(ForeignKey("modulo.id"), primary_key=True)
    nota: Mapped[float | None]
    alumno: Mapped["Alumno"] = relationship(back_populates="matriculas")
    modulo: Mapped["Modulo"] = relationship(back_populates="matriculas")
```

3.6.4 Colecciones de valores

Las colecciones no tienen por qué ser listas. Con `collection_class` se pueden mapear como conjuntos o como diccionarios indexados por un atributo:

```python
from sqlalchemy.orm import attribute_keyed_dict

class Curso(Base):
    ...
    # diccionario dni -> Alumno, en lugar de una lista
    alumnos_por_dni: Mapped[dict[str, "Alumno"]] = relationship(
        collection_class=attribute_keyed_dict("dni"), viewonly=True)

curso.alumnos_por_dni["11111111H"].nombre
```

3.7 Mapeo de la herencia

Las bases de datos relacionales no tienen herencia, así que el ORM la simula con una de tres estrategias:

Estrategia	En la base de datos	Ventajas / inconvenientes
Tabla única (*single table*)	Una tabla para toda la jerarquía con una columna **discriminadora**; las columnas de las subclases admiten `NULL`.	Consultas rápidas, sin `JOIN`. Muchas columnas nulas; no se pueden poner `NOT NULL` en campos de subclases.
Tablas unidas (*joined*)	Una tabla por clase; la de cada subclase tiene sólo sus columnas y una clave ajena a la del padre.	Modelo normalizado. Requiere `JOIN` en cada consulta.
Tabla por clase concreta (*concrete*)	Una tabla completa por cada subclase, con todas las columnas.	Sin `JOIN` para consultar una subclase; consultas polimórficas costosas (`UNION`).

```python
# Tablas unidas
class Persona(Base):
    __tablename__ = "persona"
    id: Mapped[int] = mapped_column(primary_key=True)
    nombre: Mapped[str]
    tipo: Mapped[str]                       # discriminador
    __mapper_args__ = {"polymorphic_on": "tipo",
                       "polymorphic_identity": "persona"}

class Profesor(Persona):
    __tablename__ = "profesor"
    id: Mapped[int] = mapped_column(ForeignKey("persona.id"), primary_key=True)
    departamento: Mapped[str]
    __mapper_args__ = {"polymorphic_identity": "profesor"}

class Estudiante(Persona):
    __tablename__ = "estudiante"
    id: Mapped[int] = mapped_column(ForeignKey("persona.id"), primary_key=True)
    ciclo: Mapped[str]
    __mapper_args__ = {"polymorphic_identity": "estudiante"}

# Consulta polimórfica: devuelve objetos Profesor y Estudiante
for p in session.scalars(select(Persona)):
    print(type(p).__name__, p.nombre)
```

Para usar **tabla única**, basta con no definir `__tablename__` ni `id` en las subclases (sus columnas deben ser opcionales).

3.8 Sesiones y estados de un objeto

La **sesión** (`Session`) es el objeto central del ORM. Representa una conversación con la base de datos y actúa como:

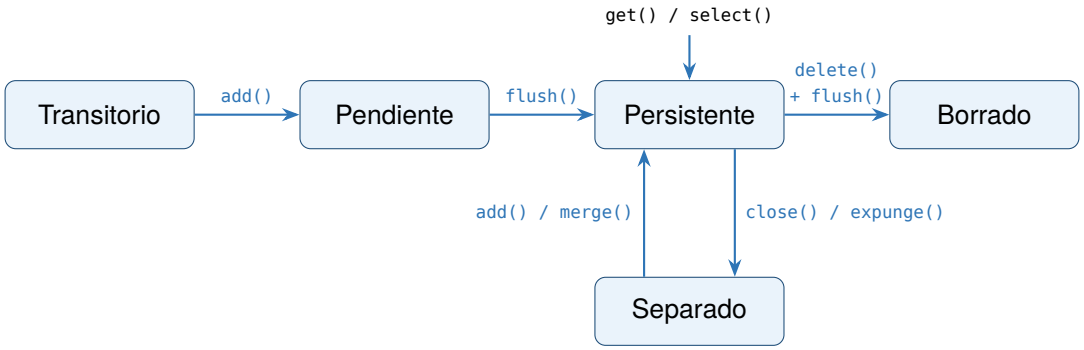
- **Unidad de trabajo** (*unit of work*): acumula los objetos nuevos, modificados y borrados, y al hacer `flush` o `commit` genera todas las sentencias SQL necesarias en el orden correcto.

- **Mapa de identidad** (*identity map*): dentro de una sesión, a cada fila le corresponde un único objeto. Si se consulta dos veces el alumno con id 1, se obtiene *el mismo* objeto.
- Gestor de la **transacción** en curso.

```
from sqlalchemy.orm import Session

with Session(engine) as session:       # se cierra al salir
    ...
    session.commit()
```

3.8.1 Estados de un objeto



Transitorio (*transient*). Objeto recién creado con `Alumno(...)`: no está en la sesión ni en la base de datos.

Pendiente (*pending*). Se ha añadido con `session.add()`, pero aún no se ha enviado el `INSERT`.

Persistente (*persistent*). Está asociado a la sesión y tiene su fila en la base de datos. Los cambios en sus atributos se detectan automáticamente.

Borrado (*deleted*). Se ha marcado con `session.delete()` y se ha enviado el `DELETE`; desaparece al confirmar.

Separado (*detached*). Tiene fila en la base de datos, pero ya no está asociado a ninguna sesión (se cerró). Sus cambios no se guardan salvo que se vuelva a asociar con `merge()`.

```
from sqlalchemy import inspect

a = Alumno(nombre="Eva", apellidos="Martín")
print(inspect(a).transient)       # True
with Session(engine) as session:
    session.add(a)
    print(inspect(a).pending)     # True
    session.flush()               # envía el INSERT (aún sin commit)
    print(inspect(a).persistent, a.id)   # True y ya tiene id
    session.commit()
print(inspect(a).detached)        # True
```

3.9 Carga, almacenamiento y modificación de objetos

```python
from sqlalchemy.orm import Session

with Session(engine) as session:
    # --- Almacenar ---
    eva = Alumno(nombre="Eva", apellidos="Martín")
    session.add(eva)
    session.add_all([Alumno(nombre="Iker", apellidos="Sanz"),
                     Alumno(nombre="Noa", apellidos="Vega")])
    session.commit()

    # --- Cargar por clave primaria ---
    a = session.get(Alumno, 1)            # None si no existe
    if a is None:
        print("No existe")

    # --- Modificar: basta con cambiar el atributo ---
    a.apellidos = "Martín Ruiz"           # la sesión detecta el cambio
    session.commit()                      # UPDATE alumno SET apellidos=...

    # --- Borrar ---
    session.delete(a)
    session.commit()

    # --- Recargar desde la base de datos ---
    session.refresh(eva)
```

3.9.1 Carga perezosa y carga ansiosa

Por defecto, las relaciones se cargan de forma **perezosa** (*lazy*): la colección `curso.alumnos` no se consulta hasta que se accede a ella. Es cómodo, pero puede provocar el **problema N+1**:

```python
cursos = session.scalars(select(Curso)).all()     # 1 consulta
for c in cursos:
    print(c.nombre, len(c.alumnos))               # +1 consulta POR CADA curso
```

Si sabemos que vamos a necesitar la relación, se pide una carga **ansiosa** (*eager*):

```python
from sqlalchemy.orm import selectinload, joinedload

cursos = session.scalars(
    select(Curso).options(selectinload(Curso.alumnos))).all()   # 2 consultas
alumnos = session.scalars(
    select(Alumno).options(joinedload(Alumno.curso))).all()      # 1 consulta con JOIN
```

> **Cuidado.** Si se accede a una relación perezosa de un objeto **separado** (después de cerrar la sesión), se produce `DetachedInstanceError`. Carga lo que necesites antes de cerrar la sesión.

3.10 Consultas

3.10.1 El lenguaje de consultas propio: `select()`

Cada ORM tiene su propio lenguaje de consultas orientado a objetos: HQL/JPQL en Hibernate, LINQ en Entity Framework. En SQLAlchemy las consultas se construyen con **expresiones Python** sobre las clases y sus atributos, que el ORM traduce al dialecto SQL del gestor.

```
from sqlalchemy import select, func, or_, desc

# Todos los alumnos ordenados
alumnos = session.scalars(select(Alumno).order_by(Alumno.apellidos)).all()

# Filtros
stmt = (select(Alumno)
        .where(Alumno.apellidos.like("G%"))
        .where(or_(Alumno.curso_id == 1, Alumno.curso_id.is_(None))))
for a in session.scalars(stmt):
    print(a)

# Un único resultado
ana = session.scalars(select(Alumno).where(Alumno.dni == "11111111H")).one_or_none()

# JOIN siguiendo una relación
stmt = select(Alumno).join(Alumno.curso).where(Curso.nombre == "2º DAM")

# Seleccionar columnas concretas -> filas (tuplas con nombre)
for nombre, curso in session.execute(
        select(Alumno.nombre, Curso.nombre).join(Alumno.curso)):
    print(nombre, curso)

# Agregados y agrupación
stmt = (select(Curso.nombre, func.count(Alumno.id).label("total"))
        .join(Curso.alumnos)
        .group_by(Curso.nombre)
        .having(func.count(Alumno.id) > 10)
        .order_by(desc("total")))
for fila in session.execute(stmt):
    print(fila.nombre, fila.total)

# Paginación
pagina = session.scalars(select(Alumno).order_by(Alumno.id)
                         .limit(20).offset(40)).all()
```

> **Nota.** `session.scalars(stmt)` devuelve directamente los objetos (la primera columna de cada fila); `session.execute(stmt)` devuelve filas. Métodos de resultado útiles: `.all()`, `.first()`, `.one()` (error si no hay exactamente uno), `.one_or_none()`.

3.10.2 Modificaciones masivas

```
from sqlalchemy import update, delete

session.execute(update(Alumno).where(Alumno.curso_id == 1)
                .values(activo=False))
session.execute(delete(Alumno).where(Alumno.activo.is_(False)))
session.commit()
```

3.10.3 Consultas SQL

Cuando hace falta, se puede ejecutar SQL directamente con `text()`, siempre con parámetros con nombre:

```
from sqlalchemy import text

filas = session.execute(
    text("SELECT nombre, apellidos FROM alumno WHERE curso_id = :c"),
    {"c": 1}).all()

# Y convertir el resultado en objetos mapeados
stmt = select(Alumno).from_statement(
    text("SELECT * FROM alumno WHERE apellidos ILIKE :patron"))
alumnos = session.scalars(stmt, {"patron": "%garcía%"}).all()

# Llamar a una función almacenada de PostgreSQL
n = session.scalar(select(func.num_alumnos(1)))
```

3.11 Gestión de transacciones

La sesión inicia una transacción automáticamente con la primera operación y la mantiene hasta `commit()` o `rollback()`. El patrón recomendado es delimitar la transacción con un bloque:

```
from sqlalchemy.orm import sessionmaker

Session = sessionmaker(bind=engine)

# commit al terminar el bloque, rollback si hay excepción, y cierre
with Session.begin() as session:
    origen = session.get(Cuenta, "ES01")
    destino = session.get(Cuenta, "ES02")
    if origen.saldo < 100:
        raise ValueError("Saldo insuficiente")   # -> rollback
    origen.saldo -= 100
    destino.saldo += 100
```

Equivale a:

```
session = Session()
try:
    ...
    session.commit()
except Exception:
    session.rollback()
    raise
finally:
    session.close()
```

3.11.1 Transacciones anidadas

```
from sqlalchemy.exc import IntegrityError
```

```python
with Session.begin() as session:
    for datos in alumnos_a_importar:
        try:
            with session.begin_nested():      # SAVEPOINT
                session.add(Alumno(**datos))
        except IntegrityError:
            print("Duplicado:", datos["dni"])
```

3.11.2 Excepciones

Las excepciones del conector se envuelven en las de `sqlalchemy.exc`, con los mismos nombres de la DB-API (`IntegrityError`, `OperationalError`...). Además, el ORM tiene las suyas, como `NoResultFound` y `MultipleResultsFound` (al usar `.one()`) o `DetachedInstanceError`.

3.12 Organización de una aplicación con ORM

Una estructura habitual separa:

```
instituto/├──
 config.toml├──
 instituto/│
   ├── __init__.py│
   ├── basedatos.py      engine y sessionmaker│
   ├── modelo.py         clases mapeadas│
   ├── repositorios.py   consultas (opcional, patrón repositorio)│
   ├── servicios.py      lógica de negocio y transacciones│
   └── main.py           interfaz (consola, GUI, API web...)└──
 tests/
    └── test_servicios.py
```

```python
# tests/conftest.py — base de datos nueva en memoria para cada prueba
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import Session
from instituto.modelo import Base

@pytest.fixture
def session():
    engine = create_engine("sqlite:///:memory:")
    Base.metadata.create_all(engine)
    with Session(engine) as s:
        yield s
```

3.13 Ejercicios

Ejercicio 3.1 — Primer mapeo

Dificultad: ★

Mapea con SQLAlchemy una clase `Libro` (isbn como clave primaria, título, autor, año, precio con `Numeric(8,2)`, disponible). Con `echo=True`, crea las tablas en SQLite, inserta cinco libros, modifica el precio de uno, borra otro y realiza las consultas: libros de un autor; libros anteriores a un año; precio medio. Observa el SQL generado en cada operación y anota qué sentencias aparecen y cuándo.

Ejercicio 3.2 — Estados de un objeto

Dificultad: ★

Escribe un programa que muestre, usando `inspect()`, el estado de un objeto en cada momento de su ciclo de vida: creación, `add`, `flush`, `commit`, cierre de sesión, modificación estando separado, `merge` en una sesión nueva, `delete` y `commit`. Comprueba también el mapa de identidad: dos `session.get` del mismo id devuelven el mismo objeto (`is`).

Ejercicio 3.3 — Relaciones

Dificultad: ★★

Modela una tienda: `Cliente` 1:N `Pedido`, `Pedido` 1:N `LineaPedido` (con cascada `delete-orphan`) y `LineaPedido` N:1 `Producto`; y `Producto` N:M `Categoria`. Crea datos de prueba y consulta: total facturado por cliente; productos que nunca se han vendido; categorías con su número de productos; el pedido de mayor importe. Provoca el problema N+1 al listar los pedidos con sus líneas, cuenta las consultas con `echo=True` y resuélvelo con carga ansiosa.

Ejercicio 3.4 — Herencia

Dificultad: ★★

Modela la jerarquía `Vehiculo` → `Coche`, `Moto`, `Furgoneta` con atributos comunes y propios. Impleméntala primero con tabla única y después con tablas unidas. Compara las tablas generadas y el SQL de una consulta polimórfica y de una consulta sólo de `Moto`. Resume en una tabla las ventajas e inconvenientes que has observado.

Ejercicio 3.5 — Migrar un DAO a ORM

Dificultad: ★★

Toma el proyecto del videoclub del Tema 2 y sustituye la capa DAO por un modelo SQLAlchemy y repositorios. Mantén la misma interfaz pública de la capa de servicio, de manera que `main.py` no cambie. Compara el número de líneas de código de ambas versiones.

Ejercicio 3.6 — Transacciones con el ORM

Dificultad: ★★★

Implementa una importación de alumnos desde un CSV en la que cada fila se inserte dentro de un `begin_nested()`: las filas con errores (DNI duplicado, curso inexistente, datos obligatorios vacíos) se registran en un fichero `errores.csv` con el motivo, y el resto

se confirman todas juntas al final. Añade una opción `--todo-o-nada` en la que un solo error deshaga toda la importación.

Proyecto del tema: Gestión académica de un instituto

Desarrolla la capa de persistencia y una aplicación de consola para gestionar la información académica de un centro usando SQLAlchemy sobre PostgreSQL (y SQLite para las pruebas).
Modelo: `Persona` (jerarquía con `Profesor` y `Alumno`), `Ciclo`, `Curso`, `Modulo` (pertenece a un curso, lo imparte un profesor), `Matricula` (objeto de asociación alumno–módulo con las notas de cada evaluación), `Falta` (alumno, módulo, fecha, justificada).
Requisitos:

1. Configuración en `config.toml` y creación del esquema con el ORM (opcional: con migraciones de Alembic).
2. Mapeo de todas las relaciones (1:1, 1:N, N:M con objeto de asociación) y de la herencia, justificando la estrategia elegida.
3. Capa de servicios con transacciones: matricular a un alumno en todos los módulos de un curso; registrar las faltas de un día para un grupo; cerrar la evaluación (calcula notas finales y marca módulos aprobados).
4. Consultas con `select()`: boletín de notas de un alumno; alumnos con más de un 15 % de faltas injustificadas en un módulo (pérdida de evaluación continua); media de cada módulo; profesores con su carga horaria.
5. Al menos una consulta con SQL nativo y una llamada a una función almacenada.
6. Ausencia de problemas N+1 en los listados (verificado con `echo=True`).
7. Exportación del boletín de un alumno a XML y a HTML con XSLT (Tema 1).
8. Pruebas con `pytest` de la capa de servicios.

TEMA 4

Bases de datos objeto-relacionales y orientadas a objetos

En este tema aprenderás a…

- Conocer las características, ventajas e inconvenientes de las bases de datos que almacenan objetos.
- Utilizar las extensiones objeto-relacionales del estándar SQL:1999 en PostgreSQL: tipos compuestos, arrays, herencia de tablas, enumerados y funciones.
- Acceder a esos tipos y a las funciones del gestor desde Python.
- Conocer el modelo de las bases de datos orientadas a objetos (estándar ODMG) y el lenguaje OQL.
- Gestionar la persistencia de objetos simples y estructurados con una base de datos orientada a objetos (ZODB).
- Gestionar transacciones en ambos tipos de gestores.

Resultado de aprendizaje asociado: **RA4**.

Actualización del currículo. Las bases de datos orientadas a objetos "puras" tuvieron su auge en los años noventa, pero hoy ocupan un nicho muy pequeño: el estándar ODMG y su lenguaje OQL dejaron de mantenerse en 2001 y gestores clásicos como db4o están descontinuados. Por eso, en este tema:

- Se da más peso a las **características objeto-relacionales**, que sí están vivas en gestores muy utilizados como PostgreSQL y Oracle.
- OQL se estudia como modelo conceptual (es el antecesor directo de HQL/JPQL y de los lenguajes de consulta de los ORM).
- La parte práctica de bases de datos orientadas a objetos se hace con **ZODB**, una base de datos de objetos nativa de Python, madura y mantenida.

4.1 Bases de datos objeto-relacionales

4.1.1 Características

Una base de datos **objeto-relacional** (BDOR) es una base de datos relacional ampliada con conceptos de la orientación a objetos, para reducir el desfase objeto-relacional **desde el lado del gestor**. Sigue habiendo tablas y SQL, pero además:

- Se pueden definir **tipos de datos de usuario**, incluidos tipos estructurados (con varios atributos) que pueden usarse como tipo de una columna.
- Una columna puede contener **colecciones**: arrays (y en algunos gestores, tablas

anidadas).

- Existe la **herencia** entre tipos y entre tablas.
- Los tipos pueden tener **métodos** (funciones asociadas).
- Existen **referencias** entre objetos (`REF`), que hacen de identificadores de objeto.

Estas extensiones se estandarizaron en **SQL:1999** (también llamado SQL3).

4.1.2 Ventajas e inconvenientes

+ Modelado más natural de datos complejos (direcciones, coordenadas, listas de teléfonos) sin crear tablas auxiliares.
+ Reutilización de tipos y comportamiento en el propio gestor.
+ Se conserva todo lo bueno del modelo relacional: SQL, transacciones, madurez, herramientas.
– Cada gestor implementa un subconjunto distinto del estándar con sintaxis propia: poca portabilidad.
– Se rompe la primera forma normal; las consultas pueden volverse más complejas.
– Los conectores y los ORM no siempre soportan bien los tipos estructurados.

4.1.3 Gestores objeto-relacionales

Oracle implementa el modelo de SQL:1999 de forma bastante completa: `CREATE TYPE ... AS OBJECT` con métodos, tablas de objetos, `REF`, `VARRAY` y tablas anidadas.

PostgreSQL se define a sí mismo como "base de datos objeto-relacional". Ofrece tipos compuestos, arrays de cualquier tipo, herencia de tablas, dominios, enumerados, rangos, tipos geométricos, `JSONB` y la posibilidad de definir tipos y operadores nuevos.

IBM Db2 e Informix también incluyen tipos estructurados y herencia.

Usaremos PostgreSQL (consulta el Apéndice A para levantarlo con Docker).

4.2 Gestión de objetos con SQL:1999

4.2.1 El estándar

Como referencia, este es el aspecto de las construcciones de SQL:1999 (la sintaxis exacta es la de Oracle, que es la más próxima al estándar):

```
-- Tipo estructurado
CREATE TYPE direccion_t AS OBJECT (
    calle VARCHAR2(80), numero NUMBER, cp CHAR(5), ciudad VARCHAR2(40)
);
-- Tipo con herencia (NOT FINAL permite subtipos) y método
CREATE TYPE persona_t AS OBJECT (
    dni CHAR(9), nombre VARCHAR2(50), direccion direccion_t,
    MEMBER FUNCTION etiqueta RETURN VARCHAR2
) NOT FINAL;
CREATE TYPE profesor_t UNDER persona_t (departamento VARCHAR2(40));
```

```
-- Colección
CREATE TYPE telefonos_t AS VARRAY(5) OF VARCHAR2(15);
-- Tabla de objetos: cada fila es un objeto con identificador (OID)
CREATE TABLE personas OF persona_t (PRIMARY KEY (dni));
-- Referencia a un objeto de otra tabla
CREATE TABLE tutoria (grupo VARCHAR2(10), tutor REF persona_t SCOPE IS personas);
SELECT t.grupo, DEREF(t.tutor).nombre FROM tutoria t;
```

4.2.2 Tipos compuestos en PostgreSQL

Un tipo compuesto es un tipo estructurado formado por varios atributos:

```
CREATE TYPE direccion AS (
    calle   VARCHAR(80),
    numero  INTEGER,
    cp      CHAR(5),
    ciudad  VARCHAR(40)
);

CREATE TABLE cliente (
    id         SERIAL PRIMARY KEY,
    nombre     VARCHAR(60) NOT NULL,
    domicilio  direccion,                -- columna de tipo estructurado
    envio      direccion
);

INSERT INTO cliente (nombre, domicilio)
VALUES ('Ana', ROW('Santiago', 12, '47001', 'Valladolid'));

-- Acceso a los atributos: (columna).atributo
SELECT nombre, (domicilio).ciudad FROM cliente
WHERE (domicilio).cp LIKE '47%';

UPDATE cliente SET domicilio.numero = 14 WHERE id = 1;
```

4.2.3 Colecciones: arrays

```
CREATE TABLE contacto (
    id         SERIAL PRIMARY KEY,
    nombre     VARCHAR(60),
    telefonos  VARCHAR(15)[],            -- array de cadenas
    etiquetas  TEXT[] DEFAULT '{}',
    direcciones direccion[]              -- array de tipos compuestos
);

INSERT INTO contacto (nombre, telefonos, etiquetas)
VALUES ('Luis', ARRAY['600111222', '983000000'], '{trabajo,amigos}');

SELECT nombre, telefonos[1] FROM contacto;           -- los índices empiezan en 1
SELECT nombre FROM contacto WHERE 'amigos' = ANY(etiquetas);
SELECT nombre FROM contacto WHERE etiquetas @> ARRAY['trabajo'];   -- contiene
SELECT nombre, unnest(telefonos) AS tlf FROM contacto;           -- array -> filas
UPDATE contacto SET etiquetas = array_append(etiquetas, 'familia') WHERE id = 1;
SELECT nombre, cardinality(telefonos) FROM contacto;
```

4.2.4 Herencia de tablas

```
CREATE TABLE persona (
    id      SERIAL PRIMARY KEY,
    dni     CHAR(9) UNIQUE,
    nombre  VARCHAR(60)
);
CREATE TABLE profesor (departamento VARCHAR(40)) INHERITS (persona);
CREATE TABLE alumno   (ciclo VARCHAR(10), curso INTEGER) INHERITS (persona);

INSERT INTO profesor (dni, nombre, departamento)
VALUES ('11111111H', 'Marta', 'Informática');

SELECT * FROM persona;          -- incluye las filas de profesor y alumno
SELECT * FROM ONLY persona;     -- sólo las filas de la tabla padre
SELECT p.nombre, p.tableoid::regclass AS tipo FROM persona p;
```

Cuidado. En PostgreSQL, las restricciones `UNIQUE` y `PRIMARY KEY` y las claves ajenas **no se heredan entre tablas**: el DNI podría repetirse entre `profesor` y `alumno`. Por eso la herencia de tablas se usa con prudencia.

4.2.5 Otros tipos: enumerados, dominios y JSONB

```
CREATE TYPE estado_pedido AS ENUM ('pendiente', 'enviado', 'entregado');
CREATE DOMAIN email AS VARCHAR(120) CHECK (VALUE ~ '^[^@]+@[^@]+\.[^@]+$');

CREATE TABLE pedido (
    id       SERIAL PRIMARY KEY,
    estado   estado_pedido DEFAULT 'pendiente',
    contacto email,
    extra    JSONB                      -- documento JSON indexable
);
SELECT id FROM pedido WHERE extra->>'urgente' = 'true';
```

4.2.6 Métodos: funciones sobre tipos

En PostgreSQL los tipos compuestos no tienen métodos propiamente dichos, pero se consigue el mismo efecto con funciones que reciben el tipo como parámetro:

```
CREATE FUNCTION texto(d direccion) RETURNS TEXT
LANGUAGE sql IMMUTABLE AS $$
    SELECT d.calle || ', ' || d.numero || ' - ' || d.cp || ' ' || d.ciudad;
$$;

SELECT nombre, texto(domicilio) FROM cliente;
-- Notación funcional como atributo: también vale (domicilio).texto
```

4.3 Acceso a las funciones del gestor desde Python

4.3.1 Conexión

Se usa el mismo conector que en el Tema 2 (psycopg). Las conexiones se abren y se cierran igual; lo nuevo es cómo se adaptan los tipos.

4.3.2 Arrays

psycopg convierte automáticamente los arrays de PostgreSQL en listas de Python y viceversa:

```
import psycopg

with psycopg.connect(conninfo) as conn:
    conn.execute(
        "INSERT INTO contacto (nombre, telefonos, etiquetas) VALUES (%s, %s, %s)",
        ("Eva", ["611222333"], ["clase", "dam"]))
    for nombre, tlfs in conn.execute(
            "SELECT nombre, telefonos FROM contacto WHERE %s = ANY(etiquetas)",
            ("dam",)):
        print(nombre, tlfs)          # tlfs es una list[str]
```

4.3.3 Tipos compuestos

Por defecto un tipo compuesto llega como una tupla. Para trabajar con objetos, se **registra** el tipo indicando la clase Python que lo representa. La forma más sencilla es usar una `NamedTuple`, porque psycopg necesita poder recorrer los campos en orden para enviarlos al servidor (con una *dataclass* habría que indicar además cómo convertirla en secuencia con el argumento `make_sequence`):

```
from typing import NamedTuple
import psycopg
from psycopg.types.composite import CompositeInfo, register_composite

class Direccion(NamedTuple):        # una tupla con nombres de campo
    calle: str
    numero: int
    cp: str
    ciudad: str

with psycopg.connect(conninfo) as conn:
    info = CompositeInfo.fetch(conn, "direccion")
    register_composite(info, conn, Direccion)   # direccion <-> Direccion

    # Guardar un objeto estructurado
    d = Direccion("Plaza Mayor", 1, "47001", "Valladolid")
    conn.execute("INSERT INTO cliente (nombre, domicilio) VALUES (%s, %s)",
                 ("Luis", d))

    # Recuperar objetos estructurados
    for nombre, dom in conn.execute("SELECT nombre, domicilio FROM cliente"):
        print(nombre, dom.ciudad if dom else "-")    # dom es Direccion
```

4.3.4 Enumerados y JSONB

```python
from enum import Enum
from psycopg.types.enum import EnumInfo, register_enum
from psycopg.types.json import Jsonb

class Estado(Enum):
    PENDIENTE = "pendiente"
    ENVIADO = "enviado"
    ENTREGADO = "entregado"

with psycopg.connect(conninfo) as conn:
    register_enum(EnumInfo.fetch(conn, "estado_pedido"), conn, Estado,
                  mapping={m: m.value for m in Estado})
    conn.execute("INSERT INTO pedido (estado, extra) VALUES (%s, %s)",
                 (Estado.ENVIADO, Jsonb({"urgente": True, "bultos": 3})))
    estado, extra = conn.execute(
        "SELECT estado, extra FROM pedido ORDER BY id DESC LIMIT 1").fetchone()
    print(estado, extra["bultos"])      # Estado.ENVIADO 3
```

4.3.5 Funciones del gestor

```python
with psycopg.connect(conninfo) as conn:
    for nombre, texto in conn.execute(
            "SELECT nombre, texto(domicilio) FROM cliente"):
        print(nombre, texto)
```

> **Con SQLAlchemy.** SQLAlchemy dispone de los tipos `ARRAY`, `JSONB` y `ENUM` para PostgreSQL (`sqlalchemy.dialects.postgresql`), y de `composite()` para mapear varias columnas a un objeto. Los tipos compuestos de PostgreSQL como tipo de columna no están soportados directamente.

4.3.6 Transacciones

Las transacciones funcionan exactamente igual que en el Tema 2: `commit`, `rollback` y `conn.transaction()`.

4.4 Bases de datos orientadas a objetos

4.4.1 Características

Una base de datos **orientada a objetos** (BDOO) almacena directamente los objetos del lenguaje de programación, sin traducirlos a tablas. Desaparece el desfase objeto-relacional: el modelo de la base de datos *es* el modelo de clases de la aplicación. Sus características principales son:

- **Identidad de objeto** (OID): cada objeto tiene un identificador único e inmutable, independiente de sus valores.
- Soporte de **objetos complejos**: atributos que son otros objetos o colecciones.
- **Encapsulación**, **herencia** y **polimorfismo**.

- **Navegación**: las relaciones se recorren siguiendo referencias, sin `JOIN`.
- **Persistencia transparente** (u ortogonal): cualquier objeto alcanzable desde un objeto persistente se vuelve persistente (*persistencia por alcance*).
- Transacciones, concurrencia y recuperación, como en cualquier SGBD.

4.4.2 El estándar ODMG

El *Object Data Management Group* publicó el estándar ODMG (última versión 3.0, 2001), que definía:

- Un **modelo de objetos** común.
- **ODL** (*Object Definition Language*): lenguaje para definir clases, atributos y relaciones.
- **OQL** (*Object Query Language*): lenguaje de consulta declarativo, basado en SQL.
- Enlaces (*bindings*) con C++, Smalltalk y Java.

4.4.3 Ventajas e inconvenientes

+ Sin desfase objeto-relacional ni código de mapeo.
+ Muy eficientes al navegar por grafos de objetos complejos (CAD, simulaciones, sistemas de telecomunicaciones).
+ Se almacenan comportamiento y tipos complejos de forma natural.
– Falta de un estándar efectivo: cada gestor tiene su API. OQL nunca llegó a implantarse por completo.
– Fuerte acoplamiento con un lenguaje de programación concreto: es difícil compartir los datos con otras aplicaciones.
– Consultas *ad hoc* e informes más difíciles que con SQL; menos herramientas.
– Cuota de mercado muy pequeña y menos profesionales que las conozcan.

4.4.4 Gestores de bases de datos orientadas a objetos

ZODB	Base de datos de objetos para Python. Usada por el servidor de aplicaciones Zope y el gestor de contenidos Plone. Libre y mantenida.
ObjectDB	BDOO para Java que usa la API estándar JPA (la misma que Hibernate).
GemStone/S	BDOO comercial para Smalltalk, en uso en sistemas financieros.
Versant / Actian NoSQL, **ObjectStore**	BDOO comerciales clásicas para C++ y Java.
db4o	Muy popular en docencia; descontinuada desde 2014.

4.5 Tipos de datos: básicos y estructurados

En el modelo ODMG se distinguen:

Tipos básicos (literales atómicos). `long`, `short`, `float`, `double`, `boolean`, `char`, `string`, `enum`, `date`, `time`, `timestamp`.

Tipos estructurados. `struct` (registro con campos con nombre, sin identidad propia).

Colecciones. `set` (sin orden, sin repetidos), `bag` (sin orden, con repetidos), `list` (con orden), `array` y `dictionary`.

Objetos. Instancias de clases definidas por el usuario, con identidad.

Un ejemplo de definición en ODL:

```
class Departamento (extent departamentos key nombre) {
    attribute string nombre;
    relationship set<Profesor> profesores inverse Profesor::departamento;
};
class Profesor (extent profesores key dni) {
    attribute string dni;
    attribute string nombre;
    attribute struct Direccion { string calle; string ciudad; } direccion;
    attribute set<string> telefonos;
    relationship Departamento departamento inverse Departamento::profesores;
    float sueldo_neto();
};
```

El `extent` es la colección de todos los objetos de la clase (el equivalente a una tabla, y el punto de partida de las consultas).

4.6 El lenguaje de consultas OQL

OQL toma la sintaxis de `SELECT` de SQL y la extiende para trabajar con objetos. Sus elementos principales son:

4.6.1 Sintaxis básica y expresiones de camino

```
select p.nombre
from p in profesores
where p.departamento.nombre = "Informática"
```

- En `from` se declaran variables que recorren colecciones (`p in profesores`, también `profesores p` o `profesores as p`).
- Las **expresiones de camino** (`p.departamento.nombre`) navegan por las relaciones sin necesidad de `JOIN`.
- Se pueden invocar métodos: `p.sueldo_neto()`.

4.6.2 Resultados estructurados

```
select struct(nombre: p.nombre, ciudad: p.direccion.ciudad)
from p in profesores
where p.sueldo_neto() > 2000
order by p.nombre
```

El resultado puede ser un conjunto de literales, de objetos o de estructuras; `select distinct` devuelve un `set` y `select` un `bag`.

4.6.3 Recorrido de colecciones anidadas

```
select d.nombre, p.nombre
from d in departamentos, p in d.profesores
where "983000000" in p.telefonos
```

4.6.4 Operadores

Aritméticos y de comparación	`+ - * / mod`, `= != < <= > >=`, `like`
Lógicos	`and`, `or`, `not`
Pertenencia	`x in coleccion`
Cuantificadores	`for all x in c: condición`, `exists x in c: condición`
Agregados	`count`, `sum`, `avg`, `min`, `max`
Conjuntos	`union`, `intersect`, `except`
Conversión	`element(c)` (único elemento), `flatten(c)`, `listtoset(l)`
Agrupación	`group by … partition`, `having`

```
// Departamentos en los que todos los profesores cobran más de 1800
select d.nombre from d in departamentos
where for all p in d.profesores: p.sueldo_neto() > 1800

// Número de profesores por departamento
select d.nombre, count(d.profesores) from d in departamentos

// Una consulta OQL puede ser una simple expresión
count(profesores)
element(select p from p in profesores where p.dni = "11111111H")
```

Herederos de OQL. Aunque OQL apenas se implementó tal cual, su idea (consultar objetos y navegar por sus relaciones con una sintaxis parecida a SQL) sobrevive en **HQL/JPQL** de Hibernate y JPA, y en los lenguajes de consulta de los ORM como el `select()` de SQLAlchemy que vimos en el Tema 3.

4.7 Una base de datos de objetos en Python: ZODB

4.7.1 Instalación y conceptos

```
$ pip install ZODB
```

- Los objetos se guardan en un **almacenamiento** (*storage*); el más sencillo es `FileStorage`, un fichero local (`.fs`).
- La base de datos tiene un **objeto raíz**, un diccionario a partir del cual se alcanzan todos los demás (*persistencia por alcance*).
- Para que ZODB detecte los cambios de un objeto, su clase debe heredar de `persistent.Persistent`.
- Todo cambio ocurre dentro de una **transacción**, que se confirma con `transaction.commit()` y se deshace con `transaction.abort()`.

4.7.2 API de la base de datos: conexión y objetos simples

```python
import ZODB, ZODB.FileStorage
import transaction
from persistent import Persistent

class Alumno(Persistent):
    def __init__(self, dni: str, nombre: str, edad: int) -> None:
        self.dni = dni
        self.nombre = nombre
        self.edad = edad

    def __repr__(self) -> str:
        return f"Alumno({self.dni}, {self.nombre}, {self.edad})"

# Apertura: almacenamiento -> base de datos -> conexión -> raíz
storage = ZODB.FileStorage.FileStorage("instituto.fs")
db = ZODB.DB(storage)
conexion = db.open()
raiz = conexion.root()

# Guardar: basta con hacerlo alcanzable desde la raíz y confirmar
raiz["ana"] = Alumno("11111111H", "Ana", 20)
transaction.commit()

# Modificar: se cambia el objeto y se confirma
raiz["ana"].edad += 1
transaction.commit()

# Cierre
conexion.close()
db.close()
```

Nota. `ZODB.DB` admite también un nombre de fichero directamente (`ZODB.DB("instituto.fs")`) y `None` para una base de datos en memoria. Y `db.transaction()` abre una conexión con una transacción en un bloque `with`:

```python
with db.transaction() as conn:
    conn.root()["luis"] = Alumno("22222222J", "Luis", 19)
```

```
# commit automático al salir (abort si hay excepción)
```

4.7.3 Objetos estructurados y colecciones

Los objetos pueden contener otros objetos y colecciones. Hay que tener en cuenta un detalle: ZODB detecta cuándo se *asigna* un atributo de un objeto `Persistent`, pero **no** cuándo se modifica por dentro una lista o un diccionario normales. Por eso se usan las colecciones persistentes:

`PersistentList`	Lista que avisa de sus cambios.
`PersistentMapping`	Diccionario que avisa de sus cambios. Adecuado para pocos elementos.
`BTrees.OOBTree`	Diccionario ordenado que se guarda en nodos separados: escala a millones de elementos y sólo carga en memoria lo que se usa. Variantes según el tipo de clave y valor (`IOBTree`: clave entera, `OOBTree`: claves y valores objeto).

```
from dataclasses import dataclass
import ZODB, transaction
from persistent import Persistent
from persistent.list import PersistentList
from BTrees.OOBTree import OOBTree

@dataclass(frozen=True)
class Direccion:                    # valor inmutable: se guarda dentro del objeto
    calle: str
    ciudad: str

class Modulo(Persistent):
    def __init__(self, codigo: str, nombre: str, horas: int) -> None:
        self.codigo, self.nombre, self.horas = codigo, nombre, horas

class Alumno(Persistent):
    def __init__(self, dni: str, nombre: str, direccion: Direccion) -> None:
        self.dni = dni
        self.nombre = nombre
        self.direccion = direccion
        self.modulos = PersistentList()     # referencias a otros objetos
        self.telefonos: list[str] = []      # lista normal (ver aviso)

    def matricular(self, m: Modulo) -> None:
        self.modulos.append(m)              # se detecta: PersistentList

    def anadir_telefono(self, t: str) -> None:
        self.telefonos = self.telefonos + [t]   # reasignar para que se detecte
        # alternativa: self.telefonos.append(t); self._p_changed = True

db = ZODB.DB("instituto.fs")
with db.transaction() as conn:
    raiz = conn.root()
    raiz.setdefault("alumnos", OOBTree())       # "extent" de alumnos por DNI
    raiz.setdefault("modulos", OOBTree())
```

```
    ad = Modulo("0486", "Acceso a Datos", 126)
    raiz["modulos"][ad.codigo] = ad
    ana = Alumno("11111111H", "Ana", Direccion("Santiago 12", "Valladolid"))
    ana.matricular(ad)                          # referencia, no copia
    ana.anadir_telefono("600111222")
    raiz["alumnos"][ana.dni] = ana
```

Cambios no detectados. Si haces `alumno.telefonos.append(...)` sobre una `list` normal y confirmas, el cambio **no se guarda**: el objeto no sabe que ha cambiado. Usa colecciones persistentes, reasigna el atributo o marca el objeto con `self._p_changed = True`.

4.7.4 Consultas

ZODB no tiene un lenguaje de consultas: las consultas se hacen **navegando** y con expresiones de Python, que cumplen el papel de OQL. Los BTrees permiten búsquedas por rango de claves de forma eficiente.

```
with db.transaction() as conn:
    raiz = conn.root()
    alumnos = raiz["alumnos"]

    # Acceso por clave (como un índice)
    ana = alumnos["11111111H"]

    # select a.nombre from a in alumnos where a.direccion.ciudad = "Valladolid"
    nombres = [a.nombre for a in alumnos.values()
               if a.direccion.ciudad == "Valladolid"]

    # Navegación: módulos de cada alumno
    for a in alumnos.values():
        print(a.nombre, [m.nombre for m in a.modulos])

    # Rango de claves: DNIs entre dos valores (eficiente en un BTree)
    for dni, a in alumnos.items(min="10000000A", max="30000000Z"):
        print(dni, a.nombre)

    # Cuantificador universal: alumnos con todos sus módulos de más de 100 h
    largos = [a for a in alumnos.values()
              if all(m.horas > 100 for m in a.modulos)]
```

Para consultas frecuentes sobre otro atributo, se mantiene un **índice** propio: otro BTree que asocie, por ejemplo, ciudad → conjunto de alumnos (`BTrees.OOBTree.OOTreeSet`), y que se actualiza al dar de alta o modificar un alumno.

4.7.5 Borrado

Para borrar un objeto basta con eliminar todas las referencias que llegan a él; el espacio se recupera al **empaquetar** la base de datos (`db.pack()`), que además elimina las versiones antiguas de los objetos.

```
with db.transaction() as conn:
    del conn.root()["alumnos"]["11111111H"]
```

```
db.pack()
```

4.7.6 Transacciones

```
import transaction
from ZODB.POSException import ConflictError

conexion = db.open()
raiz = conexion.root()
try:
    raiz["modulos"]["0486"].horas = 130
    if raiz["modulos"]["0486"].horas > 200:
        raise ValueError("Horas no válidas")
    transaction.commit()
except ValueError:
    transaction.abort()                 # descarta los cambios en memoria
except ConflictError:
    transaction.abort()                 # otro proceso modificó el mismo objeto
    print("Conflicto: vuelve a intentarlo")

sp = transaction.savepoint()            # punto intermedio
raiz["temporal"] = 1
sp.rollback()                           # deshace sólo desde el savepoint
conexion.close()
```

ZODB usa **control de concurrencia optimista**: cada conexión trabaja con una instantánea consistente de la base de datos y, si al confirmar otra transacción ha modificado los mismos objetos, se lanza `ConflictError`. Para acceso desde varios procesos o máquinas se usa el servidor ZEO o el almacenamiento RelStorage (que guarda los objetos en PostgreSQL).

4.8 Pruebas y documentación

Para probar el código que usa ZODB, se crea una base de datos en memoria en cada prueba:

```
import pytest, ZODB

@pytest.fixture
def raiz():
    db = ZODB.DB(None)                  # en memoria
    conn = db.open()
    yield conn.root()
    conn.close()
    db.close()
```

4.9 Ejercicios

Ejercicio 4.1 — Tipos compuestos y arrays

Dificultad: ★

En PostgreSQL, crea un tipo compuesto `coordenada(lat, lon)` y una tabla `ruta` con un nombre, un array de coordenadas y un array de etiquetas. Inserta varias rutas y consulta: rutas con una etiqueta dada; número de puntos de cada ruta; primera y última coordenada de cada ruta. Escribe una función SQL `distancia(coordenada, coordenada)` (fórmula de Haversine) y úsala para calcular la longitud total de cada ruta.

Ejercicio 4.2 — Objetos desde Python

Dificultad: ★★

Desde Python, registra el tipo `coordenada` con una *dataclass* y escribe un programa que importe rutas desde ficheros GPX (son XML: usa ElementTree, Tema 1) a la tabla `ruta`, y que después liste las rutas mostrando su longitud calculada por la función del gestor.

Ejercicio 4.3 — Herencia de tablas

Dificultad: ★★

Modela con herencia de tablas de PostgreSQL `empleado` → `tecnico`, `comercial`. Comprueba la diferencia entre consultar con y sin `ONLY`, y demuestra el problema de las restricciones `UNIQUE` no heredadas. Propón y aplica una solución (por ejemplo, un disparador o una tabla de DNIs).

Ejercicio 4.4 — OQL

Dificultad: ★

Dado el esquema ODL de `Departamento` y `Profesor` de este tema, ampliado con una clase `Modulo` (nombre, horas, relación con el profesor que lo imparte), escribe en OQL: los módulos de más de 100 horas con el nombre de su profesor; los departamentos con más de cinco profesores; los profesores que imparten algún módulo de más de 150 horas (`exists`); el total de horas impartidas por cada departamento. Traduce después cada consulta a SQL relacional y compara.

Ejercicio 4.5 — Primeros pasos con ZODB

Dificultad: ★★

Guarda en ZODB una colección de libros (`OOBTree` por ISBN) con sus autores (objetos compartidos: varios libros referencian al mismo autor). Comprueba que, al modificar el nombre de un autor, el cambio se ve desde todos sus libros, y que si añades un elemento a una `list` normal sin marcar `_p_changed`, el cambio se pierde tras reabrir la base de datos.

Ejercicio 4.6 — Concurrencia en ZODB

Dificultad: ★★★

Abre dos conexiones a la misma base de datos ZODB (con `transaction.TransactionManager()` distintos), modifica el mismo objeto desde las dos y confirma ambas. Captura el `ConflictError` e implementa una función `con_reintentos(funcion, intentos=3)` que vuelva a ejecutar la transacción. Compara

este control optimista con el bloqueo pesimista (`SELECT ... FOR UPDATE`) del Tema 2.

Proyecto del tema: Catálogo de un museo

Se quiere gestionar el catálogo de obras de un museo, con datos muy estructurados: cada obra tiene autores (con sus datos biográficos), un conjunto variable de técnicas y materiales, dimensiones (alto, ancho, profundidad), un historial de ubicaciones (fecha, sala, motivo) y un historial de restauraciones.

Parte A — Objeto-relacional (PostgreSQL):

1. Diseña el esquema usando tipos compuestos (`dimensiones`, `ubicacion`), arrays (técnicas, historial de ubicaciones), enumerados (estado de conservación), herencia de tablas (`obra` → `pintura`, `escultura`, `fotografia`) y `JSONB` para metadatos variables.
2. Funciones del gestor: superficie o volumen de una obra; ubicación actual; obras de una sala.
3. Aplicación Python que registre los tipos con *dataclasses* y permita altas, movimientos de sala (transaccionales) y consultas.

Parte B — Orientado a objetos (ZODB):

1. Implementa el mismo modelo con clases `Persistent`, `BTrees` para los *extents* y un índice por autor.
2. Las mismas operaciones y consultas que en la parte A, con transacciones y tratamiento de conflictos.

Parte C — Informe: compara ambas soluciones (facilidad de modelado, consultas, rendimiento con 10 000 obras generadas aleatoriamente, posibilidad de acceder a los datos desde otras herramientas). Incluye pruebas con `pytest` de ambas partes.

TEMA 5

Bases de datos XML y documentales

En este tema aprenderás a…

- Conocer las bases de datos XML nativas, sus ventajas e inconvenientes y las estrategias de almacenamiento de XML.
- Instalar y configurar el gestor BaseX y conectarse a él desde Python.
- Crear y borrar colecciones (bases de datos) y añadir, modificar y eliminar documentos.
- Consultar y actualizar documentos con XPath, XQuery y XQuery Update.
- Utilizar índices e identificadores únicos, y conocer cómo se gestionan las transacciones.
- Trabajar con una base de datos documental JSON (MongoDB) desde Python.

Resultado de aprendizaje asociado: **RA5**.

Actualización del currículo. Las bases de datos XML nativas siguen existiendo y se usan en nichos concretos (edición y publicación, documentación técnica, sector jurídico, humanidades digitales), pero hoy, cuando se habla de **bases de datos documentales**, casi siempre se piensa en bases de datos de documentos **JSON** como MongoDB, que están entre los gestores más usados del mundo. Como el RA5 habla de "bases de datos documentales nativas", este tema mantiene todos los contenidos del currículo sobre XML (con BaseX y XQuery) y añade una segunda parte con **MongoDB**, que aplica exactamente los mismos conceptos (colecciones, documentos, consultas, índices, identificadores únicos, transacciones) a JSON. Proponemos repartir el tiempo del tema aproximadamente al 50 %.

5.1 Bases de datos XML nativas

5.1.1 Almacenar XML: estrategias

Cuando una aplicación maneja muchos documentos XML, hay varias formas de guardarlos:

En ficheros. Sencillo, pero sin consultas eficientes, sin transacciones ni control de concurrencia (Tema 1).

En una base de datos relacional con soporte XML (*XML-enabled*). Hay dos variantes:

- **Descomposición** (*shredding*): el documento se trocea y sus datos se reparten en tablas. Se pierde la estructura del documento (orden, comentarios, mezcla de texto y elementos) y reconstruirlo es costoso.
- **Columna de tipo XML**: el documento completo se guarda en una columna (tipo `XML` de PostgreSQL, `XMLType` de Oracle, `xml` de SQL Server) y el gestor ofrece funciones

XPath para consultarlo.

En una base de datos XML nativa (NXD, *Native XML Database*). La unidad lógica de almacenamiento es el **documento XML**; el gestor lo guarda conservando su modelo de árbol (normalmente en estructuras internas optimizadas, no como texto) y se consulta con lenguajes del W3C: **XPath** y **XQuery**.

5.1.2 Características de una base de datos XML nativa

- Define un modelo lógico para el documento XML (elementos, atributos, texto, orden) y almacena y recupera documentos según ese modelo.
- Organiza los documentos en **colecciones**, que hacen el papel de las tablas o de las carpetas de un sistema de ficheros. Los documentos de una colección no tienen por qué seguir el mismo esquema.
- Ofrece **índices** sobre la estructura y el contenido (textos, atributos, texto completo).
- Proporciona lenguajes de consulta y actualización estándar (XPath, XQuery, XQuery Update, XQuery Full Text).
- Ofrece API de acceso para los lenguajes de programación: REST, protocolos propios cliente-servidor y, históricamente, la API **XML:DB**.

5.1.3 Ventajas e inconvenientes

+ Los documentos se guardan y recuperan tal cual, sin pérdida de estructura ni código de conversión.
+ Ideales para información semiestructurada, con estructura variable o con mucho texto (contenido mixto).
+ Lenguajes de consulta potentes y estandarizados.
+ Validación con esquemas (XSD, DTD) y transformación (XSLT) integradas.
– Peor rendimiento que una base de datos relacional para datos muy estructurados y operaciones de agregación.
– XML es un formato verboso; ocupa más espacio.
– Menor madurez en transacciones, concurrencia y herramientas que los gestores relacionales.
– Comunidad y mercado reducidos.

5.1.4 Gestores comerciales y libres

Gestor	Licencia	Comentario
BaseX	Libre (BSD)	Ligero, rápido, muy buen soporte de XQuery 3.1 y XQuery Update. Servidor y cliente gráfico. Lo usaremos en el curso.
eXist-db	Libre (LGPL)	Muy popular; incluye un servidor web para construir aplicaciones completas con XQuery. API REST y XML:DB.
MarkLogic	Comercial	Base de datos multimodelo (XML, JSON, RDF) de uso empresarial.
Oracle XML DB	Comercial	Soporte XML integrado en Oracle Database (`XMLType`).
IBM Db2 pureXML	Comercial	Almacenamiento XML nativo dentro de Db2.
Sedna, Apache Xindice	Libres	Históricos; ya no se mantienen.

5.2 Instalación y configuración de BaseX

BaseX está escrito en Java. Se puede instalar descargando el paquete de `basex.org` (requiere Java 17 o superior) o con Docker (Apéndice A). Incluye varias herramientas:

BaseX GUI Interfaz gráfica para crear bases de datos, ejecutar consultas y visualizar los documentos. Muy útil para aprender XQuery.

basexserver El servidor. Por defecto escucha en el puerto **1984**.

basexclient Cliente de terminal.

basexhttp Servidor HTTP con API REST y aplicaciones web (RESTXQ), en el puerto 8080.

```
$ basexserver -S                    # arranca el servidor en segundo plano
$ basex -c "PASSWORD dam"           # (en local) establece la contraseña de admin
$ basexclient -U admin -P dam       # se conecta como cliente
> CREATE DB biblioteca
> LIST
> exit
$ basexserver stop
```

> **Nota.** Desde BaseX 10, el usuario `admin` no tiene contraseña por defecto y no se puede conectar al servidor hasta que se le asigna una. Las opciones de configuración (puerto, carpeta de datos, tamaño de caché...) se guardan en el fichero `.basex` de la carpeta de instalación.

Para conectarnos desde Python instalamos el cliente oficial:

```
$ pip install BaseXClient
```

5.3 Establecimiento y cierre de conexiones

El cliente se comunica con el servidor por TCP. La clase `Session` representa una conexión autenticada:

```
from BaseXClient import BaseXClient

session = BaseXClient.Session("localhost", 1984, "admin", "dam")
try:
    print(session.execute("INFO"))      # información del servidor
finally:
    session.close()                     # cerrar siempre la sesión
```

El cliente no implementa `with`, así que conviene envolverlo en un gestor de contexto propio. Lo reutilizaremos en el resto de ejemplos:

```
# basex_util.py
import os
from contextlib import contextmanager
from BaseXClient import BaseXClient

@contextmanager
def conectar(host: str = "localhost", port: int = 1984):
    session = BaseXClient.Session(
        host, port,
        os.getenv("BASEX_USER", "admin"),
        os.getenv("BASEX_PASSWORD", "dam"))
    try:
        yield session
    finally:
        session.close()
```

5.4 Colecciones y documentos

En BaseX, una **base de datos** es una colección de documentos (y opcionalmente de ficheros binarios). Dentro de ella, los documentos se identifican por una **ruta** (`libros/2026/nada.xml`), de modo que las carpetas actúan como subcolecciones.

5.4.1 Creación y borrado de colecciones

Las operaciones de administración se hacen con **comandos** mediante `session.execute()` o, desde XQuery, con las funciones del módulo `db:`.

`CREATE DB nombre [entrada]`	Crea una base de datos (vacía o a partir de un fichero, carpeta o URL) y la abre.
`OPEN nombre` / `CLOSE`	Abre / cierra la base de datos sobre la que actúan los comandos.
`LIST [nombre]`	Lista las bases de datos o los documentos de una.
`DROP DB nombre`	Borra una base de datos.
`INFO DB`	Información de la base de datos abierta.
`OPTIMIZE`	Reorganiza estructuras e índices.

```python
from basex_util import conectar

with conectar() as s:
    s.execute("CREATE DB biblioteca")              # vacía
    # s.execute("CREATE DB biblioteca /ruta/a/carpeta/xml")   # con documentos
    print(s.execute("LIST"))
    s.execute("DROP DB biblioteca")
```

Las mismas operaciones en XQuery: `db:create("biblioteca")`, `db:drop("biblioteca")`, `db:list()`, `db:exists("biblioteca")`.

5.4.2 Añadir, modificar y eliminar documentos

`session.add(ruta, xml)`	Añade un documento a la base de datos abierta (comando `ADD TO`).
`session.replace(ruta, xml)`	Sustituye el documento de esa ruta, o lo añade si no existe (comando `PUT`, llamado `REPLACE` en versiones anteriores a la 10).
`DELETE ruta`	Elimina el documento o todos los de una carpeta.
`RENAME ruta nueva`	Cambia la ruta de un documento.
`session.store(ruta, bytes)`	Guarda un fichero binario (imagen, PDF...).

```python
from pathlib import Path
from basex_util import conectar

with conectar() as s:
    s.execute("CREATE DB biblioteca")
    # Añadir desde una cadena
    s.add("libros/nada.xml",
          '<libro isbn="978-84-233-0000-1"><titulo>Nada</titulo>'
          '<autor>Carmen Laforet</autor><anio>1945</anio></libro>')
    # Añadir todos los XML de una carpeta
    for f in Path("datos").glob("*.xml"):
        s.add(f"libros/{f.name}", f.read_text(encoding="utf-8"))
    # Modificar = sustituir el documento completo
    s.replace("libros/nada.xml",
              '<libro isbn="978-84-233-0000-1"><titulo>Nada</titulo>'
              '<autor>Carmen Laforet</autor><anio>1945</anio>'
              '<premio>Nadal</premio></libro>')
    # Eliminar
    s.execute("DELETE libros/obsoleto.xml")
    print(s.execute("LIST biblioteca"))
```

Para modificar *partes* de un documento sin reemplazarlo entero se usa XQuery Update (apartado 5.5.3).

5.5 Lenguajes de consulta: XPath y XQuery

5.5.1 XPath

XPath (que ya usamos con `lxml` en el Tema 1) selecciona nodos de un documento mediante expresiones de camino. En BaseX, `collection("biblioteca")` devuelve todos los documentos de la base de datos y `doc("biblioteca/libros/nada.xml")` uno concreto.

```
collection("biblioteca")//libro/titulo
collection("biblioteca")//libro[anio < 1950]/titulo/text()
collection("biblioteca")//libro[@prestado = "si"]
count(collection("biblioteca")//libro)
collection("biblioteca")//libro[contains(autor, "Cela")]
```

5.5.2 XQuery

XQuery es a XML lo que SQL es a las bases de datos relacionales. Es un lenguaje funcional que incluye XPath y añade las expresiones **FLWOR**:

`for` recorre una secuencia.

`let` asigna un valor a una variable.

`where` filtra.

`order by` ordena.

`return` construye el resultado (puede crear XML nuevo).

```
(: Libros anteriores a 1960 ordenados por año, como una lista HTML :)
<ul>{
  for $l in collection("biblioteca")//libro
  let $anio := xs:integer($l/anio)
  where $anio < 1960
  order by $anio descending
  return <li>{ $l/titulo/text() } ({ $anio })</li>
}</ul>
```

```
(: Número de libros y precio medio por autor :)
for $l in collection("biblioteca")//libro
group by $autor := $l/autor
order by count($l) descending
return <autor nombre="{ $autor }" libros="{ count($l) }"
              media="{ round(avg($l/precio), 2) }"/>
```

```
(: Combinación (join) de dos colecciones :)
for $p in collection("prestamos")//prestamo
for $l in collection("biblioteca")//libro[@isbn = $p/@isbn]
return <prestamo socio="{ $p/socio }">{ $l/titulo }</prestamo>
```

```
(: Funciones definidas por el usuario :)
declare function local:precio-iva($precio as xs:decimal) as xs:decimal {
  round($precio * 1.04, 2)
};
for $l in collection("biblioteca")//libro
return <precio isbn="{ $l/@isbn }">{ local:precio-iva($l/precio) }</precio>
```

XQuery 3.1 también trabaja con mapas, arrays y JSON (`json-doc`, `parse-json`, serialización `method=json`).

5.5.3 Actualización: XQuery Update

```
(: Insertar un nodo :)
insert node <premio>Nadal</premio>
  into collection("biblioteca")//libro[titulo = "Nada"]

insert node <libro isbn="1"><titulo>Nuevo</titulo></libro>
  as last into doc("biblioteca/catalogo.xml")/biblioteca

(: Modificar el valor de un nodo o de un atributo :)
replace value of node collection("biblioteca")//libro[@isbn="1"]/titulo
  with "Título corregido"

for $p in collection("biblioteca")//libro/precio
return replace value of node $p with round($p * 1.1, 2)

(: Sustituir un nodo completo :)
replace node collection("biblioteca")//libro[@isbn="1"]/titulo
  with <titulo idioma="es">Otro título</titulo>

(: Renombrar y borrar :)
rename node collection("biblioteca")//libro[1]/anio as "publicacion"
delete node collection("biblioteca")//libro[@isbn="1"]
delete node collection("biblioteca")//libro/@prestado
```

Las expresiones de actualización no se aplican una a una: se acumulan en una **lista de actualizaciones pendientes** que se aplica **de forma atómica** al terminar la consulta.

5.6 Realización de consultas desde Python

Las consultas se crean con `session.query(texto)`, que devuelve un objeto `Query`:

`q.bind("$var", valor, tipo)`	Asigna un valor a una variable externa de la consulta (equivalente a los parámetros de SQL).
`q.execute()`	Ejecuta la consulta y devuelve el resultado completo como cadena.
`q.iter()`	Ejecuta la consulta y devuelve un iterador de pares `(tipo, valor)`, un elemento del resultado cada vez.
`q.info()`	Información de la ejecución (tiempos, plan, índices usados).
`q.close()`	Libera la consulta en el servidor.

```
import xml.etree.ElementTree as ET
from basex_util import conectar

CONSULTA = """
declare variable $anio external;
for $l in collection("biblioteca")//libro
```

```
where xs:integer($l/anio) < $anio
order by $l/titulo
return $l
"""

with conectar() as s:
    q = s.query(CONSULTA)
    try:
        q.bind("$anio", "1960", "xs:integer")    # !`nunca concatenar valores!
        for tipo, item in q.iter():
            libro = ET.fromstring(item)           # cada item es un <libro>
            print(libro.get("isbn"), libro.findtext("titulo"))
    finally:
        q.close()

    # Consulta que devuelve un único valor
    q = s.query('count(collection("biblioteca")//libro)')
    total = int(q.execute())
    q.close()
```

Inyección XQuery. Igual que con SQL, no construyas consultas concatenando datos introducidos por el usuario: usa variables externas (`declare variable $x external;`) y `bind`.

Para transformar los resultados en objetos se puede usar ElementTree, como arriba, o la vinculación con `xsdata` del Tema 1 (`XmlParser().from_string(item, Libro)`).

5.6.1 Actualizaciones desde Python

```
def prestar(s, isbn: str) -> None:
    q = s.query("""
        declare variable $isbn external;
        let $l := collection("biblioteca")//libro[@isbn = $isbn]
        return if (empty($l)) then error(xs:QName("NOEXISTE"), "No existe " || $isbn)
               else replace value of node $l/@prestado with "si"
    """)
    try:
        q.bind("$isbn", isbn)
        q.execute()
    finally:
        q.close()

with conectar() as s:
    try:
        prestar(s, "978-84-206-3311-6")
    except OSError as e:                    # el cliente lanza IOError/OSError
        print("Error:", e)
```

5.7 Indexación

Sin índices, cada consulta recorre todos los nodos de la base de datos. BaseX mantiene varios índices que el optimizador usa automáticamente cuando una consulta puede aprovecharlos:

Estructurales (siempre)	Nombres de elementos y atributos y *resumen de caminos*: aceleran expresiones como `//libro/titulo`.
`TEXT`	Valores de los nodos de texto: `//libro[autor = "Cela"]`. Activo por defecto.
`ATTRIBUTE`	Valores de los atributos: `//libro[@isbn = "…"]`. Activo por defecto.
`TOKEN`	Palabras dentro de atributos (como `class="a b"`).
`FULLTEXT`	Búsqueda de texto completo con `contains text`. Hay que crearlo.

```python
with conectar() as s:
    s.execute("OPEN biblioteca")
    s.execute("CREATE INDEX FULLTEXT")
    s.execute("SET UPDINDEX true")   # mantener índices al actualizar (por defecto
                                     # se invalidan y se reconstruyen con OPTIMIZE)
    q = s.query('collection("biblioteca")//libro[titulo contains text "soledad"]')
    print(q.execute())
    print(q.info())                  # muestra si se ha usado el índice
    q.close()
```

5.8 Identificadores únicos

Una base de datos XML necesita identificar documentos y nodos de forma única:

- Cada **documento** se identifica por su **ruta** dentro de la base de datos (`biblioteca/libros/nada.xml`).
- Cada **nodo** tiene en BaseX un identificador interno estable, el `id`, que no cambia aunque se modifique el documento. La función `db:node-id` obtiene el identificador de un nodo y `db:get-id` recupera el nodo a partir de su identificador (en BaseX 10 o superior). También existe el valor `pre`, la posición en el documento, que sí cambia con las actualizaciones.
- A nivel de datos, lo habitual es que cada entidad tenga un atributo con un identificador de negocio (el ISBN) o generado (`random:uuid()`), declarado en el esquema con el tipo `xs:ID` para poder usar la función `id()`.

```
let $l := collection("biblioteca")//libro[titulo = "Nada"]
return db:node-id($l)            (: p. ej. 42 :)

db:get-id("biblioteca", 42)/titulo

insert node <libro id="{ random:uuid() }"><titulo>Nuevo</titulo></libro>
  into doc("biblioteca/catalogo.xml")/biblioteca
```

5.9 Gestión de transacciones

En BaseX **cada comando y cada consulta es una transacción**: se ejecuta de forma atómica y aislada. Las actualizaciones de XQuery Update se acumulan y se aplican todas a la vez al final; si se produce un error, no se aplica ninguna. Para que varias operaciones sean atómicas, se agrupan en una sola consulta XQuery:

```
(: Préstamo: se registran ambos cambios o ninguno :)
declare variable $isbn external;
declare variable $socio external;
let $l := collection("biblioteca")//libro[@isbn = $isbn]
return (
  if ($l/@prestado = "si") then error(xs:QName("PRESTADO"), "Ya prestado") else (),
  replace value of node $l/@prestado with "si",
  insert node <prestamo isbn="{ $isbn }" socio="{ $socio }"
                        fecha="{ current-date() }"/>
    into doc("prestamos/prestamos.xml")/prestamos
)
```

La concurrencia se controla con bloqueos de lectura y escritura sobre las bases de datos implicadas en cada transacción. No hay transacciones de varias sentencias controladas desde el cliente (`commit`/`rollback`) como en SQL.

5.10 Tratamiento de excepciones

- El cliente de Python lanza `IOError` (alias de `OSError`) tanto si falla la conexión como si el servidor devuelve un error en un comando o consulta. El mensaje incluye el código de error de XQuery.
- Los errores de XQuery tienen códigos estándar: `XPST0003` (error de sintaxis), `XPTY0004` (tipos incompatibles), `FODC0002` (documento no encontrado)..., además de los propios de BaseX para sus módulos.
- En XQuery se pueden lanzar errores con `error()` y capturarlos con `try {…} catch * {…}`.

```python
try:
    with conectar() as s:
        q = s.query("collection('noexiste')//libro")
        print(q.execute())
except ConnectionRefusedError:
    print("El servidor BaseX no está arrancado")
except OSError as e:
    mensaje = str(e)
    if "Access denied" in mensaje or "Access Denied" in mensaje:
        print("Usuario o contraseña incorrectos")
    else:
        print("Error de BaseX:", mensaje)
```

5.11 Bases de datos documentales JSON: MongoDB

5.11.1 Conceptos

MongoDB es una base de datos **NoSQL documental**: almacena **documentos** con estructura de JSON (internamente, BSON, un JSON binario con más tipos de datos) agrupados en **colecciones**, dentro de **bases de datos**. No hay esquema fijo: cada documento de una colección puede tener campos distintos.

Relacional	BD XML (BaseX)	MongoDB
Base de datos	Base de datos	Base de datos
Tabla	Colección / carpeta	Colección
Fila	Documento XML	Documento JSON/BSON
Columna	Elemento / atributo	Campo
Clave primaria	Ruta / atributo id	`_id`
SQL	XQuery	API de consultas / agregación

Ventajas e inconvenientes respecto a las bases de datos XML: JSON es más ligero y corresponde directamente con los diccionarios de Python; MongoDB escala horizontalmente (*sharding*) y tiene una comunidad enorme. En cambio, no tiene un lenguaje de consulta estándar como XQuery ni validación tan rica como XSD (aunque admite JSON Schema).

5.11.2 Conexión

```
$ pip install pymongo
```

```python
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure

cliente = MongoClient("mongodb://localhost:27017/",
                      serverSelectionTimeoutMS=3000)
try:
    cliente.admin.command("ping")      # comprueba la conexión
except ConnectionFailure as e:
    raise SystemExit(f"No se pudo conectar: {e}")

bd = cliente["biblioteca"]             # se crean al insertar el primer documento
libros = bd["libros"]
# ...
cliente.close()                        # MongoClient también admite with
```

5.11.3 Colecciones

```python
print(bd.list_collection_names())
bd.create_collection("socios")       # opcional: se crea sola al insertar
bd.drop_collection("socios")         # o bd.socios.drop()
cliente.drop_database("pruebas")
```

5.11.4 Añadir, modificar y eliminar documentos

```python
from datetime import datetime

r = libros.insert_one({
    "isbn": "978-84-376-0494-7",
    "titulo": "Cien años de soledad",
    "autor": {"nombre": "Gabriel García Márquez", "pais": "Colombia"},
```

```
    "anio": 1967,
    "generos": ["novela", "realismo mágico"],
    "prestado": False,
})
print(r.inserted_id)                    # ObjectId generado automáticamente

libros.insert_many([
    {"isbn": "978-84-206-3311-6", "titulo": "La colmena",
     "autor": {"nombre": "Camilo José Cela"}, "anio": 1951, "generos": ["novela"]},
    {"isbn": "978-84-233-0000-1", "titulo": "Nada",
     "autor": {"nombre": "Carmen Laforet"}, "anio": 1945},
])

# Modificar campos con operadores de actualización
libros.update_one({"isbn": "978-84-206-3311-6"},
                  {"$set": {"prestado": True, "ultimo_prestamo": datetime.now()}})
libros.update_many({"anio": {"$lt": 1950}}, {"$push": {"generos": "clásico"}})
libros.update_one({"isbn": "978-84-233-0000-1"}, {"$inc": {"visitas": 1}},
                  upsert=True)          # inserta si no existe

# Sustituir el documento completo
libros.replace_one({"isbn": "978-84-233-0000-1"},
                   {"isbn": "978-84-233-0000-1", "titulo": "Nada", "anio": 1945})

# Eliminar
libros.delete_one({"isbn": "978-84-233-0000-1"})
r = libros.delete_many({"prestado": {"$exists": False}})
print(r.deleted_count)
```

5.11.5 Consultas

```
from pymongo import ASCENDING, DESCENDING

libro = libros.find_one({"isbn": "978-84-376-0494-7"})    # dict o None

for l in libros.find({"anio": {"$gte": 1950, "$lt": 1970}}):
    print(l["titulo"])

# Proyección, ordenación, paginación
cursor = (libros.find({"generos": "novela"},          # el array contiene "novela"
                      {"_id": 0, "titulo": 1, "autor.nombre": 1})
          .sort("anio", DESCENDING).skip(0).limit(10))

# Operadores: $eq $ne $gt $gte $lt $lte $in $nin $and $or $not $exists $regex
libros.find({"$or": [{"prestado": True}, {"anio": {"$lt": 1950}}]})
libros.find({"autor.nombre": {"$regex": "^Ca", "$options": "i"}})
print(libros.count_documents({"prestado": True}))

# Agregación (equivale a GROUP BY)
pipeline = [
    {"$unwind": "$generos"},
    {"$group": {"_id": "$generos", "total": {"$sum": 1},
                "anio_medio": {"$avg": "$anio"}}},
    {"$sort": {"total": -1}},
]
for g in libros.aggregate(pipeline):
```

```
    print(g["_id"], g["total"], round(g["anio_medio"]))
```

5.11.6 Índices e identificadores únicos

Cada documento tiene un campo `_id` único (índice automático). Si no se indica, el cliente genera un `ObjectId`: un identificador de 12 bytes que incluye la fecha de creación y es único sin necesidad de coordinarse con el servidor.

```
from bson import ObjectId

doc = libros.find_one({"_id": ObjectId("665f1c2e9b1e8a3d4c2b1a00")})
print(ObjectId().generation_time)

libros.create_index([("isbn", ASCENDING)], unique=True)   # identificador de negocio
libros.create_index([("autor.nombre", ASCENDING), ("anio", DESCENDING)])
libros.create_index([("titulo", "text")])                 # texto completo
libros.find({"$text": {"$search": "soledad"}})
print(libros.index_information())
```

5.11.7 Transacciones

Las operaciones sobre **un documento** son siempre atómicas, lo que, gracias a los documentos anidados, cubre muchos casos. Para operaciones sobre varios documentos o colecciones, MongoDB admite transacciones ACID (requiere que el servidor funcione como *replica set*, aunque sea de un solo nodo):

```
with cliente.start_session() as sesion:
    with sesion.start_transaction():
        libros.update_one({"isbn": isbn, "prestado": False},
                          {"$set": {"prestado": True}}, session=sesion)
        bd.prestamos.insert_one({"isbn": isbn, "socio": socio,
                                 "fecha": datetime.now()}, session=sesion)
    # commit al salir del bloque; abort si hay excepción
```

5.11.8 Excepciones

```
from pymongo.errors import (DuplicateKeyError, ConnectionFailure,
                            ServerSelectionTimeoutError, OperationFailure, PyMongoError)
try:
    libros.insert_one({"isbn": "978-84-376-0494-7", "titulo": "Repetido"})
except DuplicateKeyError:
    print("Ya existe un libro con ese ISBN")
except ServerSelectionTimeoutError:
    print("No se encuentra el servidor")
except PyMongoError as e:              # base de todas las excepciones de pymongo
    print("Error de MongoDB:", e)
```

5.12 Ejercicios

Ejercicio 5.1 — Primeros pasos con BaseX

Dificultad: ★

Instala BaseX y, desde la interfaz gráfica, crea una base de datos a partir de la carpeta con varios documentos XML de libros. Escribe y prueba consultas XPath y XQuery para: títulos de todos los libros; libros de un autor; libros ordenados por precio; precio medio por autor; un listado HTML de los libros prestados. Guarda las consultas en ficheros `.xq`.

Ejercicio 5.2 — Administración desde Python

Dificultad: ★★

Escribe un programa de línea de comandos `basexadmin.py` con subórdenes: `crear <bd> [carpeta]`, `borrar <bd>`, `listar [bd]`, `anadir <bd> <fichero> [ruta]`, `sustituir <bd> <fichero> <ruta>`, `eliminar <bd> <ruta>` y `exportar <bd> <carpeta>` (guarda cada documento en un fichero). Gestiona todas las excepciones con mensajes claros.

Ejercicio 5.3 — XQuery con parámetros

Dificultad: ★★

Usando la base de datos de `https://www.w3schools.com/xml/books.xml` (o una similar), escribe funciones Python que ejecuten consultas parametrizadas con `bind` y devuelvan objetos (*dataclasses*): buscar por categoría; libros entre dos precios; autores con más de un libro. Comprueba con `q.info()` qué consultas usan índices.

Ejercicio 5.4 — XQuery Update

Dificultad: ★★

Implementa en Python, mediante XQuery Update, las operaciones: añadir un libro a un catálogo, cambiar su precio, añadir un elemento `<valoracion>` a todos los libros de un autor, renombrar el elemento `year` a `anio` en todos los libros y eliminar los libros sin precio. Demuestra que una consulta con varias actualizaciones que termina con `error()` no aplica ninguna.

Ejercicio 5.5 — MongoDB

Dificultad: ★★

Importa en MongoDB un conjunto de datos JSON abierto (por ejemplo, los establecimientos turísticos o los museos de Castilla y León del portal de datos abiertos). Crea los índices adecuados y escribe con `pymongo`: búsquedas por provincia y tipo; los diez municipios con más establecimientos (agregación); una búsqueda de texto completo por nombre; y una actualización masiva que normalice el nombre de las provincias.

Ejercicio 5.6 — XML vs JSON

Dificultad: ★★★

Carga los mismos 10 000 documentos (generados aleatoriamente) en BaseX como XML y en MongoDB como JSON. Implementa las mismas cinco consultas en XQuery y con la API de MongoDB y mide los tiempos con y sin índices. Redacta una comparación razonada de ambos enfoques (modelado, lenguaje de consulta, rendimiento, herramientas).

Proyecto del tema: Repositorio de recetas

Desarrolla una aplicación para gestionar un repositorio de recetas de cocina, con dos implementaciones intercambiables del almacenamiento: BaseX (XML) y MongoDB (JSON).

Cada receta tiene: identificador, título, autor, categoría, tiempo, dificultad, raciones, lista de ingredientes (nombre, cantidad, unidad), pasos ordenados (con texto que puede contener ingredientes marcados), etiquetas y valoraciones de usuarios (usuario, puntuación, comentario, fecha).

Requisitos:

1. Esquema XSD de las recetas; validación con `lxml` antes de guardar en BaseX.
2. Interfaz común `RepositorioRecetas` (clase abstracta) con dos implementaciones: `RepositorioBaseX` y `RepositorioMongo`.
3. Operaciones: crear y borrar la colección; importar recetas desde una carpeta de ficheros XML o JSON; alta, modificación (parcial: añadir valoración, cambiar tiempo) y baja de recetas; consultas por categoría, por ingrediente, por tiempo máximo, mejor valoradas (agregación) y búsqueda de texto completo.
4. Identificadores únicos generados (`uuid`) e índices adecuados en ambos gestores.
5. Operaciones que afectan a varios documentos implementadas de forma atómica en ambos gestores.
6. Exportación de una receta a HTML (XSLT, Tema 1) y de la lista de la compra de varias recetas a CSV.
7. Tratamiento de excepciones y pruebas con `pytest`.

TEMA 6

Programación de componentes de acceso a datos

En este tema aprenderás a…

- Explicar qué es un componente, sus características y las ventajas e inconvenientes de la programación orientada a componentes.
- Definir propiedades simples, indexadas, compartidas (*bound*) y restringidas (*constrained*).
- Programar componentes que generan eventos y permiten asociarles acciones.
- Usar la introspección y la reflexión de Python para construir editores de propiedades y cargar componentes dinámicamente.
- Hacer persistente el estado de un componente.
- Empaquetar, documentar, probar e integrar componentes de acceso a datos para ficheros, conectores, ORM, bases de datos de objetos y documentales.

Resultado de aprendizaje asociado: **RA6**.

Actualización del currículo. Los contenidos de este bloque en el currículo (editores de propiedades, propiedades compartidas y restringidas, introspección, persistencia del componente…) están redactados pensando en el modelo de componentes **JavaBeans**. Los conceptos siguen siendo válidos, pero en Python se implementan con otras herramientas: propiedades y descriptores, el patrón observador, el módulo `inspect` y los paquetes instalables con `pyproject.toml`. Este tema presenta cada concepto, indica su equivalente en JavaBeans y muestra cómo se hace en Python.

6.1 Concepto de componente

Un **componente** es una unidad de software **reutilizable**, **autocontenida** y con una **interfaz bien definida**, que puede combinarse con otros componentes para construir aplicaciones sin necesidad de conocer ni modificar su código interno.

Sus características principales son:

Encapsulación. Oculta su implementación; sólo se usa a través de su interfaz.

Interfaz definida. Expone **propiedades** (estado configurable), **métodos** (operaciones) y **eventos** (avisos de que algo ha ocurrido).

Reutilizable e independiente del contexto. No depende de la aplicación concreta en la que se use.

Configurable. Su comportamiento se adapta modificando sus propiedades, no su código.

Introspectable. Una herramienta puede descubrir en tiempo de ejecución qué propiedades, métodos y eventos tiene.

Persistente. Su configuración puede guardarse y restaurarse.

Desplegable. Se distribuye como una unidad (un `.jar` en Java, un paquete *wheel* en Python).

Un **componente de acceso a datos** es un componente que encapsula el almacenamiento y la recuperación de la información: la aplicación le pide "guarda este alumno" o "dame los libros prestados" sin saber si los datos están en un CSV, en PostgreSQL, en ZODB o en MongoDB.

6.2 Programación orientada a componentes

La programación orientada a componentes construye las aplicaciones **ensamblando** componentes existentes, del mismo modo que un equipo electrónico se monta con piezas estándar.

+ **Reutilización**: un componente probado se usa en muchas aplicaciones.
+ **Menor tiempo y coste de desarrollo** y mayor calidad, al partir de piezas probadas.
+ **Mantenimiento** más sencillo: se puede sustituir un componente por otra versión o implementación sin tocar el resto.
+ Permite el **desarrollo en paralelo** por equipos distintos y el uso de componentes de terceros.
– Diseñar componentes realmente reutilizables es más costoso que escribir código para un caso concreto.
– Dependencia de terceros: versiones, licencias, componentes abandonados.
– Posibles incompatibilidades entre versiones y sobrecarga por la generalidad del componente.
– Puede ser difícil adaptar un componente a necesidades que su diseño no previó.

6.2.1 De JavaBeans a Python

Concepto	JavaBeans	Python
Propiedad	Pareja `getX()` / `setX()`	`@property` o descriptor
Propiedad indexada	`getX(int)` / `setX(int, v)`	Objeto con `__getitem__` / `__setitem__`
Propiedad compartida	`PropertyChangeSupport`	Patrón observador (este tema)
Propiedad restringida	`VetoableChangeSupport`	Observador con veto mediante excepción
Eventos	Interfaces `XxxListener`	*Callbacks*, señales (Qt, `blinker`)
Introspección	`Introspector`, `BeanInfo`	`inspect`, `dataclasses.fields`, anotaciones
Editor de propiedades	`PropertyEditor`	Conversores texto ↔ valor según el tipo
Persistencia	`Serializable`, `XMLEncoder`	JSON a partir de la introspección, `pickle`
Empaquetado	Fichero `.jar` con manifiesto	Paquete *wheel* con `pyproject.toml`

6.3 Propiedades y atributos

Un **atributo** es un dato interno del objeto. Una **propiedad** es un elemento de la interfaz pública del componente que se lee y escribe como si fuera un atributo, pero cuyo acceso pasa por código del componente (validación, conversión, avisos...). En Python, gracias a `@property`, la diferencia es invisible para quien usa el componente.

6.3.1 Propiedades simples

```python
class ConfiguracionConexion:
    """Componente con propiedades simples validadas."""

    def __init__(self, servidor: str = "localhost", puerto: int = 5432) -> None:
        self.servidor = servidor
        self.puerto = puerto

    @property
    def puerto(self) -> int:
        return self._puerto

    @puerto.setter
    def puerto(self, valor: int) -> None:
        if not 1 <= valor <= 65535:
            raise ValueError("Puerto fuera de rango")
        self._puerto = valor

    @property
    def url(self) -> str:                # propiedad de sólo lectura (calculada)
        return f"{self.servidor}:{self.puerto}"
```

6.3.2 Descriptores: propiedades reutilizables

Si muchas propiedades necesitan la misma lógica, escribir un `@property` para cada una es repetitivo. Un **descriptor** es una clase que define cómo se lee y escribe un atributo, y se reutiliza en tantas propiedades como se quiera. (De hecho, `property` y los atributos de SQLAlchemy son descriptores.)

```
class Rango:
    """Descriptor: propiedad numérica con límites."""

    def __init__(self, minimo: float, maximo: float) -> None:
        self.minimo, self.maximo = minimo, maximo

    def __set_name__(self, propietario, nombre: str) -> None:
        self.nombre = nombre                # nombre de la propiedad
        self.privado = "_" + nombre

    def __get__(self, obj, tipo=None):
        if obj is None:                     # acceso desde la clase
            return self
        return getattr(obj, self.privado)

    def __set__(self, obj, valor) -> None:
        if not self.minimo <= valor <= self.maximo:
            raise ValueError(f"{self.nombre} debe estar entre "
                             f"{self.minimo} y {self.maximo}")
        setattr(obj, self.privado, valor)


class ConfiguracionPool:
    tamano = Rango(1, 100)
    timeout = Rango(0.1, 60)

    def __init__(self, tamano: int = 5, timeout: float = 30) -> None:
        self.tamano = tamano
        self.timeout = timeout
```

6.3.3 Propiedades indexadas

Una propiedad indexada representa una colección de valores a la que se accede por posición. En Python se implementa devolviendo un objeto que se comporta como una secuencia y que controla el acceso a sus elementos:

```
from collections.abc import MutableSequence

class ListaValidada(MutableSequence):
    """Secuencia que valida cada elemento al asignarlo."""

    def __init__(self, validador, valores=()) -> None:
        self._validar = validador
        self._datos: list = []
        for v in valores:
            self.append(v)

    def __getitem__(self, i):            return self._datos[i]
    def __len__(self) -> int:            return len(self._datos)
```

```
    def __delitem__(self, i) -> None:    del self._datos[i]

    def __setitem__(self, i, valor) -> None:
        self._validar(valor)
        self._datos[i] = valor

    def insert(self, i, valor) -> None:
        self._validar(valor)
        self._datos.insert(i, valor)

    def __repr__(self) -> str:
        return repr(self._datos)


def validar_servidor(s: str) -> None:
    if not s or " " in s:
        raise ValueError(f"Nombre de servidor no válido: {s!r}")

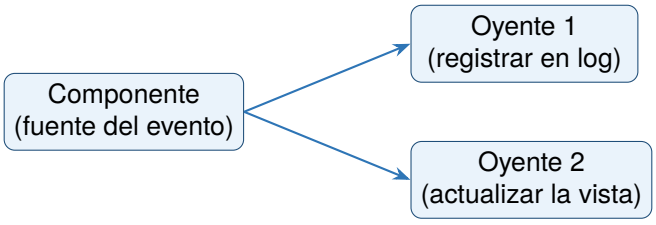
class ClusterBD:
    def __init__(self) -> None:
        self._servidores = ListaValidada(validar_servidor)

    @property
    def servidores(self) -> ListaValidada:      # propiedad indexada
        return self._servidores

c = ClusterBD()
c.servidores.append("bd1.instituto.local")
c.servidores[0] = "bd-principal"                 # validado
print(c.servidores[0], len(c.servidores))
```

6.4 Eventos y asociación de acciones a eventos

Un **evento** es un aviso que emite un componente cuando ocurre algo relevante (se ha guardado un registro, ha fallado la conexión, ha cambiado una propiedad). Otros objetos, los **oyentes** (*listeners*), se registran en el componente para que se ejecute una acción cuando se produzca el evento. Es el **patrón observador**, y permite que el componente avise a la aplicación sin conocerla.



Una implementación sencilla y reutilizable de un evento en Python:

```
from collections.abc import Callable
from typing import Any

class Evento:
    """Lista de oyentes a los que se avisa al emitir el evento."""
```

```
    def __init__(self) -> None:
        self._oyentes: list[Callable[..., Any]] = []

    def conectar(self, oyente: Callable[..., Any]) -> Callable[..., Any]:
        self._oyentes.append(oyente)
        return oyente                    # permite usarlo como decorador

    def desconectar(self, oyente: Callable[..., Any]) -> None:
        self._oyentes.remove(oyente)

    def emitir(self, *args, **kwargs) -> None:
        for oyente in list(self._oyentes):
            oyente(*args, **kwargs)
```

```
class RepositorioLibros:
    def __init__(self) -> None:
        self.libro_guardado = Evento()    # eventos públicos del componente
        self.libro_borrado = Evento()
        self._libros: dict[str, dict] = {}

    def guardar(self, libro: dict) -> None:
        self._libros[libro["isbn"]] = libro
        self.libro_guardado.emitir(libro)

    def borrar(self, isbn: str) -> None:
        libro = self._libros.pop(isbn)
        self.libro_borrado.emitir(libro)

# Asociación de acciones a eventos (en la aplicación, no en el componente)
repo = RepositorioLibros()
repo.libro_guardado.conectar(lambda l: print("Guardado:", l["titulo"]))

@repo.libro_borrado.conectar
def auditar(libro: dict) -> None:
    with open("auditoria.log", "a", encoding="utf-8") as f:
        f.write(f"BORRADO {libro['isbn']}\n")

repo.guardar({"isbn": "1", "titulo": "Nada"})
```

Nota. Las bibliotecas de interfaz gráfica tienen su propio sistema de eventos (señales y *slots* en Qt/PySide, `bind` en Tkinter). Para componentes no visuales existe la biblioteca `blinker`, que implementa señales con nombre; también la usan Flask y otros *frameworks*.

6.5 Propiedades compartidas y restringidas

Propiedad compartida (*bound property*). Cuando cambia su valor, el componente **notifica** el cambio a los oyentes registrados, indicando el nombre de la propiedad, el valor anterior y el nuevo. Permite, por ejemplo, que varios objetos compartan y sincronicen un mismo dato.

Propiedad restringida (*constrained property*). Antes de cambiar, el componente **consulta** a los oyentes, y cualquiera de ellos puede **vetar** el cambio lanzando una excepción. Si se veta, la propiedad conserva su valor.

El siguiente descriptor implementa ambas cosas y lo reutilizaremos en los componentes de acceso a datos:

```python
# componentes/propiedades.py
from dataclasses import dataclass
from typing import Any
from collections.abc import Callable

@dataclass(frozen=True)
class CambioPropiedad:
    fuente: Any
    nombre: str
    anterior: Any
    nuevo: Any

class CambioVetado(Exception):
    """Un oyente ha rechazado el cambio de una propiedad restringida."""

class SoportePropiedades:
    """Clase base (mixin) que gestiona los oyentes de cambios."""

    def __init__(self) -> None:
        self._oyentes_cambio: list[Callable[[CambioPropiedad], None]] = []
        self._oyentes_veto: list[Callable[[CambioPropiedad], None]] = []

    def al_cambiar(self, oyente: Callable[[CambioPropiedad], None]) -> None:
        self._oyentes_cambio.append(oyente)

    def al_intentar_cambiar(self, oyente: Callable[[CambioPropiedad], None]) -> None:
        self._oyentes_veto.append(oyente)


class Propiedad:
    """Descriptor de propiedad simple, compartida y/o restringida."""

    def __init__(self, defecto: Any = None, *, compartida: bool = True,
                 restringida: bool = False, tipo: type | None = None) -> None:
        self.defecto = defecto
        self.compartida = compartida
        self.restringida = restringida
        self.tipo = tipo

    def __set_name__(self, propietario, nombre: str) -> None:
        self.nombre = nombre
        self.privado = "_" + nombre

    def __get__(self, obj, tipo=None):
        if obj is None:
            return self
        return getattr(obj, self.privado, self.defecto)

    def __set__(self, obj: SoportePropiedades, valor: Any) -> None:
        if self.tipo is not None and not isinstance(valor, self.tipo):
            raise TypeError(f"{self.nombre} debe ser de tipo {self.tipo.__name__}")
        anterior = self.__get__(obj)
        if anterior == valor:
            return
        cambio = CambioPropiedad(obj, self.nombre, anterior, valor)
```

```
        if self.restringida:
            for oyente in obj._oyentes_veto:
                oyente(cambio)                  # puede lanzar CambioVetado
        setattr(obj, self.privado, valor)
        if self.compartida:
            for oyente in obj._oyentes_cambio:
                oyente(cambio)
```

```
from componentes.propiedades import (SoportePropiedades, Propiedad,
                                     CambioPropiedad, CambioVetado)

class FuenteDatos(SoportePropiedades):
    ruta = Propiedad("datos.db", restringida=True, tipo=str)
    solo_lectura = Propiedad(False, tipo=bool)

fuente = FuenteDatos()
fuente.al_cambiar(lambda c: print(f"{c.nombre}: {c.anterior!r} -> {c.nuevo!r}"))

def no_permitir_temporales(c: CambioPropiedad) -> None:
    if c.nombre == "ruta" and c.nuevo.startswith("/tmp"):
        raise CambioVetado("No se permiten bases de datos en /tmp")

fuente.al_intentar_cambiar(no_permitir_temporales)
fuente.ruta = "instituto.db"            # ruta: 'datos.db' -> 'instituto.db'
try:
    fuente.ruta = "/tmp/x.db"
except CambioVetado as e:
    print("Vetado:", e, "| valor actual:", fuente.ruta)
```

6.6 Introspección y reflexión

Introspección es la capacidad de un programa de examinar en tiempo de ejecución la estructura de los objetos: su tipo, sus atributos, sus métodos y sus firmas. **Reflexión** va un paso más allá: permite además *actuar* a partir de esa información (leer o modificar atributos por su nombre, invocar métodos, crear objetos de clases cuyo nombre sólo se conoce al ejecutar).

Las herramientas de desarrollo de componentes usan la introspección para mostrar las propiedades de un componente y permitir editarlas sin conocerlo de antemano. Python es especialmente rico en este aspecto:

`type(obj)`, `isinstance(obj, C)`	Tipo de un objeto.
`dir(obj)`, `vars(obj)`	Nombres de sus atributos; diccionario de atributos de instancia.
`getattr`, `setattr`, `hasattr`, `delattr`	Acceso a atributos por nombre (reflexión).
`callable(x)`	¿Se puede invocar?
`inspect.getmembers(obj, pred)`	Miembros que cumplen un predicado (métodos, funciones, propiedades...).
`inspect.signature(f)`	Parámetros de una función o método.
`inspect.getdoc(obj)`	Documentación.
`typing.get_type_hints(C)`	Anotaciones de tipo resueltas.
`dataclasses.fields(C)`	Campos de una *dataclass*.
`importlib.import_module(n)`	Importar un módulo cuyo nombre se conoce en ejecución.

```python
import inspect

def describir_componente(clase: type) -> None:
    """Muestra propiedades, métodos públicos y eventos de un componente."""
    print(f"Componente {clase.__name__}: {inspect.getdoc(clase) or ''}")
    for nombre, valor in vars(clase).items():
        if isinstance(valor, (Propiedad, property)):
            extra = ""
            if isinstance(valor, Propiedad):
                extra = (" [compartida]" * valor.compartida
                         + " [restringida]" * valor.restringida)
            print(f"  propiedad {nombre}{extra}")
    for nombre, metodo in inspect.getmembers(clase, inspect.isfunction):
        if not nombre.startswith("_"):
            print(f"  método    {nombre}{inspect.signature(metodo)}")
    obj = clase()
    for nombre, valor in vars(obj).items():
        if isinstance(valor, Evento):
            print(f"  evento    {nombre}")
```

6.6.1 Carga dinámica de componentes

Con la reflexión, la aplicación puede elegir el componente a usar a partir de la configuración, sin importarlo explícitamente en el código:

```python
import importlib

def crear_componente(ruta_clase: str, **propiedades):
    """crear_componente("almacenes.sqlite:AlmacenSQLite", ruta="x.db")"""
    nombre_modulo, nombre_clase = ruta_clase.split(":")
    modulo = importlib.import_module(nombre_modulo)
    clase = getattr(modulo, nombre_clase)
    componente = clase()
    for nombre, valor in propiedades.items():
        if not hasattr(clase, nombre):
            raise AttributeError(f"{nombre_clase} no tiene la propiedad {nombre}")
        setattr(componente, nombre, valor)
    return componente
```

6.7 Editores de propiedades

En JavaBeans, un **editor de propiedades** es una clase que sabe mostrar y modificar el valor de una propiedad de un tipo determinado (convertirlo a texto y desde texto, o mostrar un control gráfico), y que usan las herramientas de diseño para construir la "hoja de propiedades" de un componente.

En Python podemos construir un editor de propiedades genérico combinando introspección y conversores por tipo:

```
from pathlib import Path

# Un "editor" por tipo: convierte texto -> valor
EDITORES = {
    bool: lambda t: t.strip().lower() in ("s", "si", "sí", "true", "1"),
    int: int,
    float: float,
    str: str,
    Path: Path,
}

def editar_propiedades(componente) -> None:
    """Hoja de propiedades de consola para cualquier componente."""
    clase = type(componente)
    for nombre, desc in vars(clase).items():
        if not isinstance(desc, Propiedad):
            continue
        actual = getattr(componente, nombre)
        texto = input(f"{nombre} [{actual}]: ")
        if not texto:
            continue
        editor = EDITORES.get(desc.tipo or type(actual), str)
        try:
            setattr(componente, nombre, editor(texto))
        except (ValueError, TypeError, CambioVetado) as e:
            print(f"  Valor no aceptado: {e}")
```

> **Nota.** La misma idea, con una biblioteca gráfica (Tkinter, PySide), permite generar automáticamente un formulario de configuración para cualquier componente: un campo de texto para `str`, una casilla para `bool`, un selector numérico para `int`...

6.8 Persistencia del componente

La **persistencia del componente** es la capacidad de guardar su estado (sus propiedades configuradas) y restaurarlo más tarde, por ejemplo al reiniciar la aplicación. No hay que confundirla con los datos que el componente gestiona: aquí se guarda su *configuración*.

Con la introspección se pueden guardar las propiedades de cualquier componente en JSON:

```python
import json
from pathlib import Path

def guardar_estado(componente, fichero: Path) -> None:
    clase = type(componente)
    estado = {
        "clase": f"{clase.__module__}:{clase.__name__}",
        "propiedades": {n: getattr(componente, n) for n, d in vars(clase).items()
                        if isinstance(d, Propiedad)},
    }
    fichero.write_text(json.dumps(estado, indent=2, default=str), encoding="utf-8")

def cargar_estado(fichero: Path):
    estado = json.loads(fichero.read_text(encoding="utf-8"))
    return crear_componente(estado["clase"], **estado["propiedades"])
```

Si se usa `pickle`, los métodos especiales `__getstate__` y `__setstate__` permiten decidir qué se guarda; por ejemplo, para excluir una conexión abierta, que no se puede serializar:

```python
class AlmacenSQLite(SoportePropiedades):
    NO_PERSISTENTES = ("_conexion", "_oyentes_cambio", "_oyentes_veto",
                       "guardado", "borrado", "error")

    def __getstate__(self) -> dict:
        # ni la conexión ni los oyentes (funciones) se pueden serializar
        return {k: v for k, v in self.__dict__.items()
                if k not in self.NO_PERSISTENTES}

    def __setstate__(self, estado: dict) -> None:
        self.__init__()                     # recrea conexión (cerrada) y eventos
        self.__dict__.update(estado)        # restaura las propiedades
```

6.9 Componentes de acceso a datos

Reunimos todo lo anterior en un componente de acceso a datos. Primero se define la **interfaz** que deben cumplir todas las implementaciones. En Python se puede expresar con una clase abstracta (`abc`) o con un `Protocol` (tipado estructural):

```python
# componentes/interfaz.py
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class Alumno:
    dni: str
    nombre: str
    apellidos: str
    email: str | None = None

class AlmacenAlumnos(ABC):
    """Interfaz común de todos los componentes de acceso a alumnos."""

    @abstractmethod
    def abrir(self) -> None: ...
    @abstractmethod
```

```
    def cerrar(self) -> None: ...
    @abstractmethod
    def guardar(self, alumno: Alumno) -> None: ...
    @abstractmethod
    def buscar(self, dni: str) -> Alumno | None: ...
    @abstractmethod
    def listar(self) -> list[Alumno]: ...
    @abstractmethod
    def borrar(self, dni: str) -> bool: ...

    # Todos los componentes se pueden usar con "with"
    def __enter__(self):
        self.abrir()
        return self

    def __exit__(self, *exc) -> None:
        self.cerrar()
```

Y una implementación concreta con SQLite (Tema 2), con propiedades, eventos y validación:

```
# componentes/sqlite.py
import sqlite3
from componentes.interfaz import AlmacenAlumnos, Alumno
from componentes.propiedades import SoportePropiedades, Propiedad
from componentes.eventos import Evento

class ErrorAlmacen(Exception):
    """Error de cualquier componente de almacenamiento."""

class AlmacenSQLite(SoportePropiedades, AlmacenAlumnos):
    """Componente de acceso a alumnos almacenados en SQLite."""

    ruta = Propiedad("alumnos.db", restringida=True, tipo=str)
    solo_lectura = Propiedad(False, tipo=bool)

    def __init__(self) -> None:
        SoportePropiedades.__init__(self)
        self._conexion: sqlite3.Connection | None = None
        self.guardado = Evento()            # eventos del componente
        self.borrado = Evento()
        self.error = Evento()
        # no se puede cambiar la ruta con la base de datos abierta
        self.al_intentar_cambiar(self._vetar_si_abierta)

    def _vetar_si_abierta(self, cambio) -> None:
        if cambio.nombre == "ruta" and self._conexion is not None:
            from componentes.propiedades import CambioVetado
            raise CambioVetado("Cierra el almacén antes de cambiar la ruta")

    def abrir(self) -> None:
        try:
            self._conexion = sqlite3.connect(self.ruta)
            self._conexion.row_factory = sqlite3.Row
            self._conexion.execute(
                "CREATE TABLE IF NOT EXISTS alumno (dni TEXT PRIMARY KEY, "
                "nombre TEXT NOT NULL, apellidos TEXT NOT NULL, email TEXT)")
        except sqlite3.Error as e:
```

```
            self.error.emitir(e)
            raise ErrorAlmacen(f"No se pudo abrir {self.ruta}: {e}") from e

    def cerrar(self) -> None:
        if self._conexion is not None:
            self._conexion.close()
            self._conexion = None

    def _con(self) -> sqlite3.Connection:
        if self._conexion is None:
            raise ErrorAlmacen("El almacén no está abierto")
        return self._conexion

    def guardar(self, alumno: Alumno) -> None:
        if self.solo_lectura:
            raise ErrorAlmacen("Almacén en modo sólo lectura")
        with self._con() as c:              # transacción
            c.execute("INSERT INTO alumno VALUES (:dni, :nombre, :apellidos, :email) "
                      "ON CONFLICT(dni) DO UPDATE SET nombre=excluded.nombre, "
                      "apellidos=excluded.apellidos, email=excluded.email",
                      vars(alumno))
        self.guardado.emitir(alumno)

    def buscar(self, dni: str) -> Alumno | None:
        f = self._con().execute("SELECT * FROM alumno WHERE dni = ?",
                                (dni,)).fetchone()
        return Alumno(**dict(f)) if f else None

    def listar(self) -> list[Alumno]:
        return [Alumno(**dict(f)) for f in
                self._con().execute("SELECT * FROM alumno ORDER BY apellidos")]

    def borrar(self, dni: str) -> bool:
        if self.solo_lectura:
            raise ErrorAlmacen("Almacén en modo sólo lectura")
        with self._con() as c:
            borrado = c.execute("DELETE FROM alumno WHERE dni = ?",
                                (dni,)).rowcount == 1
        if borrado:
            self.borrado.emitir(dni)
        return borrado
```

La aplicación sólo depende de la interfaz; la implementación concreta se elige en la configuración:

```
# aplicacion.py
from componentes.interfaz import Alumno
from componentes.cargador import crear_componente   # función del apartado anterior

almacen = crear_componente("componentes.sqlite:AlmacenSQLite", ruta="instituto.db")
almacen.guardado.conectar(lambda a: print("Guardado", a.dni))

with almacen:
    almacen.guardar(Alumno("11111111H", "Ana", "García"))
    for a in almacen.listar():
        print(a)
```

Siguiendo el mismo esquema se programan componentes para el resto de tecnologías del módulo: `AlmacenCSV` y `AlmacenXML` (Tema 1), `AlmacenPostgreSQL` (Tema 2), `AlmacenSQLAlchemy` (Tema 3), `AlmacenZODB` (Tema 4) y `AlmacenBaseX` y `AlmacenMongo` (Tema 5). Una única batería de pruebas, parametrizada, comprueba que todos cumplen la interfaz:

```python
# tests/test_almacenes.py
import pytest
from componentes.interfaz import Alumno
from componentes.sqlite import AlmacenSQLite
from componentes.csv_ import AlmacenCSV    # otro componente con la misma interfaz

@pytest.fixture(params=[AlmacenSQLite, AlmacenCSV])
def almacen(request, tmp_path):
    a = request.param()
    a.ruta = str(tmp_path / "datos")
    with a:
        yield a

def test_guardar_y_buscar(almacen):
    almacen.guardar(Alumno("11111111H", "Ana", "García"))
    assert almacen.buscar("11111111H").nombre == "Ana"

def test_borrar_inexistente(almacen):
    assert almacen.borrar("00000000T") is False

def test_evento_guardado(almacen):
    recibidos = []
    almacen.guardado.conectar(recibidos.append)
    almacen.guardar(Alumno("22222222J", "Luis", "Pérez"))
    assert [a.dni for a in recibidos] == ["22222222J"]
```

6.10 Herramientas para el desarrollo de componentes no visuales

Entornos de desarrollo	PyCharm, Visual Studio Code (extensión de Python): depuración, refactorización, ejecución de pruebas, introspección de los componentes en el autocompletado.
Gestión de proyectos y dependencias	`venv` + `pip`, `uv`, `poetry`, `hatch`.
Construcción de paquetes	`build` con un *backend* (`setuptools`, `hatchling`).
Calidad	`ruff` (estilo y errores), `mypy` (comprobación de tipos estática).
Pruebas	`pytest`, `pytest-cov` (cobertura).
Documentación	*Docstrings* + `pdoc` o `Sphinx` para generar la documentación HTML.
Distribución	PyPI o un repositorio privado; instalación directa desde Git.

6.11 Empaquetado de componentes

Para que un componente se pueda reutilizar en otros proyectos hay que **empaquetarlo**: convertirlo en un paquete instalable con `pip`, con su nombre, versión, dependencias y documentación. Es el equivalente al `.jar` de Java.

6.11.1 Estructura del proyecto

```
componentes-ad/├──
 pyproject.toml├──
 README.md├──
 LICENSE├──
 src/|
   └── componentes_ad/|
       ├── __init__.py|
       ├── interfaz.py|
       ├── propiedades.py|
       ├── eventos.py|
       ├── sqlite.py|
       └── py.typed          (indica que el paquete incluye anotaciones de tipo)└──
 tests/
    └── test_almacenes.py
```

6.11.2 El fichero `pyproject.toml`

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "componentes-ad"
version = "0.1.0"
description = "Componentes de acceso a datos para el módulo de AD"
readme = "README.md"
requires-python = ">=3.12"
authors = [{ name = "2º DAM" }]
dependencies = []                       # el componente SQLite no necesita nada

[project.optional-dependencies]
postgres = ["psycopg[binary]>=3.2"]
orm = ["sqlalchemy>=2.0"]
mongo = ["pymongo>=4"]
xml = ["lxml>=5", "BaseXClient"]
dev = ["pytest", "ruff", "mypy", "pdoc"]

# Puntos de entrada: permiten descubrir los almacenes instalados (plugins)
[project.entry-points."componentes_ad.almacenes"]
sqlite = "componentes_ad.sqlite:AlmacenSQLite"
```

6.11.3 Construir, instalar y distribuir

```
$ pip install build
$ python -m build                    # genera dist/componentes_ad-0.1.0-py3-none-any.
    ↪ whl
$ pip install dist/componentes_ad-0.1.0-py3-none-any.whl
$ pip install -e ".[dev]"            # instalación editable para desarrollar
$ pip install "componentes-ad[postgres,orm]"   # con dependencias opcionales
$ pip install git+https://github.com/usuario/componentes-ad.git
```

Gracias a los puntos de entrada, una aplicación puede descubrir qué componentes de almacenamiento están instalados, incluso los que publiquen terceros en otros paquetes:

```python
from importlib.metadata import entry_points

for ep in entry_points(group="componentes_ad.almacenes"):
    print(ep.name, "->", ep.value)
    clase = ep.load()                    # importa la clase
```

6.11.4 Documentación

```
$ pdoc componentes_ad -o docs/       # genera la documentación HTML desde los
    ↪ docstrings
```

6.12 Ejercicios

Ejercicio 6.1 — Propiedades y descriptores

Dificultad: ★

Escribe un descriptor `Texto(min_longitud, max_longitud, patron=None)` que valide cadenas con una expresión regular opcional, y úsalo para definir un componente `ConfiguracionCorreo` (servidor, puerto con el descriptor `Rango`, usuario, remitente con patrón de email). Comprueba que se rechazan los valores no válidos.

Ejercicio 6.2 — Propiedades compartidas

Dificultad: ★★

Usando `SoportePropiedades` y `Propiedad`, crea un componente `Termostato` con las propiedades `temperatura` (compartida) y `consigna` (restringida: entre 15 y 28 grados, vetada fuera de ese rango por un oyente). Registra dos oyentes que muestren los cambios en consola y en un fichero de registro. Escribe pruebas con `pytest`.

Ejercicio 6.3 — Introspección

Dificultad: ★★

Escribe una función `documentar(clase) -> str` que genere, mediante introspección, un fichero Markdown con la documentación de un componente: descripción, propiedades (con su tipo, valor por defecto y si son compartidas o restringidas), métodos públicos (con su firma y *docstring*) y eventos. Aplícala a `AlmacenSQLite`.

Ejercicio 6.4 — Editor de propiedades gráfico

Dificultad: ★★★

Con Tkinter, crea una ventana que, a partir de cualquier componente que use `Propiedad`, construya automáticamente un formulario con un control adecuado al tipo de cada propiedad (`Entry`, `Checkbutton`, `Spinbox`). Al pulsar "Aplicar" se asignan los valores, mostrando en la ventana los errores y los vetos. Añade botones para guardar y cargar el estado del componente en JSON.

Ejercicio 6.5 — Empaquetado

Dificultad: ★★

Empaqueta el componente `AlmacenSQLite` con su interfaz, propiedades y eventos en un paquete `componentes-ad`. Genera el *wheel*, instálalo en un entorno virtual nuevo y crea un pequeño programa que lo use sin copiar ningún fichero del proyecto original. Genera su documentación con `pdoc`.

Proyecto del tema: Biblioteca de componentes de acceso a datos

Este es el proyecto final del módulo. Consiste en desarrollar, empaquetar e integrar una biblioteca de componentes de acceso a datos reutilizando el trabajo de los temas anteriores.

Requisitos:

1. Elige un dominio (puede ser el de uno de tus proyectos anteriores) y define su modelo con *dataclasses* y una interfaz abstracta `Almacen` con las operaciones de alta, baja, modificación, búsqueda por clave, listado con filtros y operaciones transaccionales.
2. Programa **al menos un componente por cada tecnología del módulo**:
   - ficheros (CSV, JSON o XML);
   - conector de base de datos relacional (SQLite y/o PostgreSQL con su conector);
   - mapeo objeto-relacional (SQLAlchemy);
   - base de datos objeto-relacional u orientada a objetos (PostgreSQL con tipos compuestos o ZODB);
   - base de datos documental nativa (BaseX y/o MongoDB).
3. Todos los componentes tienen propiedades configurables (con al menos una compartida y una restringida), emiten eventos (`guardado`, `borrado`, `error`) y se pueden usar con `with`.
4. Una función de fábrica crea el componente a partir de un fichero de configuración usando reflexión o puntos de entrada, y el estado de configuración del componente es persistente.
5. Una herramienta de migración que copie todos los datos de un componente a otro (por ejemplo, de CSV a PostgreSQL o de SQLite a MongoDB) usando sólo la interfaz común.
6. Batería de pruebas parametrizada con `pytest` que se ejecuta contra todos los componentes.

7. Empaquetado con `pyproject.toml`, dependencias opcionales por tecnología y documentación generada con `pdoc`.
8. Integración: una aplicación (de consola, gráfica o web) que use los componentes y permita cambiar de almacenamiento sólo modificando la configuración.

Entrega: repositorio Git con el código, el paquete construido, la documentación y una memoria breve que justifique el diseño y compare las tecnologías empleadas.

APÉNDICE A

Preparación del entorno de trabajo

A.1 Python y el editor

1. Instala **Python 3.12 o superior** desde `https://www.python.org` (en Windows, marca la opción *Add python.exe to PATH*). En Linux suele venir instalado; comprueba la versión con `python3 --version`.
2. Instala un editor: **Visual Studio Code** con la extensión *Python* de Microsoft, o **PyCharm Community**.
3. Para cada proyecto, crea un entorno virtual (`python -m venv .venv`) y selecciónalo como intérprete en el editor.

Paquetes que usaremos a lo largo del curso:

```
$ pip install lxml xsdata xmltodict pytest
$ pip install "psycopg[binary]" sqlalchemy
$ pip install ZODB BaseXClient pymongo
```

A.2 Servidores de bases de datos con Docker

Para no instalar cada gestor en el equipo, los levantaremos en contenedores con **Docker** (Docker Desktop en Windows y macOS; Docker Engine en Linux). Guarda este fichero como `compose.yaml` en una carpeta del curso:

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: dam
      POSTGRES_PASSWORD: dam
      POSTGRES_DB: instituto
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]

  basex:
    image: basex/basexhttp:latest
    ports: ["1984:1984", "8080:8080"]
    volumes: [basexdata:/srv/basex/data]

  mongo:
    image: mongo:7
    command: ["--replSet", "rs0", "--bind_ip_all"]
    ports: ["27017:27017"]
    volumes: [mongodata:/data/db]

volumes:
  pgdata:
```

```
  basexdata:
  mongodata:
```

```
$ docker compose up -d              # arranca los tres servidores
$ docker compose ps                 # comprueba que están en marcha
$ docker compose stop               # los detiene (los datos se conservan)
$ docker compose down -v            # los elimina, !`incluidos los datos!
```

A.2.1 PostgreSQL

Conexión: servidor `localhost`, puerto `5432`, base de datos `instituto`, usuario y contraseña `dam`. Para abrir una consola SQL:

```
$ docker compose exec postgres psql -U dam -d instituto
```

También puedes usar un cliente gráfico como **DBeaver** o **pgAdmin**.

A.2.2 BaseX

Antes de conectarte por primera vez hay que asignar una contraseña al usuario `admin`:

```
$ docker compose exec basex basex -c "ALTER PASSWORD admin dam"
```

Conexión desde Python: servidor `localhost`, puerto `1984`, usuario `admin`, contraseña `dam`. En `http://localhost:8080` está disponible la interfaz web (DBA). Para practicar XQuery es muy cómodo instalar también la **BaseX GUI** desde `https://basex.org`.

> **Nota.** Si la imagen de Docker cambia su forma de configurar la contraseña, consulta la documentación de BaseX (*Docker* y *User Management*). La alternativa es descargar BaseX (necesita Java 17+) y ejecutar `basexserver` directamente.

A.2.3 MongoDB

La primera vez hay que inicializar el *replica set* (necesario para las transacciones):

```
$ docker compose exec mongo mongosh --eval \
    "rs.initiate({_id: 'rs0', members: [{_id: 0, host: 'localhost:27017'}]})"
```

Conexión desde Python: `MongoClient("mongodb://localhost:27017/")`. Como cliente gráfico se recomienda **MongoDB Compass**.

> **Cuidado.** Esta configuración no tiene autenticación en MongoDB y usa contraseñas triviales en el resto: es adecuada sólo para el equipo de desarrollo, nunca para un servidor accesible desde Internet.

A.3 Referencias

- Documentación oficial de Python: `https://docs.python.org/es/3/`

- DB-API 2.0 (PEP 249): `https://peps.python.org/pep-0249/`
- psycopg 3: `https://www.psycopg.org/psycopg3/docs/`
- SQLAlchemy 2: `https://docs.sqlalchemy.org/en/20/`
- lxml: `https://lxml.de` xsdata: `https://xsdata.readthedocs.io`
- PostgreSQL: `https://www.postgresql.org/docs/`
- ZODB: `https://zodb.org`
- BaseX: `https://docs.basex.org` XQuery 3.1: `https://www.w3.org/TR/xquery-31/`
- MongoDB y PyMongo: `https://www.mongodb.com/docs/languages/python/pymongo-driver/current/`
- Empaquetado en Python: `https://packaging.python.org/es/latest/`